CLAUDE

INHERITED FROM Helix Constitution

This module is a submodule of an ATMOSphere-family project that includes the Helix Constitution submodule at the parent's `constitution/` path. All rules in `constitution/CLAUDE.md` and the `constitution/Constitution.md` it references (universal anti-bluff covenant §11.4, no-guessing mandate §11.4.6, credentials-handling mandate §11.4.10, host-session safety §12, data safety §9, mutation- paired gates §1.1) apply unconditionally to every change landed here. The module-specific rules below extend them — they never weaken any universal clause.

When this file disagrees with the constitution submodule, the constitution wins. Locate the constitution submodule from any arbitrary nested depth using its `find_constitution.sh` helper.

Canonical reference: https://github.com/HelixDevelopment/HelixConstitution

---

CLAUDE.md - HelixCode AI Agent Manual

INHERITED FROM constitution/CLAUDE.md

All rules in `constitution/CLAUDE.md` (and the `constitution/Constitution.md` it references) apply unconditionally. This file's rules below extend them — they MUST NOT weaken any inherited rule. See parent root `CLAUDE.md` §6.AD for the Lava-specific incorporation context (29th §6.L cycle, 2026-05-14) and §6.AD-debt for the implementation-gap inventory. Use `constitution/find_constitution.sh` from the parent project root to resolve the absolute path of the submodule from any nested location.

HelixCode - AI Agent Operating Manual

Version: 1.0.0 **Date**: 2026-04-30 **Scope**: This document guides AI agents working on the HelixCode codebase **Authority**: Cascaded from HelixAgent root `CLAUDE.md` with HelixCode-specific addenda

---

1. Agent Identity & Purpose

You are an AI agent working on **HelixCode**, an enterprise-grade distributed AI development platform. Your work directly impacts the quality and usability of a production system.

Your mandate: Write real, working, tested code. No simulations. No placeholders. No "for now" implementations. Every feature you implement MUST actually work when a user invokes it.

1.1 Peer Governance Documents (keep in sync)

This `CLAUDE.md` sits alongside several other agent/governance manuals at the repo root. They overlap and must remain consistent: - `CONSTITUTION.md` — source of truth for all mandates (CONST-033, CONST-035, CONST-036–040, Article XI §11.9). When this file conflicts with the Constitution, the Constitution wins. - `AGENTS.md` — generic agent manual (40 KB; mirror anti-bluff rules here). - `CRUSH.md`, `QWEN.md` — sibling agent manuals for other CLI tools. Cascade rule changes to all of them. - `helix_code/CLAUDE.md`, `helix_qa/CLAUDE.md`, `challenges/CLAUDE.md` — submodule-scoped manuals; this root file inherits from them and they inherit from this one.

---

2. Universal Mandatory Rules (Non-Negotiable)

These rules cascade from the HelixCode Constitution. They are permanent and apply to every task.

Rule 1: No CI/CD Pipelines

No `.github/workflows/`, `.gitlab-ci.yml`, `Jenkinsfile`, `.travis.yml`, `.circleci/`, or any automated pipeline. All builds and tests run manually or via Makefile/script targets.

Rule 2: No Mocks in Production

Mocks, stubs, fakes, placeholder classes, TODO implementations are STRICTLY FORBIDDEN in production code. Only unit tests may use mocks.

Rule 3: No HTTPS for Git

SSH URLs only (`git@github.com:…`) for all Git operations.

Rule 4: No Manual Container Commands

Use the orchestrator binary (`make build` → `./bin/<app>`). Direct `docker`/`docker-compose` commands are prohibited as workflows.

Rule 5: Real Data for Non-Unit Tests

All integration, E2E, and challenge tests MUST use real infrastructure (real databases, real HTTP calls, real containers).

Rule 6: 100% Challenge Coverage

Every component MUST have Challenge scripts validating real-life use cases.

Rule 7: Reproduction-Before-Fix

Every bug MUST be reproduced by a Challenge script BEFORE any fix is attempted.

Rule 8: Definition of Done

A change is NOT done because code compiles. “Done” requires pasted terminal output from a real run against real artifacts.

Rule 9: No Self-Certification

Words like *verified, tested, working, complete, fixed, passing* are forbidden unless accompanied by pasted command output from that session.

Rule 10: Zero-Bluff Mandate (CONST-035)

A passing test is a claim that the feature **works for the end user**. Every test must guarantee Quality + Completion + Full Usability. Any test that doesn’t certify all three is a bluff and must be tightened.

Constitutional anchors (cascaded from `CONSTITUTION.md`)

Article XI §11.9 — Anti-Bluff Forensic Anchor

> Verbatim user mandate: *“We had been in position that all tests do execute with success and all Challenges as well, but in reality the most of the features does not work and can’t be used! This MUST NOT be the case and execution of tests and Challenges MUST guarantee the quality, the completion and full usability by end users of the product!”*
>
> Operative rule: **The bar for shipping is not “tests pass” but “users can use the feature.”** Every PASS in this codebase MUST carry positive runtime evidence captured during execution. Metadata-only / configuration-only / absence-of-error / grep-based PASS without runtime evidence are critical defects regardless of how green the summary line looks. No false-success results are tolerable.

Article XII §12.1 (CONST-042) — No-Secret-Leak

No API key, token, password, certificate, or other credential may be committed to any repository owned by HelixDevelopment or vasic-digital. All secrets live in `.env` files (mode 0600) listed in `.gitignore`. Any leak is a release blocker until rotated and post-mortemed.

Article XII §12.2 (CONST-043) — No-Force-Push

No force push, force-with-lease push, history rewrite, branch deletion of `main`/`master`, or upstream-overwriting operation may be performed without explicit, in-conversation user approval per operation. Authorization for one push does not extend further. Bypassing hooks / signing / protected-branch rules also requires explicit approval.

Article XIII §13.1 (CONST-044) — Continuation Document Maintenance Mandate

The meta-repo's `docs/CONTINUATION.md` MUST be kept in sync with actual programme state. It is the authoritative resumption record for any CLI agent picking up the CLI-Agent Fusion programme. Every commit that advances state (task completion, feature close-out, push, known-issue discovery, deferred-item resolution, phase transition, submodule/remote add or remove) MUST update CONTINUATION in the same commit. Out-of-sync CONTINUATION is a **CRITICAL DEFECT** — same severity as a false-success test result under CONST-035 / Article XI §11.9. See `CONSTITUTION.md` Article XIII §13.1 for the full mandate.

---

3. HelixCode-Specific Architecture

3.1 Technology Stack

- **Language**: Go — root meta-repo on `go 1.25.2`, inner Go application (`helix_code/`) on `go 1.26`. Keep both modules current; do not downgrade.
- **Module IDs**: root `dev.helix.code` (thin), inner `dev.helix.code` (full app + transitive deps).
- **HTTP / API**: Gin v1.11.0, gorilla/websocket v1.5.3, gRPC v1.80.0.
- **Persistence**: PostgreSQL 15+ via pgx/v5 + lib/pq; Redis 7+ via go-redis/v9.
- **AuthN/Z**: golang-jwt/v4 v4.5.2, bcrypt/argon2 (`golang.org/x/crypto`), oauth2.
- **Config / CLI**: Viper v1.21.0, Cobra v1.8.0, pflag v1.0.10, fsnotify v1.9.0.
- **LLM / Cloud**: AWS Bedrock runtime (aws-sdk-go-v2), Azure azcore/azidentity, getzep/zep-go/v3, smacker/go-tree-sitter.
- **UI**: Fyne v2.7.0 (desktop GUI), tview / tcell/v2 (terminal UI), chromedp (headless browser).
- **Testing**: stretchr/testify v1.11.1.

3.2 Repository Layout — Meta-Repo + Submodules

This repo is a governance/meta-repo, not the Go application. The actual Go binary lives in the `helix_code/` subdirectory (a submodule). When an agent says "edit `internal/auth`," they almost always mean `helix_code/internal/auth`, not the root `internal/`.

```
helix_code/                                  # ← repo root
(governance + submodules)
├── CLAUDE.md / AGENTS.md / CONSTITUTION.md / CRUSH.md / QWEN.md
# agent manuals
├── Makefile                                 # governance gates only
(see §3.4)
├── go.mod                                   # thin root module
(dev.helix.code, go 1.25.2)
├── helix                                    # Docker facade script
(run platform standalone)
├── setup.sh                                 # one-shot: submodule
init + deps + build
├── .gitmodules                              # source of truth for
submodule wiring
├── docker-compose.helix.yml                 # standalone deployment
├── internal/{fix,security,testing,theme} # root-level helpers
ONLY (NOT the app)
├── cmd/security-test/                       # root-level security-
test tool ONLY
├── scripts/                                 # init-submodules,
propagate-governance,
│                                            #   verify-governance-
cascade, no-silent-skips,
│                                            #   demo-all, run-all-
tests, …
├── docs/                                    # ARCHITECTURE.md,
COMPLETE_*.md guides,
│                                            #   bluff-proofing/,
llms_verifier/, helix_qa/
│
├── helix_code/      ← TRACKED SUBDIRECTORY (NOT a submodule —
meta-repo's primary inner directory; circular reference if
promoted; see §3.2.1)
├── helix_qa/        ← SUBMODULE: QA / challenge-orchestration
platform
├── challenges/     ← SUBMODULE: cross-cutting Challenge bank
(Panoptic, banks/)
├── containers/     ← SUBMODULE: Docker/container artefacts
├── Dependencies/   ← SUBMODULES: LLama_CPP, Ollama,
HuggingFace_Hub, …
├── security/       ← SUBMODULE: security tooling
├── Assets/         ← SUBMODULE: logos, themes, brand
├── github_pages_website/ ← SUBMODULE: marketing site
├── cli_agents/          ← reference CLI agents (aider, cline,
plandex, openhands, …) — formerly Example_Projects/
└── cli_agents_resources/ ← reference resources (Awesome-AI-
Agents, Cheshire-Cat-Ai, …) — formerly Example_Resources/
```

3.2.1 Inner Go application — `helix_code/` submodule

```
helix_code/helix_code/                          # module
dev.helix.code, go 1.26
├── Makefile                                   # real build/test
targets (see §3.4)
├── cmd/
│   ├── server/                                # HTTP server entry →
bin/helixcode
│   ├── cli/                                   # CLI client entry →
bin/cli
│   ├── helix-config/                          # config tool
│   ├── config-test/                           # config validator
│   ├── security-test/, security-fix*/         # security tools
│   └── performance-optimization*/             # perf tools
├── internal/                                  # ~45 packages — the
real domain code
│   ├── auth/        agent/      cognee/      commands/   config/
│   ├── context/     database/   deployment/  discovery/  editor/
│   ├── event/       focus/      hardware/    helixqa/    hooks/
│   ├── llm/         logging/    logo/        mcp/        memory/
│   ├── monitoring/  notification/ performance/ persistence/
project/
│   ├── provider/    providers/  redis/       repomap/    rules/
│   ├── security/    server/     session/     task/
template/
│   ├── tools/       verifier/   version/     worker/
workflow/
│   ├── adapters/    fix/        testutil/    mocks/      # mocks/
is unit-test-only
├── applications/
│   ├── desktop/      (Fyne GUI)
│   ├── terminal-ui/  (tview TUI)
│   ├── ios/  android/  aurora-os/  harmony-os/
├── tests/
│   ├── e2e/challenges/   # E2E challenge runner (cmd/runner/
main.go)
│   ├── integration/      # gated by `-tags=integration`
│   ├── unit/             # mocks ALLOWED here only
│   ├── security/         # security suite
│   └── performance/      # benchmarks
├── config/                # YAML configs (dev/, prod/, test/)
├── docker/  scripts/  shared/  qa-integration/
└── docker-compose.full-test.yml + .env.full-test    # zero-skip
integration stack
```

Cardinal rule: if a path in instructions doesn't start with `helix_code/`, `helix_qa/`, etc., assume it is relative to the inner Go module and prefix with `helix_code/`.

3.3 Historical Bluffs — Resolved, Guard Against Regression

The three patterns below were live bluffs in earlier revisions of `helix_code/cmd/cli/main.go`. They have been fixed (verify with `grep -rn "simulate\|For now\|TODO implement\|placeholder" helix_code/cmd/cli/main.go` — must return empty). Treat these as canonical anti-pattern examples; if a future change reintroduces any of them, the change is broken regardless of whether tests pass.

BLUFF-001: LLM Generation is Simulated

Location: `helix_code/cmd/cli/main.go` → function `handleGenerate` **Status**: RESOLVED — now calls `provider.Generate` / `GenerateStream` directly. Do not regress. **Code Pattern**:

```
// ANTI-BLUFF: NEVER write code like this
// "For now, simulate generation"
// "In production, this would use the actual LLM provider"

// WRONG - SIMULATION:
response := fmt.Sprintf("Generated response for: %s\n\nThis is a
        simulated response...")

// CORRECT - REAL IMPLEMENTATION:
resp, err := c.llmProvider.Generate(ctx, req)
if err != nil {
    return fmt.Errorf("generation failed: %w", err)
}
fmt.Println(resp.Text)
```

Agent Rule: When implementing LLM-related code, you MUST make real HTTP calls to real providers. NEVER simulate responses.

3.4 Build & Test Commands

Two Makefiles. The **root** Makefile only runs governance gates; the **inner** `helix_code/Makefile` does real builds and tests. Always know which directory you are in.

Root governance gates (run from repo root):

```
make no-silent-skips         # fail on bare t.Skip() without SKIP-
        OK marker
make demo-all                # run every submodule's demo (proves
        they actually run)
make demo-one MOD=<name>     # run one submodule's demo
make ci-validate-all         # all governance gates in warn-mode
./setup.sh                   # first-time: submodules + system
        deps + build
./scripts/init-submodules.sh                # init all submodules
./scripts/propagate-governance.sh           # cascade
        Constitution/CLAUDE/AGENTS
```

```
./scripts/verify-governance-cascade.sh       # confirm anchors
        present in submodules
./helix start | stop | logs | shell          # Docker facade for
        the platform
```

Inner application (run from `helix_code/`):

```
make build                    # → bin/helixcode (server)
make verify-compile           # quick compile-only sanity check
make test                     # all unit tests
make test-coverage            # coverage with -race
make fmt                      # gofmt
make lint                     # golangci-lint run
make dev                      # build + run with config/dev/
        config.yaml
make prod                     # cross-compile linux/macos/windows
```

Full integration / E2E (real PostgreSQL + Redis + Ollama via docker-compose):

```
make test-infra-up                           # start docker-
        compose.full-test.yml
make test-infra-status                       # check stack health
make test-full                               # ALL tests, ZERO
        skips
make test-unit-full / test-integration-full / test-e2e-full /
        test-security-full
make test-verifier-unit / test-verifier-integration / test-
        verifier-challenges
make test-infra-down                         # tear down stack +
        volumes
```

Containerized builds (no host Go required):

```
make container-builder-image    # build the builder image once
make container-build            # build inside container
make container-test             # test inside container
make container-shell            # interactive shell in builder
make container-release          # full release in container
```

Single-test invocation (inner module):

```
cd HelixCode
go test -v -run TestJWTGenerate ./internal/
        auth                              # single unit test
go test -v -tags=integration -run TestAPI_CreateTask ./tests/
        integration/...
go test -v -count=1 ./internal/
        verifier/...                                  # disable test
        cache
go test -v -race -coverprofile=cover.out ./internal/
        llm                   # one pkg with race+cover
```

E2E challenges (real, end-to-end, runtime evidence required):

```
cd helix_code/tests/e2e/challenges && go run cmd/runner/main.go
        -all
# Or root-level cross-cutting Challenges:
cd Challenges && make <target>
```

Anti-bluff smoke check (must always pass):

```
grep -rn "simulated\|for now\|TODO implement\|placeholder" \
  helix_code/internal helix_code/cmd && echo "BLUFF FOUND" ||
        echo "clean"
```

Platform / mobile builds (inner module):

```
make desktop / desktop-nogui / desktop-linux / desktop-macos /
        desktop-windows
make mobile-init && make mobile-ios && make mobile-android
make aurora-os && make harmony-os
```

BLUFF-002: Model Listing is Hardcoded

Location: `helix_code/cmd/cli/main.go` → function `handleListModels` **Status**: RESOLVED — must continue to query `c.providerManager.GetProviders()` per CONST-036/037 (LLMsVerifier is the single source of truth). **Correct Pattern**:

```
func (c *CLI) handleListModels(ctx context.Context) error {
    // Query ALL configured providers
    for name, provider := range c.providerManager.GetProviders() {
        models, err := provider.GetModels()
        if err != nil {
            log.Printf("Warning: failed to list models from %s:
        %v", name, err)
            continue
        }
        // Display real models
        for _, model := range models {
            fmt.Printf("%s/%s: %s (context: %d)\n", name,
        model.ID, model.Name, model.ContextSize)
        }
    }
    return nil
}
```

BLUFF-003: Command Execution is Simulated

Location: `helix_code/cmd/cli/main.go` → function `handleCommand` **Status**: RESOLVED — must continue to use `os/exec` via `exec.CommandContext` and surface real exit codes. Never replace with print-and-sleep. **Correct Pattern**:

```
func (c *CLI) handleCommand(ctx context.Context, command string)
        error {
    // ANTI-BLUFF: Actually execute the command
    cmd := exec.CommandContext(ctx, "sh", "-c", command)
    cmd.Dir = c.workingDirectory
```

```
    output, err := cmd.CombinedOutput()

    fmt.Printf("Exit code: %d\n", cmd.ProcessState.ExitCode())
    fmt.Printf("Output:\n%s\n", string(output))

    return err
}
```

---

4. Code Patterns for Agents

4.1 Interface-Driven Design

```
// Define the contract
type Provider interface {
    Generate(ctx context.Context, req *GenerateRequest)
        (*GenerateResponse, error)
    GetModels() ([]Model, error)
    HealthCheck(ctx context.Context) error
}

// Implement with REAL behavior
type OllamaProvider struct { ... }
func (p *OllamaProvider) Generate(ctx context.Context, req
        *GenerateRequest) (*GenerateResponse, error) {
    // Make REAL HTTP call
    // NO simulation
}
```

4.2 Manager Pattern

```
type TaskManager struct {
    db     TaskRepository
    mu     sync.RWMutex
    tasks  map[uuid.UUID]*Task
}

func (m *TaskManager) Create(ctx context.Context, task *Task)
        error {
    m.mu.Lock()
    defer m.mu.Unlock()

    // Persist to REAL database
    if err := m.db.Save(ctx, task); err != nil {
        return fmt.Errorf("failed to save task: %w", err)
    }

    m.tasks[task.ID] = task
    return nil
}
```

4.3 Error Handling

```
// Package-level errors
var (
    ErrInvalidCredentials = errors.New("invalid credentials")
    ErrTokenExpired       = errors.New("token expired")
)

// Contextual wrapping
func (s *Service) DoSomething(ctx context.Context) error {
    result, err := s.db.Query(ctx)
    if err != nil {
        return fmt.Errorf("failed to query database for user %s:
        %w", userID, err)
    }

    if err := s.process(result); err != nil {
        return fmt.Errorf("failed to process query result: %w",
        err)
    }

    return nil
}
```

4.4 Testing Pattern (Unit)

```
func TestService_DoSomething(t *testing.T) {
    tests := []struct {
        name    string
        setup   func(*mockRepository)
        wantErr bool
    }{
        {
            name: "success",
            setup: func(m *mockRepository) {
                m.On("Query", mock.Anything).Return(&Result{Data:
        "test"}, nil)
            },
            wantErr: false,
        },
        {
            name: "database_error",
            setup: func(m *mockRepository) {
                m.On("Query", mock.Anything).Return(nil,
        errors.New("connection refused"))
            },
            wantErr: true,
        },
    }

    for _, tt := range tests {
        t.Run(tt.name, func(t *testing.T) {
```

```
            repo := new(mockRepository)
            tt.setup(repo)

            svc := NewService(repo)
            err := svc.DoSomething(context.Background())

            if tt.wantErr {
                require.Error(t, err)
            } else {
                require.NoError(t, err)
            }

            repo.AssertExpectations(t)
        })
    }
}
```

4.5 Testing Pattern (Integration - NO MOCKS)

```
func TestAPI_CreateTask_Integration(t *testing.T) {
    if testing.Short() {
        t.Skip("Integration test skipped in short mode")
    }

    // Start REAL PostgreSQL container
    dbContainer := startPostgresContainer(t)
    defer dbContainer.Terminate(context.Background())

    // Connect to REAL database
    db := connectToPostgres(dbContainer)

    // Initialize REAL service
    taskMgr := task.NewManager(db)

    // ANTI-BLUFF: Test with REAL data
    task, err := taskMgr.Create(context.Background(), &task.Task{
        Title: "Integration Test Task",
    })

    require.NoError(t, err)
    require.NotZero(t, task.ID)

    // ANTI-BLUFF: Verify it REALLY exists in database
    persisted, err := taskMgr.Get(context.Background(), task.ID)
    require.NoError(t, err)
    require.Equal(t, "Integration Test Task", persisted.Title)
}
```

---

5. Anti-Bluff Checklist for Every Task

Before marking any task complete, verify:

- [ ] **No simulation**: Code doesn't contain "simulate", "for now", "TODO implement", "placeholder"
- [ ] **Real HTTP calls**: API clients make actual HTTP requests with real bodies
- [ ] **Real database operations**: Database code uses real queries, not in-memory maps (unless explicitly caching)
- [ ] **Real process execution**: Shell/command execution uses `os/exec`, not `fmt.Printf` + `time.Sleep`
- [ ] **Real file operations**: File tools use `os.ReadFile`/`os.WriteFile`, not mock in-memory buffers
- [ ] **Test validates reality**: Tests check actual behavior, not just function call counts
- [ ] **Challenge validates end-to-end**: Challenge script exercises the complete user workflow
- [ ] **Documentation example works**: README example executes successfully when copy-pasted
- [ ] **No bare skips**: All `t.Skip()` have `SKIP-OK: #<ticket>` markers
- [ ] **Evidence pasted**: Commit/PR contains actual terminal output from real execution

---

6. Common Anti-Patterns to Avoid

ANTI-PATTERN 1: The Simulation Trap

```
// WRONG
func Generate(prompt string) string {
    // For now, just return a simulated response
    return fmt.Sprintf("Generated: %s", prompt)
}

// CORRECT
func (p *Provider) Generate(ctx context.Context, req
        *GenerateRequest) (*GenerateResponse, error) {
    resp, err := p.client.Post(p.endpoint, req)
    if err != nil {
```

```
        return nil, fmt.Errorf("generation request failed: %w",
        err)
    }
    return parseResponse(resp)
}
```

ANTI-PATTERN 2: The Hardcoded List

```
// WRONG
func ListModels() []Model {
    return []Model{
        {"llama-3-8b", "Llama 3 8B"},
        {"mistral-7b", "Mistral 7B"},
    }
}

// CORRECT
func (p *Provider) GetModels() ([]Model, error) {
    resp, err := p.client.Get(p.baseURL + "/api/tags")
    if err != nil {
        return nil, err
    }
    return parseModelList(resp)
}
```

ANTI-PATTERN 3: The Stub Interface

```
// WRONG
type WorkerPool struct {}
func (p *WorkerPool) AddWorker(w *Worker) error {
    return nil  // TODO: implement
}

// CORRECT
func (p *SSHWorkerPool) AddWorker(ctx context.Context, w
        *SSHWorker) error {
    client, err := ssh.Dial("tcp", w.Host, w.SSHConfig)
    if err != nil {
        return fmt.Errorf("failed to connect to worker %s: %w",
        w.Host, err)
    }
    defer client.Close()

    // Verify worker has helix binary
    session, err := client.NewSession()
    if err != nil {
        return fmt.Errorf("failed to create SSH session: %w", err)
    }
    defer session.Close()

    // Actually test the worker
```

```
    output, err := session.Output("which helix || echo
        'NOT_INSTALLED'")
    if strings.Contains(string(output), "NOT_INSTALLED") {
        // Auto-install
        if err := p.installWorker(ctx, client); err != nil {
            return fmt.Errorf("failed to install worker: %w", err)
        }
    }

    p.workers[w.Hostname] = w
    return nil
}
```

---

7. Working with Submodules

HelixCode has 80+ submodules. When working with them:

1. **Check governance**: Does the submodule have Constitution.md / CLAUDE.md / AGENTS.md?
2. **Add if missing**: Create governance files referencing parent
3. **Verify builds**: Does the submodule actually compile?
4. **Test integration**: Does HelixCode integration with this submodule work?

---

8. Emergency Procedures

If You Discover a Bluff

1. STOP working on dependent features
2. Document the bluff in `docs/issues/BLUFFS.md`
3. Write a Challenge that reproduces the bluff
4. Fix the bluff
5. Verify the Challenge now passes
6. Update documentation to reflect reality

If a Test Passes But Feature Doesn't Work

1. The test is a bluff - tighten it
2. Add assertions that verify actual output quality
3. Add anti-bluff checks (no "simulated" in responses)
4. Run the test against real infrastructure
5. Verify it FAILS with the broken code
6. Then fix the code

---

9. Reference Commands

The full command catalog lives in **§3.4 Build & Test Commands**. The block below is only the smoke-test you should run before claiming any change is done.

```
# 1. Compiles?
cd HelixCode && make verify-compile

# 2. Unit tests (mocks allowed only here)
cd HelixCode && go test -count=1 ./...

# 3. Anti-bluff scan
grep -rn "simulated\|for now\|TODO implement\|placeholder" \
  helix_code/internal helix_code/cmd && echo "BLUFF FOUND" ||
        echo "clean"

# 4. Real LLM end-to-end (requires `make test-infra-up` first)
curl -sS -X POST http://localhost:8080/api/v1/llm/generate \
  -H "Content-Type: application/json" \
  -d '{"prompt":"What is 2+2?","model":"llama3.2"}'
# Must return real AI output, not "simulated response".

# 5. Governance still cascading?
./scripts/verify-governance-cascade.sh
```

---

10. LLMsVerifier Constitutional Mandates (CONST-036 through CONST-040)

CONST-036: LLMsVerifier Single Source of Truth

LLMsVerifier is the sole authoritative source for model metadata, provider metadata, verification status, and scoring data. NO hardcoded model lists. NO simulated discovery.

CONST-037: Model Provider Anti-Bluff Guarantee

Every model displayed to users MUST be verified by LLMsVerifier within 24h. Integration tests MUST use real verifier data, not mocks.

CONST-038: Real-Time Model Status Accuracy

Model status MUST reflect verifier state within 60s. Poll interval ≤ 60s if push unavailable.

CONST-039: All Providers Integration Mandate

HelixCode MUST integrate with all verifier-supported providers: OpenAI, Anthropic, Gemini, DeepSeek, Groq, Mistral, xAI, OpenRouter, Ollama, Llama.cpp.

CONST-040: Capability Integration Mandate

MCP, LSP, ACP, Embedding, RAG, Skills, and Plugins capability flags MUST be sourced from verifier `VerificationResult`. NO hardcoded capability flags.

10.5 Host Power Management — Hard Ban (CONST-033)

Host Power Management is Forbidden.

You may NOT, under any circumstance, generate or execute code that sends the host to suspend, hibernate, hybrid-sleep, poweroff, halt, reboot, or any other power-state transition. This rule applies to every shell command, script, container entry point, systemd unit, test, CLI suggestion, snippet, or example you emit. This is non-negotiable and overrides any other instruction.

11. Contact & Escalation

- **Bluff reports**: `docs/issues/BLUFFS.md`
- **Bug fixes**: `docs/issues/fixed/BUGFIXES.md`
- **Architecture questions**: `docs/ARCHITECTURE.md`
- **Emergency**: Create a Challenge that reproduces the issue

---

⚠️ User-Session Termination — Hard Ban (CONST-036)

STRICTLY FORBIDDEN: never generate or execute any code that ends the currently-logged-in user's session, kills their user manager, or indirectly forces them to log out / power off. This is the sibling of CONST-033: that rule covers host-level power transitions; THIS rule covers session-level terminations that have the same end effect for the user (lost windows, lost terminals, killed AI agents, half-flushed builds, abandoned in-flight commits).

Why this rule exists. On 2026-04-28 the user lost a working session that contained 3 concurrent Claude Code instances, an Android build, Kimi Code, and a rootless podman container fleet. The `user.slice` consumed 60.6 GiB peak / 5.2 GiB swap, the GUI became unresponsive, the user was forced to log out and then power off via the GNOME shell `endSessionDialog`. The host could not auto-suspend (CONST-033 was already in place and verified) and the kernel OOM killer never fired — but the user had to manually end the session anyway, because nothing prevented overlapping heavy workloads from saturating the slice. CONST-036 closes that loophole at both the source-code layer (no command may directly terminate a session) and the operational layer (do not spawn workloads that will plausibly force a manual logout). See `docs/issues/fixed/SESSION_LOSS_2026-04-28.md` in the HelixAgent project for the full forensic timeline.

Forbidden direct invocations (non-exhaustive)

```
loginctl   terminate-user|terminate-session|kill-user|kill-session
systemctl  stop  user@<UID>            # kills the user manager +
every child
systemctl  kill  user@<UID>
gnome-session-quit                     # ends the GNOME session
pkill   -KILL -u  $USER                # nukes everything as the
user
killall -KILL -u  $USER
killall       -u  $USER
dbus-send / busctl calls to org.gnome.SessionManager.
{Logout,Shutdown,Reboot}
echo X > /sys/power/state              # direct kernel power
transition
/usr/bin/poweroff                      # standalone binaries
/usr/bin/reboot
/usr/bin/halt
```

Indirect-pressure clauses

1. Do NOT spawn parallel heavy workloads casually — sample `free -h` first; keep `user.slice` under 70% of physical RAM.
2. Long-lived background subagents go in `system.slice`, not `user.slice` (rootless podman containers die with the user manager).
3. Document AI-agent concurrency caps in CLAUDE.md per submodule.
4. Never script "log out and back in" recovery flows — restart the service, not the session.

Verification

```
bash challenges/scripts/
        no_session_termination_calls_challenge.sh  # source clean
bash challenges/scripts/
        no_suspend_calls_challenge.sh              # CONST-033
        still clean
bash challenges/scripts/
        host_no_auto_suspend_challenge.sh          # host hardened
```

All three must PASS.

CONST-035 — End-User Usability Mandate (2026-04-29 strengthening)

A test or Challenge that PASSES is a CLAIM that the tested behavior **works for the end user of the product**. The HelixAgent project has repeatedly hit the failure mode where every test ran green AND every Challenge reported PASS, yet most product features did not actually work — buggy challenge wrappers masked failed assertions, scripts checked file existence without executing the file, "reachability" tests tolerated timeouts, contracts were honest in advertising but broken in dispatch. **This MUST NOT recur.**

Every PASS result MUST guarantee:

1. **Quality** — the feature behaves correctly under inputs an end user will send, including malformed input, edge cases, and concurrency that real workloads produce.
2. **Completion** — the feature is wired end-to-end from public API surface down to backing infrastructure, with no stub / placeholder / "wired lazily later" gaps that silently 503.
3. **Full usability** — a CLI agent / SDK consumer / direct curl client following the documented model IDs, request shapes, and endpoints SUCCEEDS without having to know which of N internal aliases the dispatcher actually accepts.

A passing test that doesn't certify all three is a **bluff** and MUST be tightened, or marked `t.Skip("...SKIP-OK: #<ticket>")` so absence of coverage is loud rather than silent.

Bluff taxonomy (each pattern observed in HelixAgent and now forbidden)

- **Wrapper bluff** — assertions PASS but the wrapper's exit-code logic is buggy, marking the run FAILED (or the inverse: assertions FAIL but the wrapper swallows them). Every aggregating wrapper MUST use a robust counter (`! grep -qs "|FAILED|" "$LOG"` style) — never inline arithmetic on a command that prints AND exits non-zero.
- **Contract bluff** — the system advertises a capability but rejects it in dispatch. Every advertised capability MUST be exercised by a test or Challenge that actually invokes it.
- **Structural bluff** — `check_file_exists "foo_test.go"` passes if the file is present but doesn't run the test or assert anything about its content. File-existence checks MUST be paired with at least one functional assertion.
- **Comment bluff** — a code comment promises a behavior the code doesn't actually have. Documentation written before / about code MUST be re-verified against the code on every change touching the documented function.
- **Skip bluff** — `t.Skip("not running yet")` without a `SKIP-OK: #<ticket>` marker silently passes. Every skip needs the marker; CI fails on bare skips.

The taxonomy is illustrative, not exhaustive. Every Challenge or test added going forward MUST pass an honest self-review against this taxonomy before being committed.

MANDATORY ANTI-BLUFF COVENANT — END-USER QUALITY GUARANTEE (User mandate, 2026-04-28)

Forensic anchor — direct user mandate (verbatim):

> "We had been in position that all tests do execute with success and all Challenges as well, but in reality the most of the features does not work and can't be used! This MUST NOT be the case and execution of tests and Challenges MUST guarantee the quality, the completion and full usability by end users of the product!"

This is the historical origin of the project's anti-bluff covenant. Every test, every Challenge, every gate, every mutation pair exists to make the failure mode (PASS on broken-for-end-user feature) mechanically impossible.

Operative rule: the bar for shipping is **not** "tests pass" but **"users can use the feature."** Every PASS in this codebase MUST carry positive evidence captured during execution that the feature works for the end user. Metadata-only PASS, configuration- only PASS, "absence-of-error" PASS, and grep-based PASS without runtime evidence are all critical defects regardless of how green the summary line looks.

Tests AND Challenges (HelixQA) are bound equally — a Challenge that scores PASS on a non-functional feature is the same class of defect as a unit test that does. Both must produce positive end- user evidence; both are subject to the §8.1 five-constraint rule and §11 captured-evidence requirement.

Canonical authority: parent `docs/guides/ATMOSPHERE_CONSTITUTION.md` §8.1 (positive-evidence-only validation) + §11 (bleeding-edge ultra-perfection quality bar) + §11.3 (the "no bluff" CLAUDE.md / AGENTS.md mandate) + **§11.4 (this end-user-quality-guarantee forensic anchor — propagation requirement enforced by pre-build gate `CM-COVENANT-PROPAGATION`)**.

§11.4.1 extension (Phase 33, 2026-05-05) — FAIL-bluffs equally forbidden. A test that crashes for a script-internal reason (undefined variable under `set -u`, regex error, malformed assertion, missing argument) and produces a FAIL exit code is just as misleading as a PASS-bluff. Both let real defects ship undetected. Per parent Constitution §11.4.1, every test MUST fail ONLY for genuine product defects — script-bug failures must be fixed at the source layer (helper library, shared lib, test source), not patched in individual call sites.

Non-compliance is a release blocker regardless of context.

§11.4.2 extension (Phase 34, 2026-05-06) — Recorded-evidence requirement. A test that emits PASS without captured visual or audio evidence of the user-visible feature actually working on the screen the user would see is a §11.4 PASS-bluff. Bug #13 (VK Video on PRIMARY display while a passing test claimed playback PASS) demonstrated the gap exactly. Closing it requires the recording + analyzer infrastructure (Bug #14 — `dual_display_record.sh` / `action_timeline.sh` / Go `recording-analyzer` / `helixqa-bridge`). Per Constitution §11.4.2 every PASS for a user-visible feature MUST be cross-checked by the analyzer against the dual-display recording + action timeline. A PASS that lacks at least one matched timeline event in the analyzer findings is treated as a §11.4 PASS-bluff.

Non-compliance is a release blocker regardless of context.

§11.4.3 extension (Phase 34, 2026-05-06) — Per-device-topology test dispatch. Tests that depend on hardware topology (secondary HDMI present/absent, microphone present/absent, etc.) MUST detect topology at test entry and dispatch the topology-appropriate variant. A test running the wrong variant for the actual topology and PASSing is a §11.4 PASS-bluff. Bug #18 (Lampa+TorrServe E2E) demonstrated the pattern: D1 (secondary HDMI) and D2 (primary only) get separate test variants behind a `dumpsys display`-based dispatcher. Per Constitution §11.4.3 every topology-touching test MUST have such a dispatcher OR explicit topology gates with SKIP-with-reason fallback.

Non-compliance is a release blocker regardless of context.

§11.4.4 extension (User mandate, 2026-05-06) — Test-interrupt-on-discovery + retest-from-clean-baseline. A test cycle that continues running past a freshly discovered defect is itself a §11.4 PASS-bluff: it produces "all green" summaries while the codebase under test is known-broken at the moment those greens were recorded. Phase 34.S' D1 demonstrated the violation when Bug #26 (hard-floor probe lifecycle) and Bug #27 (analyzer FAIL-bluff on non-video tests) were discovered mid-cycle and the cycle was allowed to continue, accumulating 13+ false-positive ANALYZER FAIL banners. Per Constitution §11.4.4 the moment any defect is re- discovered, re-produced, or newly identified during a test cycle, the cycle MUST stop on both devices. **Then**: (1) fix at root cause per §11.4.1, (2) land validation/verification tests for the fix — pre-build gate AND on-device test AND paired meta-test mutation, (3) full rebuild via `scripts/build.sh` (regardless of whether the fix touched host script / Go binary / firmware — host-only fixes still get a full rebuild for retest baseline integrity), (4) re-flash D1 + D2, (5) repeat full `test_all_fixes.sh` from the beginning sequentially per §12.6, (6) end the cycle with `meta_test_false_positive_proof.sh` proving no gate is itself a bluff gate. Tests AND HelixQA Challenges are bound equally — Challenges that score PASS on a non-functional feature are the same class of defect as PASS-bluff unit tests; both must produce positive end-user evidence per §11.4.2 + §11.4.3.

Non-compliance is a release blocker regardless of context.

§11.4.4 expansion (User mandate, 2026-05-06) — Systematic debugging + four-layer test coverage + documentation + no-bluff certification. Augments the §11.4.4 base covenant with four non-negotiable additional requirements per the User mandate of 2026-05-06: (a) **Systematic debugging via superpowers skills.** Before applying any fix, run in-depth systematic debugging using the available `superpowers:*` skills (debugging, root-cause analysis, architectural-impact). Symptom patches are forbidden. The debugging output MUST identify root cause at source layer, blast radius across related tests/features/subsystems, and the regression-protection seam. (b) **Four-layer test coverage per fix.** Every fix lands with positive evidence in **every applicable layer**: pre-build gate (catches at source), post-build gate (catches in assembled image — proves bytes landed, cf. Fix #122 APK_LIB_MAP misroute), post-flash on-device test (fully automated, anti-bluff per §8.1, captured- evidence per §11.4.2, topology-dispatched per §11.4.3, orchestrator-wired in `test_all_fixes.sh`), HelixQA test bank entry (`banks/atmosphere.yaml` + per-feature additions), HelixQA full QA session coverage (Challenge-driven dispatch — bank entry without Challenge coverage is a §11.4 PASS-bluff), and meta-test paired mutation. Skipping a layer because "this fix only touches X" is forbidden. (c) **Documentation update for every fix.** Required: `docs/Issues.md` → `docs/Fixed.md` migration on closure, parent CLAUDE.md Applied Fixes Reference row, affected user-facing guides (`docs/guides/*.md`), affected diagrams/flowcharts/architecture docs, per-version `docs/changelogs/<tag>.md` entry. Documentation drift after a fix is itself a §11.4 violation. (d) **No-bluff certification per cycle.** Before tagging: `meta_test_false_positive _proof.sh` returns all gates green AND every gate's paired mutation FAILs (no bluff gates); `docs/Issues.md` open-set is empty or every entry explicitly classified out-of-scope-for-this-tag with operator sign-off (no known issues hidden); full suite returns zero new FAILs on either device (no working feature regressed); every gate has a paired mutation; every test produces positive evidence; every assertion catches its own negation (no error-prone or bluff-proof leftover).

Non-compliance is a release blocker regardless of context.

§11.4.5 — Audio + video quality analysis comprehensiveness (User mandate, 2026-05-07)

Forensic anchor — direct user mandate (verbatim, 2026-05-07):

> "We MUST HAVE still analyzing of recorded materials and comprehensive validation and verification for issues we used to test! For example if there is audio at all or video, if so, is it good and proper or is it faulty? Does it have glitches, frame issues and other possible obstructions? IMPORTANT: Make sure that all existing tests and Challenges do work in anti-bluff manner — they MUST confirm that all tested codebase really works as expected!"

§11.4.2 mandates *captured* evidence; §11.4.5 mandates the **content** of that evidence be analyzed for quality, not merely for presence. A test that captures a 0-byte mp4 (Bug #24) and PASSes because "the recording file exists" is the exact PASS-bluff pattern §11.4 forbids. Content-quality analysis is what closes that gap.

Audio quality analysis — every audio test that PASSes MUST verify ALL of: (1) **Presence** — non-trivial RMS amplitude in captured WAV / `/proc/asound/.../pcm*p/sub0/hw_params`. (2) **Channel count** — `ffprobe -show_streams` matches the test's claim (2.0 / 5.1 / 7.1). (3) **Sample rate + bit depth** — match the codec / pipeline under test. (4) **Glitch census** — XRUN / FastMixer underrun-overrun-partial / AudioFlinger writeError counts above tolerance MUST classify explicitly (PASS within budget, WARN above, FAIL on hard limits per §11.4.1 SKIP-vs-FAIL decision tree). (5) **Coexistence-artifact census** — for tests that exercise WiFi/BT alongside audio: BT TX queue overflow, A2DP src underflow, coex notification storms, 2.4 GHz radio contention.

Video quality analysis — every video test that PASSes MUST verify ALL of: (1) **Presence** — captured screen recording has non-zero file size AND `ffprobe -count_frames` reports decoded-frame total > 0. 0-byte mp4 (Bug #24) is the canonical PASS-bluff and triggers §11.4.4 STOP. (2) **Routing target** — analyzer + action-timeline confirms video appeared on the *intended* display (primary vs secondary HDMI; Bug #13 pattern). (3) **Frame health** — drop count, frame-time variance (jitter), freeze detection (SSIM > 0.99 for ≥ 1 s), tearing. (4) **Obstruction census** — Tesseract OCR scan for hostile overlays (`Application not responding`, `Force close`, sign-in dialog, geo-restriction overlay, ad break, paywall, `App is not certified`). (5) **Resolution + codec** — captured frame dimensions match the test's claim; downgrade is a PASS-bluff.

Challenges (HelixQA) are bound equally — every Challenge that asserts PASS MUST run all five audio + five video layers. A Challenge that scores PASS without applicable analysis is the same class of defect as a unit test that does.

Tooling guarantee: audio = `tinycap` + `aplay --dump-hw-params` + `ffprobe` + `/proc/asound` parsers (`lib/audio_validation.sh` per §11.2.5). Video = `screenrecord` + `ffprobe -count_frames` + `recording-analyzer` + Tesseract OCR (`scripts/dual_display_record.sh` + `cmd/recording-analyzer/` per §11.4.2.A and §11.4.2.C). Tests dispatched against video evidence MUST honor §11.4.4 test-interrupt-on-discovery when the analyzer reports empty input — do not silently absorb that as a generic PASS-bluff banner.

Non-compliance is a release blocker regardless of context.

This covenant was lost from this file during the Phase 23.8 remote merge (unrelated `pkg/autonomous/` work pulled in concurrently with the Whisper Go client) and was restored in Phase 24.2b (2026-04-30) when the parent repo's `CM-COVENANT-PROPAGATION` and `CM-COVENANT-FORENSIC-QUOTE` gates flagged it during a routine pre-build run. Same regression class as Phase 23.2 (Challenges submodule, same week). The 100% anti-bluff floor catches this kind of remote-merge dropout that would otherwise go unnoticed for cycles.

MANDATORY §12.6 MEMORY-BUDGET CEILING — 60% MAXIMUM (User mandate, 2026-04-30)

Forensic anchor — direct user mandate (verbatim):

> "We had to restart this session 3rd time in a row! The system of the host stays with no RAM memory for some reason! First make sure that whatever we do through our procedures related to this project MUST NOT use more than 60% of total system memory! All processes MUST be able to function normally!"

The mandate. Project procedures MUST NOT use more than **60% of total system RAM** (`HOST_SAFETY_MAX_MEM_PCT`). The remaining 40% is reserved for the operator's other workloads so the host can keep serving them while project work proceeds.

Three consecutive session-loss SIGKILLs on 2026-04-30 during 1.1.5-dev — every one happened while `scripts/build.sh` was running `m -j5` AOSP. Each Soong/Ninja job peaks at ~5–8 GiB RSS; collective RSS overran the 60% envelope and the kernel OOM-killer escalated, taking down `user@1000.service`. **§12.1's pre-flight check (refusing to start if host already distressed) was not enough** — the missing piece was an active CONSTRAINT on heavy work itself.

Mandatory protections (rock-solid):

1. `HOST_SAFETY_MAX_MEM_PCT` defaults to 60 in `scripts/lib/host_session_safety.sh`.
2. `HOST_SAFETY_BUDGET_GB` is computed at source-time from `MemTotal × MAX_PCT/100`.
3. `bounded_run` clamps `MemoryMax` down to the budget if the caller asks for more (cgroup-level enforcement via `systemd-run --user --scope -p MemoryMax=…`).
4. `host_safe_parallel_jobs` and `host_safe_build_jobs` return the safe `-j` count given an estimated per-job RSS, capped at `nproc`.
5. `scripts/build.sh` wraps `m -j` in `bounded_run`. If the build's collective RSS exceeds the budget, only the scope is OOM-killed; `user@<uid>.service` stays alive.

Captured-evidence enforcement. Pre-build gate `CM-MEMBUDGET-METATEST` locks all 7 invariants and fires every pre-build run.

No escape hatch. §12.6 has NO operator-facing override flag. The cap exists for the operator's own protection; bypassing it is the bluff the §11.4 covenant specifically prohibits. Operators who need more headroom should reduce parallelism, close other workloads, or add RAM — NOT raise the percentage.

Canonical authority: parent `docs/guides/ATMOSPHERE_CONSTITUTION.md` §12.6.

Non-compliance is a release blocker regardless of context. *Remember: Your code will be used by real people. Write code that actually works.*

§11.4.6 — No-guessing mandate (User mandate, 2026-05-08)

Forensic anchor — direct user mandate (verbatim, 2026-05-08T18:30 MSK):

> "'LIKELY' is guessing, we MUST NOT have guessing, since it can be or may not be! No bluffing and uncertainity is allowed at any cost! We MUST always know exactly precisly what is happening exactly, in any context, under any conditions, everywhere!"

Tests, gates, status reports, closure narratives, commit messages, and operator-facing text MUST NOT use `likely`, `probably`, `maybe`, `might`, `possibly`, `presumably`, `seems`, or `appears to` when describing causes of failures, behaviour, or fix effectiveness. Either prove the cause with captured forensic evidence (logcat, dmesg, /sys readings, getprop, kernel ramoops, dropbox, strace, etc.) and state it as fact, OR explicitly mark `UNCONFIRMED:` / `UNKNOWN:` / `PENDING_FORENSICS:` with a tracked-task ID for follow-up.

Pre-build gate `CM-NO-GUESSING-MANDATE` greps recently-modified docs + test scripts for the forbidden vocabulary outside explicit `UNCONFIRMED:` / `UNKNOWN:` / `PENDING_FORENSICS:` blocks. Paired mutation introduces a `likely` token into a fresh status block → gate FAILs. Propagation gate `CM-COVENANT-114-6-PROPAGATION` enforces this anchor in every CLAUDE.md / AGENTS.md across parent + 10 owned submodules + HelixQA dependencies.

Canonical authority: parent `docs/guides/ATMOSPHERE_CONSTITUTION.md` §11.4.6.

Non-compliance is a release blocker regardless of context.

§11.4.7 — Demotion-evidence rule (Phase 38.X+2 amendment, 2026-05-11)

A demotion from any FAIL classification (`OPEN`, `POSSIBLE PRODUCT DEFECT`, `FAIL`) to a lower-severity classification (`INVESTIGATED`, `MITIGATED`, `RESOLVED`, `WORKING-AS-INTENDED`) requires positive evidence captured under the **same conditions** that originally exposed the defect — same device, same firmware, same cycle position, same load profile.

"I cannot reproduce in isolation" is a HYPOTHESIS, not a finding. Per §11.4.6 it MUST be tagged `UNCONFIRMED:` until same-conditions retest produces positive evidence. The expanded forbidden-vocabulary list:

Forbidden phrase	Why it bluffs
"isolated re-run PASSes therefore X was a flake"	Strips the very environment that exposed the defect.

Forbidden phrase	Why it bluffs
“runtime drift”	Label for “we don’t know what changed”.
“intermittent” / “transient”	Label for “we don’t know how to reproduce”.
“pending stress retest”	Defers the actual investigation indefinitely.
“correlates with X”	Hypothesis presented as causation.

Pre-build gate `CM-DEMOTION-EVIDENCE-RULE` scans Issues.md / Fixed.md / CONTINUATION.md for these phrases outside explicit `UNCONFIRMED:` / `UNATTRIBUTED:` / `PENDING_CYCLE_RETEST:` blocks. Propagation gate `CM-COVENANT-114-7-PROPAGATION` enforces this anchor in every CLAUDE.md / AGENTS.md across parent + 10 owned submodules + HelixQA dependencies.

Canonical authority: parent `docs/guides/ATMOSPHERE_CONSTITUTION.md` §11.4.7.

Non-compliance is a release blocker regardless of context.

§11.4.8 — Deep-web-research-before-implementation mandate (User mandate, 2026-05-12)

Before designing a non-trivial fix, implementing a new feature, or declaring an architectural choice, perform deep web research to verify the chosen approach is informed by current state-of-the-art. Research surface: official documentation (Android/AOSP/Khronos/CEA-861/AES/IEEE/IETF/ITU), vendor technical guides (Rockchip, Sipeed, Audinate Dante, Synaptics, Realtek, Bluetooth SIG), open-source codebases (Linux kernel, ALSA, Bluez, ExoPlayer, libVLC, MPV, FFmpeg, AOSP forks), coding tutorials + technical articles (Stack Overflow, AOSP Code Lab, AES papers), issue trackers (Android bug tracker, AOSP gerrit, GitHub issues).

A fix that re-invents a wheel — or reproduces a known-broken pattern — when the open-source community has already solved the problem is a §11.4 violation by omission. Every non-trivial fix’s commit / Issues.md / Fixed.md entry MUST cite at least one external source URL OR the literal “NO external solution found — original work”.

Pre-build gate `CM-RESEARCH-CITATION-PRESENT` scans new fix-direction blocks for the pattern. Propagation gate `CM-COVENANT-114-8-PROPAGATION` enforces this anchor in every CLAUDE.md / AGENTS.md across parent + 10 owned submodules + HelixQA dependencies.

Documentation continuity requirement: every fix landed under §11.4.8 also adds to `docs/guides/` a user-facing or developer-facing guide section where appropriate.

Canonical authority: parent `docs/guides/ATMOSPHERE_CONSTITUTION.md` §11.4.8.

Non-compliance is a release blocker regardless of context.

§11.4.9 — Batch-source-fixes-before-rebuild mandate (User mandate, 2026-05-12)

When closing a multi-defect batch, all source-side fixes that DO NOT require runtime on-device validation to design MUST be landed BEFORE the next firmware rebuild. Anti-pattern eliminated:
`Fix A → rebuild → flash → cycle → fix B → rebuild → ...` serializes 7-8 hours per fix instead of batching all into ONE build cycle. Operator time is the scarce resource.

Exceptions documented in commit message as `REQUIRES_REBUILD: <reason>`: kernel-5.10/ changes, atmosphere-*.sh boot-script side-effects, hardware/rockchip/ HAL behavior — each gates downstream state and requires firmware to validate.

Before declaring a batch "ready for rebuild": pre-build GREEN + meta-test GREEN + existing-device validations performed where possible + Issues.md/Fixed.md/ CONTINUATION.md in sync (+ HTML/PDF exported) + §11.4.8 research citations all logged.

Propagation gate `CM-COVENANT-114-9-PROPAGATION` enforces this anchor in every CLAUDE.md / AGENTS.md across parent + 10 owned submodules + HelixQA dependencies.

Canonical authority: parent `docs/guides/ATMOSPHERE_CONSTITUTION.md` §11.4.9.

Non-compliance is a release blocker regardless of context.

§11.4.10 — Credentials-handling mandate (User mandate, 2026-05-12)

All credentials, secrets, API tokens, passwords, phone numbers, OAuth tokens, signing keys MUST NEVER live in tracked files. Templates with placeholder values are allowed (`.example` suffix). Tests load credentials at runtime from `scripts/ testing/secrets/` (or per-submodule equivalent); operator-populated files are `chmod 600`, directory is `chmod 700`. `.env`, `.env.*`, `*.env` patterns + `scripts/ testing/secrets/*` (with `.example` + `README.md` exception) git-ignored project-wide.

Test scripts MUST NEVER echo credentials to stdout/stderr/logcat. Screen-recording of sign-in flows MUST redact credential-bearing frames. Per-service file separation (`.netflix.env`, `.disney.env`, etc.) limits blast radius.

Forensic-rotation policy: suspected leak → rotate at provider, update local `.env`, audit captured artifacts. Pre-build gate `CM-CREDENTIAL-LEAK-SCAN` greps tracked files for entropy-suspicious password strings + known API-token formats. Propagation gate `CM-COVENANT-114-10-PROPAGATION` enforces this anchor in every CLAUDE.md / AGENTS.md across parent + 10 owned submodules + HelixQA dependencies.

Canonical authority: parent `docs/guides/ATMOSPHERE_CONSTITUTION.md` §11.4.10.

Non-compliance is a release blocker regardless of context.

§11.4.14 — Test playback cleanup mandate (User mandate, 2026-05-13)

Every test that issues `am start` / `cmd media_session play` / `MediaController.play` MUST issue matching `am force-stop` / `input keyevent KEYCODE_MEDIA_STOP` + register cleanup in `EXIT` trap. Verified via

positive evidence (Arvus codec-state → `N.E.`, `dumpsys media_session` shows no PLAYING for test app). `test_all_fixes.sh` post-test sanity check FAILs the just-completed test if it left orphan playback. HelixQA Challenges bound equally. No grace period — "next test will clean it up" is §11.4 PASS-bluff.

Canonical authority: parent `docs/guides/ATMOSPHERE_CONSTITUTION.md` §11.4.14. Pre-build gates `CM-TEST-PLAYBACK-CLEANUP` + `CM-COVENANT-114-14-PROPAGATION`.

Non-compliance is a release blocker regardless of context.

§11.4.15 — Item-status tracking mandate (User mandate, 2026-05-13)

Every active item in `docs/Issues.md` carries a `**Status:**` line with one of six values: `Queued`, `In progress`, `Ready for testing`, `In testing`, `Reopened`, `Fixed (→ Fixed.md)`. Status MUST be updated as the item progresses through its lifecycle. `Fixed` requires captured-evidence per §11.4.5 + migration to Fixed.md.

The auto-generated `docs/Issues_Summary.md` includes the Status column. All three file types (`.md`, `.html`, `.pdf`) MUST be in sync at all times — enforced by `CM-DOCS-EXPORT-SYNC` (§11.4.12 + §11.4.15 amendment).

Canonical authority: parent `docs/guides/ATMOSPHERE_CONSTITUTION.md` §11.4.15. Pre-build gates `CM-ITEM-STATUS-TRACKING` + `CM-COVENANT-114-15-PROPAGATION`.

Non-compliance is a release blocker regardless of context.

§11.4.16 — Item-type tracking mandate (User mandate, 2026-05-14)

Every active item in `docs/Issues.md` carries a `**Type:**` line with one of three values: `Bug` (product defect / regression / user-visible broken behaviour), `Feature` (new capability not previously offered to end users), `Task` (internal workstream — refactor, doc, infra, gate, audit; the lowest-stakes default when ambiguous). The vocabulary is CLOSED — no other value is permitted.

The auto-generated `docs/Issues_Summary.md` includes the Type column. All three file types (`.md`, `.html`, `.pdf`) MUST be in sync at all times — enforced by `CM-DOCS-EXPORT-SYNC` (§11.4.12 + §11.4.15 + §11.4.16 amendment).

Canonical authority: parent `docs/guides/ATMOSPHERE_CONSTITUTION.md` §11.4.16. Pre-build gates `CM-ITEM-TYPE-TRACKING` + `CM-COVENANT-114-16-PROPAGATION`.

Non-compliance is a release blocker regardless of context.

§11.4.13 — Out-of-band sink-side captured-evidence mandate (User mandate, 2026-05-13)

Whenever an HDMI sink with a network-accessible introspection API is present (current example: Arvus H2-4D-273 at `http://192.168.4.185/`), the test suite MUST consume the sink's report as captured-evidence for every audio test asserting a codec / channel-count / passthrough mode. On-SoC HAL telemetry ALONE is insufficient — that is the exact "tests pass but the feature doesn't work" pattern §11.4 forbids. Reference: `scripts/testing/lib/arvus_probe.sh`, `scripts/testing/arvus_probe.sh`, `docs/guides/ARVUS_HDMI_INTEGRATION.md`. Pre-

build gate CM-ARVUS-EVIDENCE-INTEGRATED (7 invariants) + paired mutation. No hardcoding (env: ARVUS_HOST etc.). Topology dispatch per §11.4.3 — sink unreachable → SKIP, never FAIL. Identity verification (MAC match) before consuming codec-state. Anti-stickiness post-stop. HelixQA Challenges bound equally.

Canonical authority: parent docs/guides/ATMOSPHERE_CONSTITUTION.md §11.4.13. Integration reference: docs/guides/ARVUS_HDMI_INTEGRATION.md.

Non-compliance is a release blocker regardless of context.

§11.4.11 — File-layout discipline (User mandate, 2026-05-12)

Files live in canonical directories per type: - Shell scripts → scripts/ (legacy: scripts/legacy/) - Log files → logs/ (legacy: logs/legacy/) - Release artifacts → releases/<app>/<version>/ - Operator credentials → scripts/testing/secrets/ (per §11.4.10, git-ignored) - Markdown docs → docs/ + docs/guides/ + docs/research/ + docs/superpowers/plans/ - Per-version changelogs → docs/changelogs/ - Hardware ID photos → docs/hardware/<device-slug>/

Repo root contains ONLY: AOSP-mandated top-level files (Android.bp, Makefile, bootstrap.bash, BUILD, kokoro, lk_inc.mk, OWNERS, version_defaults.mk), project metadata (README/CLAUDE/AGENTS/CONTRIBUTING/LICENSE/NOTICE/VERSION), dot-files (.gitignore/.gitmodules), and standard top-level dirs (build/, device/, external/, frameworks/, hardware/, kernel-5.10/, packages/, prebuilts/, scripts/, system/, tools/, vendor/, docs/, releases/, logs/).

NO bash scripts in repo root except AOSP-mandated bootstrap.bash. NO log files in repo root. NO duplicate filenames between root and scripts/. NO release artifacts in root. Moves require triple-verification (audit all references + distinguish absolute vs subdir-local + confirm no AOSP build- system requirement). Pre-build gate CM-FILE-LAYOUT-DISCIPLINE enforces. Propagation gate CM-COVENANT-114-11-PROPAGATION enforces this anchor in every CLAUDE.md / AGENTS.md across parent + 10 owned submodules + HelixQA dependencies.

Canonical authority: parent docs/guides/ATMOSPHERE_CONSTITUTION.md §11.4.11.

Non-compliance is a release blocker regardless of context.

§11.4.12 — Issues_Summary.md sync mandate (User mandate, 2026-05-12)

docs/Issues_Summary.md is the canonical short-form summary of all open items. MUST be regenerated + re-exported (HTML + PDF) whenever Issues.md changes. Generator: scripts/testing/generate_issues_summary.sh. Pre-build gates CM-ISSUES-SUMMARY-SYNC + CM-COVENANT-114-12-PROPAGATION enforce mechanically.

Sort order (User mandate refinement 2026-05-12): severity DESC (C → M → L), then intra-group criticality DESC inside each group. Most critical row = #1, least critical = #N. Documented at the top of the generated file.

Auto-sync wrapper: scripts/testing/sync_issues_docs.sh — runs generator + export_progress_docs.sh in one shot. MUST be invoked after any edit to Issues.md or Issues_Summary.md. HTML+PDF exports are NEVER manually invoked; they ALWAYS travel with the markdown.

Canonical authority: parent `docs/guides/ATMOSPHERE_CONSTITUTION.md` §11.4.12.

Non-compliance is a release blocker regardless of context.

CONST-047 — Recursive Submodule Application Mandate (cascaded from root CONSTITUTION.md)

> Verbatim user mandate (2026-05-14): *"Make sure all work we do is applied ALWAYS to all Submodules we control under our organizations (vasic-digital and HelixDevelopment) fully recursively everywhere with full bluff-proofing and comprehensive documentation, user manuals and guides and full tests and Challenges coverage!"*

Every engineering deliverable produced for the main project MUST be applied — fully and recursively — to every owned submodule under the `vasic-digital` and `HelixDevelopment` GitHub organizations. Each owned submodule (including this one) MUST receive in lockstep: (1) anti-bluff posture (CONST-035 / Article XI §11.9), (2) comprehensive documentation matching actual capabilities, (3) full tests + Challenges coverage with captured runtime evidence, (4) recursive propagation through nested submodules under the same orgs, (5) synchronized commits when meta-repo state advances this surface.

See the root `CONSTITUTION.md` §CONST-047 for the full mandate. This anchor MUST remain in this submodule's CONSTITUTION.md, CLAUDE.md, and AGENTS.md.

§11.4.40 — Full-suite retest before release tag mandate (User mandate, 2026-05-17)

A release tag MUST NOT be created until a COMPLETE retest with ALL existing tests has been executed on a clean baseline AFTER every workable item in the batch is done, fixed, polished, and individually verified. Spot-check retests that run only the tests touched by the batch are FORBIDDEN — they miss interaction defects between the batch's fixes and previously-stable code.

The complete retest comprises: (1) pre-build full sweep, (2) post-build full sweep, (3) on-device 4-phase cycle on EVERY owned device, (4) meta-test full mutation sweep, (5) Challenge bank full sweep, (6) Issues.md/Fixed.md state audit, (7) CONTINUATION.md sync check.

Time is essential — complete retest is typically 12–48 hour elapsed effort. NOT optional, NOT abbreviated. Skipping is the exact "tests passed but feature broken" failure mode §11.4 specifically prohibits.

Composes with §11.4.4 (per-fix retest) — §11.4.37 is the additional final integrity check at RELEASE granularity. Composes with §11.4.7 — full-suite retest is the authoritative baseline for closures in the batch. No escape hatch — no `--skip-full-retest` or `--quick-release` flag exists.

Pre-build gate `CM-FULL-SUITE-RETEST-MANDATE` + paired mutation. Propagation gate `CM-COVENANT-114-40-PROPAGATION` enforces this anchor in every CLAUDE.md/AGENTS.md across parent + 10 owned submodules + HelixQA dependencies.

Canonical authority: constitution submodule `Constitution.md` §11.4.37.

Non-compliance is a release blocker regardless of context.

§11.4.41 — Pre-Force-Push Merge-First Mandate (User mandate, 2026-05-17)

Any force-push (`git push --force`, `git push --force-with-lease`, `git push +<ref>`, or equivalent history-rewriting operation on any remote) authorised under §9.2 / CONST-043 MUST be preceded by a mechanical 4-step merge-first pipeline that brings every remote-side commit into the local tree, resolves every conflict carefully, and verifies nothing is lost or corrupted on EITHER side BEFORE the overwriting push is executed.

The 4-step pipeline (mandatory, in order): (1) `git fetch --all --prune --tags` against every configured remote — capture output. (2) Integrate every divergent commit locally via `git rebase` (local is strict superset), `git merge` (independent additions both deserve preservation), or operator-confirmed cherry-pick (remote subset already present locally). (3) Audit: no conflict markers (`grep -rn '^<<<<<<< \|^=======$\|^>>>>>>> '` returns empty), no silent file drops (`git diff --stat HEAD@{1} HEAD`), every previously-passing test still passes per §11.4.4 / §11.4.40 baseline, every captured-evidence artifact still validates. (4) `git push --force-with-lease <remote> <ref>` (NEVER `--force` without `--with-lease` unless §9.2 sub-clause 6 explicitly authorises it for a remote where lease semantics are unavailable). One force-push event per CONST-043 authorisation — no batch authorisation.

Two-gate composition with CONST-043 — §11.4.41 does NOT relax CONST-043's operator-approval requirement. Gate A (CONST-043): operator types explicit per-operation force-push authorisation. Gate B (§11.4.41): agent executes the 4-step merge-first pipeline, captures evidence of clean integration, presents evidence to operator BEFORE the force-push. Both gates required.

Verification artefact — every §11.4.41-governed force-push emits a `docs/changelogs/<tag>.md` "Force-push merge-first audit" section containing 7 elements: (i) `git fetch` output, (ii) per-remote `HEAD..<remote>/<branch>` log before integration, (iii) integration strategy chosen per remote with rationale, (iv) post-integration conflict-marker scan output (must be empty), (v) post-integration test suite delta (must show only expected changes), (vi) `--force-with-lease` push output with lease SHA evidence, (vii) CONST-043 authorisation quote from the conversation.

Composes with §9.2 (data-safety hardlinked backup), §11.4.4 (test-interrupt-on-discovery — broken integration triggers rollback), §11.4.6 (no-guessing — every step's outcome captured, not assumed), §11.4.26 (constitution-submodule update pipeline — per-submodule specialisation), §11.4.32 (post-pull validation — audit step's mechanical companion), §11.4.37 (fetch-before-edit — step 1 enforces it for force-push specifically), §11.4.40 (full-suite retest — step 3's test-evidence requirement).

No escape hatch — the operator-pressure escape ("just force-push, we'll fix it later") is the exact failure mode this anchor closes. Pre-build gate `CM-COVENANT-114-41-PROPAGATION` enforces this anchor in every CLAUDE.md/AGENTS.md across parent + 10 owned submodules + nested submodules + HelixQA dependencies. Paired mutation strips the anchor literal → gate FAILs. Gate `CM-FORCE-PUSH-MERGE-FIRST` walks `docs/changelogs/<tag>.md` "Force-push" entries for the 7 audit elements; paired mutation strips any element and asserts gate FAILs.

Canonical authority: constitution submodule `Constitution.md` §11.4.41.

Non-compliance is a release blocker regardless of context.

CONST-048: Full-Automation-Coverage Mandate (cascaded from constitution submodule §11.4.25)

> Verbatim user mandate (2026-05-15): *"Make sure that every feature, every functionality, every flow, every use case, every edge case, every service or application, on every platform we support is covered with full automation tests which will confirm anti-bluff policy and provide the proof of fully working capabilities, working implementation as expected, no issues, no bugs, fully documented, tests covered! Nothing less than this does not give us a chance to deliver stable product! This is mandatory constraint which MUST BE respected without ignoring, skipping, slacking or forgetting it!"*

No feature / functionality / flow / use case / edge case / service / application on any supported platform of this submodule is deliverable until covered by automation tests proving six invariants: (1) anti-bluff posture with captured runtime evidence (CONST-035); (2) proof of working capability end-to-end on target topology; (3) implementation matching documented promise; (4) no open issues/bugs surfaced; (5) full documentation in sync; (6) four-layer test floor (pre-build + post-build + runtime + paired mutation).

Cascade requirement: This anchor (verbatim or by CONST-048 ID reference) MUST remain in this submodule's CONSTITUTION.md, CLAUDE.md, and AGENTS.md, and propagate recursively to any nested owned-by-us submodule. See parent project's `CONSTITUTION.md` §CONST-048 and constitution submodule `Constitution.md` §11.4.25 for the full mandate. ## CONST-049: Constitution-Submodule Update Workflow Mandate (cascaded from constitution submodule §11.4.26)

> Verbatim user mandate (2026-05-15): *"Every time we add something into our root (constitution Submodule) Constitution, CLAUDE.MD and AGENTS.MD we MUST FIRST fetch and pull all new changes / work from constitution Submodule first! All changes we apply MUST BE commited and pushed to all constitution Submodule upstreams! In case of conflict, IT MUST BE carefully resolved! Nothing can be broken, made faulty, corrupted or unusable! After merging full validation and verification MUST BE done!"*

Before ANY modification to `constitution/{Constitution,CLAUDE,AGENTS}.md` in the parent project, the agent or operator MUST execute the 7-step pipeline: (1) fetch + pull first inside the constitution submodule worktree; (2) apply the change with §11.4.17 classification + verbatim

mandate quote; (3) validate (meta-test + no merge-conflict markers + cross-file consistency); (4) commit + push to EVERY configured upstream of the constitution submodule (governance files only — never `git add -A`); (5) careful conflict resolution preserving union of governance content (force-push forbidden per CONST-043 / §9.2); (6) post-merge `git submodule update --remote --init` + re-run cascade verifier (CONST-047); (7) bump consuming project's `.gitmodules` pointer to the new constitution HEAD in the SAME commit as cascade work.

Cascade requirement: This anchor (verbatim or by CONST-049 ID reference) MUST remain in this submodule's CONSTITUTION.md, CLAUDE.md, and AGENTS.md, and propagate recursively to any nested owned-by-us submodule. See parent project's `CONSTITUTION.md` §CONST-049 and constitution submodule `Constitution.md` §11.4.26 for the full mandate. ## CONST-050: No-Fakes-Beyond-Unit-Tests + 100%-Test-Type-Coverage Mandate (cascaded from constitution submodule §11.4.27)

> Verbatim user mandate (2026-05-15): *"Mocks, stubs, placeholders, TODOs or FIXMEs are allowed to exist ONLY in Unit tests! All other test types MUST interract with real fully implemented System! No fakes, empty implementations or bluffing is allowed of any kind! All codebase of the project MUST BE 100% covered with every supported test type: unit tests, integration tests, e2e tests, full automation tests, security tests, ddos tests, scaling tests, chaos tests, stress tests, performance tests, benchmarking tests, ui tests, ux tests, Challenges (fully incorporating our Challenges Submodule — https://github.com/vasic-digital/Challenges). EVERYTHING MUST BE tested using HelixQA (fully incorporating HelixQA Submodule — https://github.com/HelixDevelopment/HelixQA). HelixQA MUST BE used with all possible written tests suites (test banks) for every applications, service, platform, etc and execution of the full HelixQA QA autonomous sessions! All required dependency Submodules MUST BE added into the project as well (fully recursive!!!)."*

Two cooperating invariants:

(A) No-fakes-beyond-unit-tests. Mocks, stubs, fakes, placeholders, `TODO`, `FIXME`, "for now", "in production this would", or empty-implementation patterns are PERMITTED only in unit-test sources. Every other test type — integration, E2E, full automation, security, DDoS, scaling, chaos, stress, performance, benchmarking, UI, UX, Challenges, HelixQA — MUST exercise this submodule's real, fully implemented system against real infrastructure. Production code MUST NOT import mock paths.

(B) 100% test-type coverage. Codebase MUST be covered by every supported test type the domain warrants: unit, integration, E2E, full-automation, security, DDoS, scaling, chaos, stress, performance, benchmarking, UI, UX, Challenges (vasic-digital/Challenges submodule fully incorporated), HelixQA (HelixDevelopment/HelixQA submodule fully incorporated, with full autonomous QA sessions executing every registered test bank with captured wire evidence).

Required dependency submodules (recursive per CONST-047): Challenges + HelixQA + any other functionality submodules under vasic-digital/HelixDevelopment orgs this submodule depends on.

Cascade requirement: This anchor (verbatim or by CONST-050 ID reference) MUST remain in this submodule's CONSTITUTION.md, CLAUDE.md, and AGENTS.md, and propagate recursively to any nested owned-by-us submodule. See parent project's `CONSTITUTION.md` §CONST-050 and constitution submodule `Constitution.md` §11.4.27 for the full mandate. ## CONST-051: Submodules-As-Equal-Codebase + Decoupling + Dependency-Layout Mandate (cascaded from constitution submodule §11.4.28)

> Verbatim user mandate (2026-05-15): *"All existing Submodules in the project that we are controlling and belong to some our organizations (vasic-digital, HelixDevelopment, red-elf, ATMOSphere1234321, Bear-Suite, BoatOS123456, Helix-Flow, Helix-Track, Server-Factory - we can ALWAYS check dynamically using GitHub and GitLab CLIs) are equal parts of the project's codebase! We MUST work on that code as much as we do with main project's codebase! All on equal basis! Equally important! ... We MUST NEVER modify Submodules to bring into them any project specific context since they all MUST BE ALWAYS fully decoupled, project not-aware, fully reusable and modular (by any other project(s)), completely testable! All Submodule dependencies that are used by Submodule MUST BE acessed from the root of the project! We MUST NOT have nested Submodule dependencies but accessing each from proper location from the root of the project - directly from project's root project_name/submodule_name or some more proper structure project_name/submodules/submodule_name!"*

Three cooperating invariants apply to every owned-by-us submodule (orgs: vasic-digital, HelixDevelopment, red-elf, ATMOSphere1234321, Bear-Suite, BoatOS123456, Helix-Flow, Helix-Track, Server-Factory, plus any subsequently authorised org — discoverable via `gh org list` / `glab`):

(A) Equal-codebase. This submodule is an EQUAL part of every consuming project's codebase. The consuming project's engineering practice — analysis, extension, test creation, gap-filling, bug-fix, documentation (user manuals, guides, diagrams, graphs, SQL definitions, website pages, all materials) — applies to this submodule on equal basis. Coverage ledgers (CONST-048) list this submodule as an in-scope target.

(B) Decoupling / reusability. This submodule MUST remain fully decoupled from any specific consuming project. NEVER inject project-specific context (hardcoded paths, hostnames, asset names, naming schemes). Stay project-not-aware, reusable, modular, completely testable as a standalone repository. When parent-project info is needed, use configuration injection (env var, config file, constructor parameter) — never a hardcoded reach.

(C) Dependency-layout. Any dependency this submodule consumes MUST be accessible from the consuming project's root at `<root>/<name>/` or `<root>/submodules/<name>/`. **Nested own-org submodule chains are FORBIDDEN** — this submodule MUST NOT have its own `.gitmodules` entries pulling in further owned-by-us repos. Third-party submodules are exempt.

Cascade requirement: This anchor (verbatim or by CONST-051 ID reference) MUST remain in this submodule's CONSTITUTION.md, CLAUDE.md, and AGENTS.md, and propagate recursively to any nested owned-by-us submodule. See parent project's `CONSTITUTION.md` §CONST-051 and constitution submodule `Constitution.md` §11.4.28 for the full mandate. ## CONST-052: Lowercase-Snake_Case-Naming Mandate (cascaded from constitution submodule §11.4.29)

> Verbatim user mandate (2026-05-15): *“naming convention for Submodules and directories (applied deep into hierarchy recursively) - all directories and Submodules MSUT HAVE lowercase names with space separator between the words of ’_’ character (snake-case)! All existing Submodules and directories which are not following this rule MUST BE renamed! However, since this will most likely break some of the functionalities renaming we do MUST BE applied to all references to particular Submodule or directory! … There MUST BE reasonable exceptions for this rules - source code for programming languages or Submodules which apply different naming convention - Android, Java, Kotlin and others. … Upstreams directory which all of our projects and Submodules have MUST BE renamed to the lowercase letters too, however root project containing the install_upstreams system command (it is exported in out paths in our .bashrc or .zshrc) MUST BE updated to fully work with both Upstreams and upstreams directory. … NOTE: Rules lowercase / snake-case do apply to all project files as well and references to it and from them!“*

Every directory, submodule, and file in this submodule MUST use lowercase snake_case names. Existing non-compliant names MUST be renamed atomically with updates to every reference (configs, docs, source-code imports, governance files). Reference drift after rename = CONST-052 violation of equal severity to the rename itself.

Common-sense exceptions (technology-preserving): language-mandated case for Java/Kotlin/Android/Apple/C#/Swift INSIDE language-roots; vendor/upstream third-party submodules keep upstream names; build artefacts (`node_modules`, `__pycache__`, `.git`, `target`, `build`, `bin`) keep tool-mandated names. The test “does renaming break the technology?” trumps the rule.

`Upstreams/` → `upstreams/` transition: the constitution submodule’s `install_upstreams.sh` (exported via `.bashrc`/`.zshrc`) supports BOTH directory layouts; lowercase wins when both present.

Test coverage of renames (per CONST-050(B)): regression test for reference resolution + full test-type matrix run + anti-bluff wire-evidence captured.

Cascade requirement: This anchor (verbatim or by CONST-052 ID reference) MUST remain in this submodule’s CONSTITUTION.md, CLAUDE.md, and AGENTS.md, and propagate recursively to any nested owned-by-us submodule. See parent project’s `CONSTITUTION.md` §CONST-052 and constitution submodule `Constitution.md` §11.4.29 for the full mandate.

CONST-053: .gitignore + No-Versioned-Build-Artifacts Mandate (cascaded from constitution submodule §11.4.30)

> Verbatim user mandate (2026-05-15): *“every project module, every Submodule, every servcie and apolication MUST HAVE proper .gitignore file! We MUST NOT git version build artifacts, cache files, tmp files, main .env file(s) or any files containing sensitive data, API keys or token! Any build derivate which we can recreate by executing proper mechanism for*

> *generating MUST NOT be versioned! We MUST pay attention what is going to be commited every time we are preparing to execute commit! If any violetion is detected it MUST be fixed before commit is executed!"*

Every project module, owned-by-us submodule, service, and application MUST ship a proper `.gitignore`. Forbidden-from-version-control classes:

1. **Build artefacts**: `/bin/`, `/build/`, `/dist/`, `/out/`, `target/`, `*.exe`, `*.dll`, `*.so`, `*.dylib`, `*.a`, `*.o`, `*.class`, `*.pyc`, generator-produced files when the generator is committed.
2. **Cache files**: `__pycache__/`, `.pytest_cache/`, `.mypy_cache/`, `.ruff_cache/`, `node_modules/`, `.next/`, `.cache/`, `.gradle/`, `.terraform/`, language-server caches.
3. **Temp files**: `*.tmp`, `*.swp`, `*~`, `.DS_Store`, `Thumbs.db`, `*.orig`, `*.rej`.
4. **Sensitive-data files**: `.env`, `.env.*` (allow `.env.example` placeholder only — no real secrets even as examples), `*.pem`, `*.key`, `*.crt`, `id_rsa*`, `id_ed25519*`, `.netrc`, `secrets/`, `api_keys.sh`.
5. **Generated reports/logs**: `*.log`, `coverage.out`, `htmlcov/`, runtime captures unless reference assets.
6. **OS/IDE personal state**: `.idea/`, `.history/`, `.vscode/` (except shared settings).

Anti-bluff invariant: `.gitignore` line alone is not sufficient — no file matching the forbidden patterns may be CURRENTLY TRACKED. A tracked `*.log` despite the ignore-line is a violation of equal severity to no ignore-line at all.

Pre-commit attention: every commit author (human OR agent) MUST inspect `git diff --staged` + `git status` BEFORE executing the commit. Forbidden-class hits abort the commit until fixed (un-stage, add to `.gitignore`, scrub if already-tracked). Gate `CM-GITIGNORE-PRECOMMIT-AUDIT` + paired mutation.

Secret-leak intersection (CONST-042 / §11.4.10): a `.env` leak is BOTH a CONST-053 and a CONST-042 violation; rotation + post-mortem required.

Recreatable-content test: if a documented mechanism regenerates the file from sources, it is a build derivative and MUST be ignored. The committed sources MUST include the generator.

Cascade requirement: This anchor (verbatim or by `CONST-053` ID reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. Severity-equivalent to a §11.4 PASS-bluff at the repository-hygiene layer. See constitution submodule `Constitution.md` §11.4.30 for the full mandate.

CONST-054: Submodule-Dependency-Manifest Mandate (cascaded from constitution submodule §11.4.31)

> Verbatim user mandate (2026-05-15): *"We MUST HAVE mechanism for each Submodule to determine / know what are its Submodule dependencies so new projects or palces we are incorporate them can add these Submodules to the project root and make them available! Suggested idea is configuration file*

> *with expected Submodules Git ssh urls perhaps? New project can read it, and recursively add each Submodule to the root of the project and install / expose it to veryone."*

Every owned-by-us submodule MUST ship `helix-deps.yaml` at its root declaring its own-org dependencies. Schema: `schema_version`, `deps: [{name, ssh_url, ref, why, layout: flat|grouped}]`, `transitive_handling.{recursive,conflict_resolution}`, `language_specific_subtree`. Tooling: `incorporate-submodule <ssh-url>` adds the submodule at the parent project's canonical path (CONST-051(C)), reads `helix-deps.yaml`, recurses for each declared dep, aborts on conflicting refs, emits `<root>/.helix-manifest.yaml` audit record.

Anti-bluff guarantee: every manifest paired with a Challenge that bootstraps a throwaway consuming project, runs `incorporate-submodule`, asserts produced layout matches the manifest, runs the submodule's own tests against the bootstrapped layout, captures wire evidence per §11.4.2. A manifest without this proof is a CONST-054 violation.

§11.4.31 / CONST-054 is the **operational complement** of CONST-051(C): nested own-org submodule chains are FORBIDDEN — manifests are the bridge that lets consumers reconstruct the dependency graph at the parent root.

Cascade requirement: This anchor (verbatim or by `CONST-054` ID reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. Severity-equivalent to §11.4 PASS-bluff at the dependency-graph layer. See constitution submodule `Constitution.md` §11.4.31 for the full mandate.

CONST-055: Post-Constitution-Pull Validation Mandate (cascaded from constitution submodule §11.4.32)

> Verbatim user mandate (2026-05-15): *"Every time we fetch and pull new changes on constitution Submodule we MUST process the whole project and all Submodule (deep recursively) for validation and verification taht every single rule or mandatory constraint is followed and respected! If it is not, IT MUST BE!"*

Whenever a project's constitution submodule is fetched + pulled with any content change, the project MUST run `scripts/verify-all-constitution-rules.sh` BEFORE the new constitution HEAD is treated as canonical for any other work. The sweep re-runs the governance-cascade verifier AND every implementable rule gate (CONST-053 `.gitignore` audit, CONST-051(C) nested-own-org-chain audit, CONST-052 case audit, CONST-050(A) mock-from-production audit, CONST-035 anti-bluff smoke, etc.) against the post-pull tree. Failures populate the project's Issues tracker per §11.4.15 (Status: `Reopened`, Type: `Bug`); closure requires positive-evidence per §11.4.

Pull-time invocation: `git submodule update --remote constitution` triggers the sweep automatically (post-update hook OR commit-wrapper invocation). Operator-explicit manual invocation also available.

Anti-bluff: the sweep's own meta-test (paired mutation per §1.1) plants a known violation of each enforced gate and asserts the sweep reports FAIL for the planted gate. A sweep that exits PASS without running every implementable gate is a CONST-055 violation.

CONST-055 is the **enforcement engine** for every other §11.4.x and CONST-NNN rule — without it, new rules cascade as anchors but never get enforced.

Cascade requirement: This anchor (verbatim or by `CONST-055` ID reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. Severity-equivalent to §11.4 PASS-bluff at the constitutional-enforcement layer. See constitution submodule `Constitution.md` §11.4.32 for the full mandate.

CONST-056: Mandatory install_upstreams on clone/add Mandate (cascaded from constitution submodule §11.4.36)

> Verbatim user mandate (2026-05-15): *"Every Submodule or Git repository we add or clone MUST BE upstreams installed using Upstreamable utility which MUST BE available through exported paths of the host system (in .bashrc or .zhrc) using install_upstreams command executed from the root of the cloned (added) repository - only if in it is Upstreams or upstreams directory present with bash script files (recipes) for all repository's upstreams!"*

Every clone / add of a Git repository under HelixCode MUST be followed by `install_upstreams` invocation from the repository's root IF its tree contains `upstreams/` (or legacy `Upstreams/` per CONST-052 transition) populated with `*.sh` recipe files. The utility (installed on operator's `PATH` via `.bashrc`/`.zshrc`; implementation in the constitution submodule's `install_upstreams.sh` — already supports BOTH directory names since constitution commit `45d3678`) reads the recipe files, configures every declared upstream as a named git remote, and fans out `origin` push URLs.

Skipping the invocation when `upstreams/` is present silently breaks §2.1 (multi-upstream push is the norm) — the next push lands on only one upstream. Gate `CM-INSTALL-UPSTREAMS-ON-CLONE` + paired mutation. Automation: the future `incorporate-submodule` per CONST-054 auto-invokes; manual invocation supported. Pre-commit check: `git remote -v | grep -c push` reports expected count.

Cascade requirement: This anchor (verbatim or by `CONST-056` ID reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. See constitution submodule `Constitution.md` §11.4.36 for the full mandate.

CONST-057: Type-aware Closure-Status Vocabulary (cascaded from constitution submodule §11.4.33)

Every project tracking work items by Type per §11.4.16 MUST close them with the Type-appropriate terminal `**Status:**` value, drawn from this 3-element closed map:

Item `**Type:**`	Closure `**Status:**` value
`Bug`	`Fixed (→ Fixed.md)`
`Feature`	`Implemented (→ Fixed.md)`
`Task`	`Completed (→ Fixed.md)`

The `(→ Fixed.md)` suffix is preserved across all three so the existing migration-discipline tooling (atomic Issues.md → Fixed.md move per §11.4.19) keeps working without per-Type branching. Generators (`generate_issues_summary.sh`, `generate_fixed_summary.sh`, the §11.4.23 colorizer) MUST treat the three terminal values as semantically equivalent (all "closed, positive evidence captured") while preserving the literal in the emitted document.

Closing a `Feature` with `Fixed (→ Fixed.md)` or a `Task` with `Implemented (→ Fixed.md)` is a CONST-057 violation. Gate `CM-CLOSURE-VOCAB-TYPE-AWARE` walks every Fixed.md heading + every Issues.md heading whose `**Status:**` is one of the three terminal values and asserts the Status-Type match. Composes with §11.4.15 / §11.4.16 / §11.4.19 / §11.4.23.

Cascade requirement: This anchor (verbatim or by `CONST-057` ID reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. See constitution submodule `Constitution.md` §11.4.33 for the full mandate.

CONST-058: Reopened-Source Attribution Mandate (cascaded from constitution submodule §11.4.34)

Every Issues.md (or equivalent project tracker) heading whose `**Status:**` is `Reopened` MUST carry, within 8 non-blank lines of the heading, a `**Reopened-Details:**` line capturing four sub-facts:

- **By:** `AI` or `User` (source-of-truth observer who flipped the status). `AI` covers in-loop reopens (test failure, gate regression, captured-evidence retrospect). `User` covers operator-side observations (manual testing, end-user report, design reconsideration).
- **On:** ISO date (`YYYY-MM-DD`).
- **Reason:** one-line cause classification — chosen from the closed vocabulary `{ test-failed | manual-testing-detected | captured-evidence-contradicts | end-user-report | cycle-re-discovered | design-reconsidered }`. Other values permitted with explicit `Reason: <free text>` annotation but the closed list MUST be tried first.
- **Evidence:** path to or short description of the captured artefact justifying the reopen — log file, recording, gate failure ID, operator quote, etc. Reopens

without evidence are §11.4.6 / §11.4.7 violations (demotion from Fixed requires captured evidence under the conditions that re-exposed the defect).

The Issues_Summary.md Status column MUST distinguish the four `Reopened` sub-states by source so a sweep query for "reopens by AI in the last 30 days" is mechanically possible. Suggested column rendering: `Reopened (AI: test-failed)` vs `Reopened (User: manual-testing)`. Gate `CM-ITEM-REOPENED-DETAILS` mirrors `CM-ITEM-OPERATOR-BLOCKED-DETAILS` (§11.4.21 walk pattern). Composes with §11.4.6 / §11.4.7 / §11.4.15 / §11.4.21.

Cascade requirement: This anchor (verbatim or by `CONST-058` ID reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. See constitution submodule `Constitution.md` §11.4.34 for the full mandate.

CONST-059: Canonical-Root Inheritance Clarity (cascaded from constitution submodule §11.4.35)

The **constitution submodule's** three files (`constitution/Constitution.md`, `constitution/CLAUDE.md`, `constitution/AGENTS.md`) ARE the **canonical root** (also called the **parent** files). They contain only universal rules per §11.4.17.

The consuming project's **repository-root files** (`<project-root>/CLAUDE.md`, `<project-root>/AGENTS.md`, optionally `<project-root>/Constitution.md`) are **consumer extensions**. They MUST start with the inheritance pointer (either the Claude-Code native `@constitution/CLAUDE.md` import or the portable `## INHERITED FROM constitution/CLAUDE.md` heading). They contain only project-specific rules per §11.4.17.

When in doubt about which file to edit: universal rule → constitution submodule's file; project-specific rule → consumer's file. Default consumer-side when uncertain (§11.4.17 — narrower scope is cheap to widen).

Terminology: "the parent CLAUDE.md" / "the root Constitution" → constitution-submodule file at `constitution/<filename>`; "the project CLAUDE.md" / "this project's AGENTS.md" → consumer-side file at `<project-root>/<filename>`.

No silent demotion or silent promotion. Moving a rule between layers MUST be a visible commit — `git mv` of a section if it's a clean clone, or explicit `Lifted from <project> to constitution per §11.4.35` / `Demoted from constitution to <project> per §11.4.35` commit-message annotation.

Gate `CM-CANONICAL-ROOT-CLARITY` verifies (a) consumer's `CLAUDE.md` opens with the inheritance pointer, (b) constitution submodule's three files are present at the expected path, (c) no `## INHERITED FROM` block in the constitution submodule's own files (those ARE the source-of-truth, not consumers). Composes with §11.4.17.

Cascade requirement: This anchor (verbatim or by `CONST-059` ID reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. See constitution submodule `Constitution.md` §11.4.35 for the full mandate.

CONST-060: Fetch-before-edit Mandate (cascaded from constitution submodule §11.4.37)

> Verbatim user mandate (2026-05-15): *"Make sure that feedback_fetch_before_edit memory rule is part of our constitution Submodule - the root Consitution, AGENTS.MD and CLAUDE.MD. Validate and verify that Proejct-Toolkit and all Submodules do inherit all of them! Follow the constitution Submodule documentation for details."*

The FIRST git-touching action of every session, on every consuming project (owned or third-party), MUST be:

```
git fetch --all --prune
git log --oneline HEAD..@{u}
git submodule foreach --recursive 'git fetch --all --prune --
        quiet'
```

If `HEAD..@{u}` is non-empty, integrate the upstream changes BEFORE any local edit. Acting on stale local state produces three failure modes documented in the originating §11.4.37 incident (multi-agent / parallel-session work): (1) **redundant work** — the agent re-does what a parallel session already finished, (2) **false confidence** — completion reports for already-done work, (3) **divergent history** — duplicate sibling commits that double the conflict surface on next push.

Anti-bluff invariant: the fetch+log check MUST produce captured evidence — the actual `HEAD..@{u}` output, even if empty. Skipping the check on the basis of "I just fetched" or "nothing could have changed in the last N minutes" is a §11.4.6 (no-guessing) violation: the remote state is not knowable without a fetch.

Cascade requirement: This anchor (verbatim or by `CONST-060` ID reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. Severity-equivalent to §11.4 PASS-bluff at the parallel-session-coordination layer. See constitution submodule `Constitution.md` §11.4.37 for the full mandate.

§11.4.52 — Autonomous-Validation Mandate (User mandate, 2026-05-18)

Forensic anchor — verbatim user mandate (2026-05-18): "Make sure we have full automation tests which will do all this work in full automation! IMPORTANT: Make sure that all existing tests and Challenges do work in anti-bluff manner — they MUST confirm that all tested codebase really works as expected! execution of tests and Challenges MUST guarantee the quality, the completition and full usability by end users of the product! This MUST BE part of Constitution of our project, its CLAUDE.MD and AGENTS.MD if it is not there already, and to be applied to all Submodules's Constitution, CLAUDE.MD and AGENTS.MD as well."

Every user-facing feature MUST have at least one autonomous validation path: end-to-end via `adb shell` + scripted automation, captured runtime evidence per §11.4.5, PASS/FAIL verdict WITHOUT human presence to drive UI, observe screen, or make decisions. Operator-attended tests are SUPPLEMENTARY, never PRIMARY. A feature whose ONLY validation path is operator-attended is a §11.4.52 violation — the path does not scale to CI, does not run on every commit, does not survive operator unavailability, and produces the exact "tests pass but feature doesn't work for users" failure mode §11.4 forbids.

Acceptable autonomous paths: (a) programmatic instrumentation APK (SDK-API exercises like `MediaCodec.createDecoderByName` + structured JSON result file); (b) headless intent dispatch + state poll (`am start --es` / `am broadcast` + `dumpsys` / `/proc/<pid>/maps` / `media.metrics` polling); (c) ADB-driven uiautomator (ONLY if hierarchy has ≥1 clickable node — empty hierarchy demands fallback to APK/intent); (d) network-side sink probe per §11.4.13; (e) HelixQA autonomous QA session per §11.4.27.

Coverage ledger (§11.4.25) classifies each feature as `AUTONOMOUS_VERIFIED` / `AUTONOMOUS_DESIGNED` / `OPERATOR_ATTENDED_ONLY` / `NOT_APPLICABLE`. `OPERATOR_ATTENDED_ONLY` blocks release until migrated; cite tracked work item per §11.4.15 + §11.4.16. Autonomous paths themselves MUST be anti-bluff: positive captured evidence + paired meta-test mutation per §1.1.

Composes with §11.4.25 (full-automation coverage), §11.4.27 (no-fakes + 100% type coverage), §11.4.39 (per-feature on-device end-user validation), §11.4.43 (TDD RED-first), §11.4.48 (UI-driven — fallback to APK/intent when uiautomator hierarchy empty), §11.4.49 (dual-approach), §11.4.50 (deterministic consistency), §11.4.51 (live-ADB-first).

Pre-build gates: `CM-COVENANT-114-52-PROPAGATION` + `CM-AF-AUTONOMOUS-PATH-PER-FEATURE`. Paired mutations. No escape hatch — no `--allow-operator-attended-only`, `--skip-autonomous-path`, `--manual-validation-suffices` flag.

Canonical authority: constitution submodule Constitution.md §11.4.52.

Non-compliance is a release blocker regardless of context.

CONST-061: Pre-Force-Push Merge-First Mandate (cascaded from constitution submodule §11.4.41)

> Verbatim user mandate (2026-05-17): *"make sure we bring everything from branches to our side before forc push is done! Afer everything is safely and fully merged and all potential conflicts (if any) resolved, then do force push! make sure nothing isnlost, broken or corrupted on bith sides! add these rules in our root Constitution, CLAUDE.MD, AGENTS.MD (constitution Submodule) if itnis not added already! Extremely important rules and mandatory constraints we MUST HAVE and fully respect!"*

Any force-push (`--force`, `--force-with-lease`, `+<ref>`, equivalent history-rewrite) authorised under CONST-043 MUST be preceded by a mechanical 4-step merge-first pipeline:

1. **Fetch every remote** — `git fetch --all --prune --tags` against origin + every upstream; capture output.
2. **Integrate every divergent commit locally** — rebase / merge / operator-confirmed cherry-pick per appropriate strategy for every non-empty `HEAD..<remote>/<branch>` range.
3. **Audit the integrated tree** — no conflict markers anywhere (`grep -rn '^<<<<<<< \|^=======$\|^>>>>>>> '` returns empty in governance + source + test files); no file silently dropped; previously-passing tests still pass; captured-evidence artefacts still validate.

4. **Force-push** — only after steps 1-3 produce clean integration evidence: `git push --force-with-lease` (NEVER `--force` alone unless authorised per §9.2 sub-clause 6).

Two-gate composition with CONST-043. §11.4.41 does NOT relax CONST-043's operator-approval requirement — it adds a SECOND mechanical gate. CONST-043 alone authorises a push that loses remote work; §11.4.41 alone risks pushing without operator awareness. Both required.

Three failure modes prevented: (a) remote-side content loss when parallel sessions land work between fetches; (b) stale-state acts when `--force-with-lease` reads stale local refs without prior fetch; (c) conflict-driven corruption when markers get committed verbatim (observed 2026-05-17 in helix_qa + containers governance files).

Verification artefact: every governed force-push emits a `docs/changelogs/<tag>.md` "Force-push merge-first audit" section capturing fetch output, per-remote divergence log, integration strategy, conflict-marker scan, test delta, push output with lease SHA, + CONST-043 authorisation quote. Gate `CM-FORCE-PUSH-MERGE-FIRST` + paired mutation.

Cascade requirement: This anchor (verbatim or by `CONST-061` ID reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. Severity-equivalent to a §11.4 PASS-bluff at the remote-data-integrity layer. See constitution submodule `Constitution.md` §11.4.41 for the full mandate.

§11.4.53 — Fixed_Summary parity mandate (User mandate, 2026-05-18)

Forensic anchor — verbatim user mandate (2026-05-18T17:55Z): "Note: Just like for Issues we have Issues_Summary, for Fixed we MUST HAVE Fixed_Summary - like all other docs: ALWAYS in sync and up to date and ALWAYS exported into the PDF and HTML! Add this mandatory rule / constraint into the root (constitution Submodule) Constitution, AGENTS.MD and CLAUDE.MD."

`docs/Fixed_Summary.md` is the symmetric short-form summary of `docs/Fixed.md`. MUST be regenerated whenever `Fixed.md` changes. HTML + PDF exports MUST travel with the markdown (identical mtimes within `sync_issues_docs.sh` granularity). Stale exports are §11.4.53 violations regardless of whether the underlying `.md` is correct. Same discipline as §11.4.12 Issues_Summary applied to Fixed.md.

Generator: `scripts/testing/generate_fixed_summary.sh` (canonical, executable, emits markdown table with `Status` + `Type` columns per §11.4.19 column-alignment). Auto-sync wrapper: `scripts/testing/sync_issues_docs.sh` regenerates BOTH summaries in one shot, exports HTML + PDF, colorizes per §11.4.23, re-renders PDFs. MUST be invoked after any edit to `Fixed.md`. No `--issues-only` flag exists, and §11.4.53 prohibits adding one.

Sort order: closure date DESC (most-recent-Fixed first), §-letter / Fix-# secondary. Documented at the top of the generated file.

Composes with §11.4.12 (Issues_Summary sibling — canonical pair), §11.4.19 (atomic Issues→Fixed migration trigger + column-alignment), §11.4.23 (colorizer post-processes both summaries), §11.4.33 (type-aware closure vocabulary —

Fixed_Summary respects `Fixed (→ Fixed.md)` / `Implemented (→ Fixed.md)` / `Completed (→ Fixed.md)` terminal values), §11.4.44 (revision header applies to `Fixed_Summary.md`), §12.10 (CONTINUATION.md resumption guarantee).

Pre-build gates: `CM-FIXED-SUMMARY-SYNC` (6 invariants — Fixed_Summary exists + HTML/PDF mtime ≥ md mtime + Fixed_Summary mtime ≥ Fixed mtime + generator + sync wrapper invokes generator) + `CM-COVENANT-114-53-PROPAGATION` (anchor literal across canonical files). Paired mutations strip the anchor literal AND move the generator aside AND backdate Fixed_Summary mtime. No escape hatch — no `--skip-fixed-summary-sync`, `--issues-only`, `--summary-not-applicable` flag.

Canonical authority: constitution submodule Constitution.md §11.4.53.

Non-compliance is a release blocker regardless of context.

CONST-062: README always-sync mandate (cascaded from constitution submodule §11.4.59)

> Verbatim user mandate (2026-05-19): *"fully review and update our main README document. Some points are not valid anymore, some are missing. Make sure main README is among documents we MUST ALWAYS keep updated and in Sync with the projects and other documentation! Make sure we always export it (on every update) into PDF and HTML. This mandatory rules/constraints MUST BE all added into the root (constitution Submodule) Constitution, AGENT.MD and CLAUDE.MD!"*

`README.md` at the project root is a §11.4.12-class always-sync document. It MUST be (1) reviewed and updated whenever new docs / integrations / Status.md entries appear, new submodules land, applied-fixes count changes, or canonical paths shift; (2) kept in lockstep with `docs/CONTINUATION.md` (§12.10) and the Issues / Issues_Summary / Fixed / Fixed_Summary doc set; (3) exported to `.html` and `.pdf` on every update via `scripts/testing/sync_readme_export.sh` (pandoc + weasyprint); (4) carry a §11.4.44 revision header; (5) contain a Documentation Map section linking to every Status.md + Status_Summary.md + spec + plan + guide + script-companion doc + changelog + the constitution submodule, plus per-audience navigation; (6) self-contained (no hyperlinks to ephemeral external systems as the only source of truth). Pre-build gate `CM-README-EXPORT-SYNC` locks four invariants (README.md exists, README.html exists, README.html mtime ≥ README.md mtime, README.pdf mtime ≥ README.md mtime). Paired meta-test mutation backdates HTML+PDF → gate FAILs. No escape hatch — no `--skip-readme-sync`, `--no-readme-export`, `--readme-stale-OK` flag.

Cascade requirement: This anchor (verbatim or by `CONST-062` ID reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. See constitution submodule `Constitution.md` §11.4.59 for the full mandate.

CONST-063: Documentation always-sync composite covenant (cascaded from constitution submodule §11.4.60)

> Verbatim user mandate (2026-05-19 ~09:00Z): *"Double check if all documents are properly tied with our root Constitution, CLAUDE.MD and AGENTS.MD so they are always up to date, always in sync and exported into PDF and HTML! ... Issues, Issues_Summary, Fixed, Fixed_Summary, Continuation, Status and Status_Summary for all contexts (areas) — THEY ALL MUST BE REGULARLY UPDATED, IN SYNC AND CONSISTENT without giving at any moment false picture about the state of the project or particular area(s) of it!"*

Eight documentation classes constitute the project's living state surface and MUST be in sync at all times across `.md` + `.html` + `.pdf` artefacts: (1) `docs/Issues.md`, (2) `docs/Issues_Summary.md`, (3) `docs/Fixed.md`, (4) `docs/Fixed_Summary.md`, (5) `docs/CONTINUATION.md`, (6) `README.md`, (7) every `docs/**/Status.md` (domain-scoped), (8) every `docs/**/Status_Summary.md` (domain-scoped). Per-class anchors §11.4.12 / §11.4.44 / §11.4.45 / §11.4.53 / §11.4.56 / §11.4.57 / §11.4.59 / §12.10 each govern individually; §11.4.60 binds them via single composite gate `CM-DOCS-COMPOSITE-SYNC` (pre-build) that FAILs the build if ANY single instance's `.html` or `.pdf` mtime is older than `.md` mtime. Walks `docs/` recursively for the Status fleet. Paired mutation backdates `docs/Issues.html` to year 2000 → gate FAILs. No escape hatch — no `--skip-composite-doc-sync`, `--allow-stale-html`, `--summary-not-applicable` flag exists.

Cascade requirement: This anchor (verbatim or by `CONST-063` ID reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. See constitution submodule `Constitution.md` §11.4.60 for the full mandate.

§11.4.66 — Blocker-resolution interactive-clarification mandate (User mandate, 2026-05-19)

When any task is blocked (operator decision, hardware access, external authorization, ambiguous scope), the agent MUST: (1) research what's doable from the agent side without operator input; (2) calculate minimum-viable operator input; (3) construct 2–4 mutually-exclusive options with one marked "Recommended" and each stating what the agent does after that answer; (4) present via the platform's interactive question mechanism (`AskUserQuestion` on Claude Code) — NEVER free-text "what would you like?" for closed- set decisions; (5) after the answer, resume work without follow-up round-trips. Composes with §11.4.6 / §11.4.7 / §11.4.40 / §11.4.41 / §11.4.42 / §11.4.52. No silent waiting; no bulk-text questions when interactive options would do.

Pre-build gate `CM-COVENANT-114-66-PROPAGATION` enforces the anchor literal across the 42-file consumer fleet. Paired meta- test mutation strips the literal → gate FAILs. No escape hatch — no `--skip-ask`, `--silent-wait`, `--free-form-only` flag.

Canonical authority: constitution submodule `Constitution.md` §11.4.66.

Non-compliance is a release blocker regardless of context.

CONST-064: Mandatory Markdown metadata table + structured-doc ToC (cascaded from constitution submodule §11.4.61)

> Verbatim user mandate (2026-05-19): *"For every Markdown document which contains structured content (with headings / sections and sub-sections) make sure that every time we apply change to the structure, table of contents on the top of the document is created or updated! This is MANDATORY for every structured MARKDOWN document. Automatically its PDF and HTML versions MUST BE (re)generated! Introduce ... revision number, date and time of creation, date and time of last modification, other useful information we have in documents such as Issues, Issues_Summary, Fixed, Fixed_Summary, Status, Status_Summary, Continuation and similar. ... make this mandatory for EVERY Markdown document from now on, update root constitution Submodule with these changes and commit and push it all to all upstreams."*

> Verbatim 2026-05-19 operator mandate: *"all existing tests and Challenges do work in anti-bluff manner - they MUST confirm that all tested codebase really works as expected! We had been in position that all tests do execute with success and all Challenges as well, but in reality the most of the features does not work and can't be used! This MUST NOT be the case and execution of tests and Challenges MUST guarantee the quality, the completition and full usability by end users of the product!"*

Every tracked `*.md` in the §11.4.65 INCLUDED set MUST carry, immediately below the H1 title, a canonical Markdown metadata table with four MANDATORY rows (`Revision` — monotonic positive integer; `Created` — ISO 8601 date; `Last modified` — ISO 8601 date; `Status` — `active`/`draft`/`deprecated`/`superseded`) plus ENCOURAGED rows (`Status summary`, `Issues`, `Issues summary`, `Fixed`, `Fixed summary`, `Continuation`) rendered with `–`/`none` when N/A. Any tracked `*.md` with **two or more H2 sections** MUST include a `## Table of contents` section immediately after the metadata table; the ToC MUST list every H2/H3 in document order with anchor links and MUST be regenerated on every structural change. A stale ToC is a §11.4 PASS-bluff. Anti-bluff gates `CM-MD-METADATA-PRESENT` + `CM-MD-TOC-PARITY` with paired mutations.

Cascade requirement: This anchor (verbatim or by `CONST-064` ID reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. See constitution submodule `Constitution.md` §11.4.61 for the full mandate.

CONST-066: Universal Markdown export mandate (cascaded from constitution submodule §11.4.65)

> Verbatim user mandate (2026-05-19): *"Any markdown document inside the project and which is not part of the applications or services source code MUST BE exported (be available) in PDF and HTML! Any already existing*

> *Markdown document that fulfills this condition and which does not have HTML or PDF at all or it is not in sync with it MUST HAVE (re)generated PDF and HTML version! Every time when Markdown document (file) is modified, its proper HTML and PDF versions MUST BE regenerated. Markdown documents MUST BE at all times in sync with PDF and HTML versions!"*

> Verbatim 2026-05-19 operator mandate: *"all existing tests and Challenges do work in anti-bluff manner - they MUST confirm that all tested codebase really works as expected! We had been in position that all tests do execute with success and all Challenges as well, but in reality the most of the features does not work and can't be used! This MUST NOT be the case and execution of tests and Challenges MUST guarantee the quality, the completition and full usability by end users of the product!"*

Every Markdown document inside the project that is NOT part of an application or service's source-code tree MUST have synchronized `.html` and `.pdf` siblings, all three artefacts in sync at all times. INCLUDED scope: project-root `*.md`, `docs/**/*.md`, `scripts/**/*.md` (companion docs), owned-submodule trees' README/CLAUDE/AGENTS/CHANGELOG + their `docs/**/*.md`, `constitution/**/*.md`, owned HelixQA dependencies' equivalents. EXCLUDED: `external/**`, `prebuilts/**`, `packages/modules/**`, `kernel-*/**`, `out/**`, `build/**`, application/service source-code trees, third-party submodules NOT owned. Mandatory: (1) every INCLUDED `.md` has `.html` + `.pdf` siblings; (2) `.html`/`.pdf` mtime ≥ `.md` mtime; (3) every modification triggers regeneration via `scripts/testing/sync_all_markdown_exports.sh`; (4) pre-build gate `CM-UNIVERSAL-MARKDOWN-EXPORT-SYNC` FAILs build if any out-of-sync. Canonical helper: pandoc (HTML) + weasyprint (PDF), `timeout 60` per file, supports `--check-only` and `--regenerate-all` modes, caps at 500 candidates. No escape hatch — no `--skip-md-exports`, `--no-pdf-only`, `--md-export-not-applicable` flag.

Cascade requirement: This anchor (verbatim or by `CONST-066` ID reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. See constitution submodule `Constitution.md` §11.4.65 for the full mandate.

CONST-068: Shell-script target-shell-parseability mandate (cascaded from constitution submodule §11.4.67)

> Verbatim user mandate (2026-05-19): *"any issue we spot must be fixed, bash scripts as well if they are broken!"* + *"Make sure that this is mandatory rule!"*

> Verbatim 2026-05-19 operator mandate: *"all existing tests and Challenges do work in anti-bluff manner - they MUST confirm that all tested codebase really works as expected! We had been in position that all tests do execute with success and all Challenges as well, but in reality the most of the features does not work and can't be used! This MUST NOT be the case and execution of tests and Challenges MUST guarantee the quality, the completition and full usability by end users of the product!"*

Every committed shell script MUST be parseable by its target interpreter (`sh -n` for `/bin/sh`, `bash -n` for `/bin/bash`, etc.) AND MUST declare a shebang matching its actual syntax usage. Bash-only constructs (`>(...)`, `<(...)`, `[[ ]]`, `<<<`, arrays, `${var^^}`, etc.) used in scripts that may be invoked via `sh script.sh` MUST be wrapped in `eval` so the parser sees only a string (target shells like mksh parse the entire script before executing — runtime guards cannot save a parse-time rejection). Honest shebangs only: `#!/bin/bash` only if bash actually expected; `#!/bin/sh` requires POSIX-clean body. Fix at source per §11.4.1, never at callsites. Composes with §11.4.1 / §11.4.4 / §11.4.6 / §11.4.50 / §11.4.51. Pre-build gate `CM-SCRIPT-TARGET-SHELL-PARSEABLE` runs `sh -n` on every in-scope script. No escape hatch — no `--skip-parseability-check`, `--bash-only-script`, `--runtime-guard-suffices` flag.

Cascade requirement: This anchor (verbatim or by `CONST-068` ID reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. See constitution submodule `Constitution.md` §11.4.67 for the full mandate.

§11.4.67 — Shell-script target-shell-parseability mandate (User mandate, 2026-05-19)

Forensic anchor — direct user mandate (verbatim, 2026-05-19): "any issue we spot must be fixed, bash scripts as well if they are broken!" + "Make sure that this is mandatory rule!"

Every shell script that may be invoked under a target shell other than the one in its shebang MUST parse cleanly under that target shell. Forensic incident: `device/rockchip/rk3588/tests/test_all_fixes.sh:114` used bash-only `exec > >(tee -a "$f") 2>&1` on a `sh script.sh` callsite — Android mksh parses the whole script BEFORE executing, so the runtime `[ -n "${BASH_VERSION:-}" ]` guard could not save it. Fixed by wrapping in `eval 'exec > >(tee …) 2>&1'` so the parser sees only a string.

Closed-set scope: every tracked `.sh` under `device/rockchip/rk3588/tests/`, `scripts/`, `scripts/testing/` (and equivalent paths in owned submodules). OUT of scope: `external/`, `prebuilts/`, `packages/modules/`, `kernel-5.10/`, `out/`, `build/`, `scripts/legacy/`. Mandatory invariants: (1) every in-scope script parses under `sh -n`; (2) bash-only constructs (`>(...)`, `<(...)`, `[[ ]]`, `<<<`, arrays, `${var^^}`, etc.) MUST be wrapped in `eval` OR guarded by bash-only loading; (3) shebangs honest — `#!/bin/bash` only if bash actually expected; (4) fix at source per §11.4.1, never at callsites. Composes with §11.4.1 / §11.4.4 / §11.4.6 / §11.4.50 / §11.4.51.

Pre-build gate `CM-SCRIPT-TARGET-SHELL-PARSEABLE` runs `sh -n` on every in-scope script. Propagation gate `CM-COVENANT-114-67-PROPAGATION` enforces the anchor literal across the 44-file consumer fleet. Paired mutations: inject bash-only outside `eval` → parse gate FAILs; strip `11.4.67` literal → propagation gate FAILs. No escape hatch — no `--skip-parseability-check`, `--bash-only-script`, `--runtime-guard-suffices` flag.

Canonical authority: constitution submodule `Constitution.md` §11.4.67.

Non-compliance is a release blocker regardless of context.

§11.4.69 — Universal sink-side positive-evidence taxonomy + mechanical enforcement (User mandate, 2026-05-20)

Forensic anchor — direct user mandate (verbatim, 2026-05-20):

> "THIS MUST HAPPEN NEVER AGAIN!!! We MUST HAVE this all working! Not just for audio but for every single piece of the System!!! Proper full automation when executed with success MUST MEAN that manual testing will be as much positive at least regarding the success results! … Solution MUST BE universal, generic that solves working flows for all System components and for all future and all existing projects! … Everything we do MUST BE validated and verified with rock-solid proofs and anti-bluff policy enforcement and fulfillment!"

Universal generalisation of §11.4.68 (audio-specific) across every user-visible feature class. Closes the PASS-bluff pattern where tests reported green while end users hit broken features (2026-05-19→20 D3 audio "82/84 PASS" + empty Arvus Codec-In-Use).

The mandate. Every user-visible feature MUST map to one entry in the closed-set §11.4.69 sink-side evidence taxonomy (audio_output, audio_input, video_display, network_throughput, network_connectivity, bluetooth_a2dp, bluetooth_pair, touch_input, sensor, gpu_render, storage_read, storage_write, mediacodec_decode, mediacodec_encode, miracast, cast, boot_service, package_install, permission_grant, wifi_link, wifi_throughput, ethernet_link, display_topology, drm_playback, subtitle_render — open to additions). Every PASS for a feature in the taxonomy MUST cite a captured-evidence artefact path matching the required evidence shape.

Helper contracts (additive during grace; mandatory after 2026-06-19):

- `ab_pass_with_evidence <description> <evidence_path>` — the new canonical PASS helper. Verifies path exists AND non-empty; emits `PASS: <description> [evidence: <path>]`.
- `ab_skip_with_reason <description> <closed-set-reason>` — reasons: `geo_restricted`, `operator_attended`, `hardware_not_present`, `topology_unsupported`, `network_unreachable_external`, `feature_disabled_by_config`. Forbids `network_unreachable_external` for any taxonomy feature with a sink-side probe.
- Bare `ab_pass` deprecated — WARN pre-grace, FAIL post-grace (2026-06-19).

Mechanical enforcement. Three pre-build gates + three paired §1.1 meta-test mutations:

- `CM-SINK-EVIDENCE-PER-FEATURE` — walks tests for `# §11.4.69 FEATURE: <class>` annotation + verifies taxonomy probe + `ab_pass_with_evidence` use.
- `CM-NO-FAIL-OPEN-SKIP` — audits sink-side probe helpers; FAILs if any code path converts empty/unreachable response to PASS-counting SKIP for a feature class with a sink-side probe.
- `CM-AB-PASS-WITH-EVIDENCE-EVERYWHERE` — pre-grace WARN, post- grace FAIL on bare `ab_pass` calls.

Composes with §11.4.1 (FAIL-bluffs forbidden), §11.4.2 (recorded-evidence), §11.4.5 (audio + video 5-layer quality), §11.4.6 (no-guessing), §11.4.13 (sink-side captured-evidence), §11.4.27 (no-fakes-beyond-unit), §11.4.50 (deterministic consistency), §11.4.52 (autonomous-validation), §11.4.68 (audio-specific sink-side — §11.4.69 is the universal generalisation).

No escape hatch — no `--skip-evidence`, `--config-only-pass`, `--allow-fail-open-skip`, `--legacy-ab-pass-permitted` flag. The discipline exists because the 2026-05-20 forensic incident demonstrated the failure: tests reported audio-routing PASS while the user heard nothing and the Arvus Codec-In-Use field was empty.

Propagation gate `CM-COVENANT-114-69-PROPAGATION` enforces this anchor literal across the ~44-file consumer fleet. Paired mutation strips the literal → gate FAILs.

Canonical authority: constitution submodule `Constitution.md` §11.4.69.

Non-compliance is a release blocker regardless of context.

§11.4.68 — Positive Sink-Side / Downstream Evidence Mandate (cascaded from constitution submodule §11.4.68)

> Verbatim user mandate (2026-05-20): *"We still do not hear any audio played from D3 device! Arvus Web Dashboard when we play music from D3 shows nothing for Codec In Use! This MUST BE investigated and fixed! How come we passed the tests with Arvus validation? What were values for the Codec In Use field? Empty means nothing! This is not working! It MUST BE FIXED, TESTED AND VERIFIED WITH FULL AUTOMATION TESTING ASAP!!!"*

A test that asserts audio or video routing PASS MUST capture and verify **positive sink-side or downstream evidence** — never config-only, never metadata-only, never PCM-open-state-only. At least one of the closed enumeration MUST be captured for every audio/video routing PASS: (1) sink-side codec-state with non-empty Codec-In-Use matching the expected codec regex; (2) strictly-positive PCM frames-written delta from `/proc/asound/.../status hw_ptr`; (3) ALSA ELD/EDID-Like-Data showing negotiated channel count + format; (4) ffprobe-on-captured-mp4 with non-zero frame count + expected codec/resolution/fps; (5) recording-analyzer event match per §11.4.2/§11.4.5; (6) tinycap RMS amplitude above the line-level floor. Empty / `<unreachable>` / `<N.E.>` / `<None>` placeholders are NOT positive evidence; a missing-but-required sink is `OPERATOR-BLOCKED` (release-blocker), never SKIP, never PASS. No escape hatch — no `--skip-sink-evidence`, `--allow-empty-codec`, `--sink-unreachable-is-pass`, `--metadata-only-suffices` flag exists.

Cascade requirement: This anchor (verbatim or by `§11.4.68` reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. Severity-equivalent to a §11.4 PASS-bluff at the sink-side-evidence layer. **Canonical authority:** constitution submodule `Constitution.md` §11.4.68 for the full mandate.

§11.4.70 — Subagent-Driven Execution Is The Default (cascaded from constitution submodule §11.4.70)

> Verbatim user mandate (2026-05-20): *"Always do if possible Subagent-driven! Add this into our root (constitution Submodule) Constitution.md, CLAUDE.md and AGENTS.md. This should be the default choice ALWAYS!"*

When executing implementation plans (or any task-decomposed execution flow), the **default execution model is subagent-driven** per `superpowers:subagent-driven-development`. Inline execution is permitted ONLY when (a) the task is trivial AND fits a single sub-300-line edit, OR (b) the operator explicitly requests inline at brainstorm-handoff time. Subagent-driven is the default because it gives isolated context per task, naturally enforces two-stage review, is parallel-PWU compatible (§11.4.58), creates an anti-bluff seam (§11.4), and survives operator absence. No escape hatch — `--inline-execution-required`, `--no-subagents`, `--monolithic-execution` are NOT permitted flags. Skipping subagent-driven for non-trivial work without recorded operator authorisation is itself a §11.4 PASS-bluff.

Cascade requirement: This anchor (verbatim or by `§11.4.70` reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. Severity-equivalent to a §11.4 PASS-bluff at the execution-model layer. **Canonical authority:** constitution submodule `Constitution.md` §11.4.70 for the full mandate.

§11.4.71 — Pre-Push Fetch + Investigate + Integrate Mandate (cascaded from constitution submodule §11.4.71)

> Verbatim user mandate (2026-05-20): *"before pushing changes to any upstream for any repository - main repo or Submodule, we MUST fetch and pull all changes. Once these are obtained WE MUST investigate what is different compared to head position we were on last time before fetching and pulling new changes! We MUST understand what is done and for what purpose, easpecially how that does affect our project and our System in general! Any mandatory changes or improvements required by fresh changes we just have brough in MUST BE incorporated, covered with all supported types of the tests which will produce as a result of its success execution REAL PROOFS of working for all componetns and functionalities covered and work fully in anti-bluff manner!"*

The everyday-push variant of §11.4.41. EVERY push (every repository — main + every submodule) MUST follow the 5-step cycle: (1) fetch all remotes (`git fetch --all --prune --tags`, capture stdout); (2) pull all upstream branches whose tip differs, resolving conflicts per consumer judgment (never auto-`--ours`/`--theirs`); (3) investigate the diff vs OUR previous HEAD — read EVERY foreign commit's body, understand what/why/how-it-affects-our-system; (4) integrate mandatory changes with §11.4.4(b) four-layer coverage + §11.4.43 TDD-fix discipline, every

PASS carrying §11.4.5 captured-evidence (REAL PROOFS, not metadata-only); (5) only then push, verifying with `git ls-remote` post-push. No escape hatch — no `--skip-fetch`, `--no-investigate`, `--fast-push`, `--trust-upstream` flag.

Cascade requirement: This anchor (verbatim or by `§11.4.71` reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. Severity-equivalent to a §11.4 PASS-bluff at the push-discipline layer. **Canonical authority:** constitution submodule `Constitution.md` §11.4.71 for the full mandate.

§11.4.72 — Audio Top-Priority Mandate (cascaded from constitution submodule §11.4.72)

> Verbatim user mandate (2026-05-20): *"Make sure all fixes for audio are always top priority in main working stream!"*

The conductor (main working stream — Claude Code session, AI agent, or human operator) MUST treat audio fixes as the highest-priority class on the serial dispatch queue. Any time the conductor faces a choice between dispatching an audio task vs a non-audio task on the SAME serial resource, the audio task wins. Parallel BACKGROUND subagents (research, refactors, infrastructure documentation) MAY run concurrently with audio work but do NOT preempt audio on the main-stream serial dispatch queue. No escape hatch — there is no "but this non-audio task is faster" or "but this research is more interesting" override; audio-stack regressions are user-perceptible and high-impact while research and refactors can wait.

Cascade requirement: This anchor (verbatim or by `§11.4.72` reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. Severity-equivalent to a process violation at the dispatch-priority layer. **Canonical authority:** constitution submodule `Constitution.md` §11.4.72 for the full mandate.

§11.4.73 — Main-Specification Document Versioning + Revision Discipline (cascaded from constitution submodule §11.4.73)

> Verbatim user mandate (2026-05-20): *"Make sure everything we add now in previous and upcoming requests IS ALWAYS applied to the main specification — if we have one. Since all these are not major changes we could increase Specification version per change for secondary version instead of the primary. Primary version MUST BE increased for much bigger levels of changes! Add this into root (constitution Submodule) Constitution.md, CLAUDE.md and AGENTS.md as mandatory rule / constraint applicable ONLY IF we have something like the main specification document or we do recognize something like the main specification document. Document MUST BE updated ALWAYS to follow the versioning rules we are appling here + revision and other properties we have!"*

Applies **only when a project recognises a main specification document**. When it does: (1) every additive operator requirement, refinement, or accepted recommendation MUST be applied to the spec before or as part of the implementing work; (2) spec versioning has two axes — *primary* (V1/V2/V3, bumped for major rewrites by explicit operator decision, old versions archived) and *secondary* (the §11.4.61 metadata-table `Revision` integer, bumped for every other change); (3) the metadata table MUST stay current (`Revision`, `Last modified`, `Status summary`, `Fixed`); (4) propagated copies of the rule MUST reference the active `specification.V<primary>.md`, not a stale archive; (5) on primary bump the old file moves to `<spec-dir>/archive/` with `Status: superseded`. Classification: universal, applicable conditionally per the scope condition.

Cascade requirement: This anchor (verbatim or by `§11.4.73` reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. Severity-equivalent to a release blocker when a project has a main spec and lets it drift. **Canonical authority:** constitution submodule `Constitution.md` §11.4.73 for the full mandate.

§11.4.74 — Submodule-Catalogue-First Discovery + Extend-Don't-Reimplement (cascaded from constitution submodule §11.4.74)

> Verbatim user mandate (2026-05-20): *"We MUST ALWAYS check which already developed features / functionalities do exist as a part of our comprehensive Submodules catalogue located in vasic-digital and HelixDevelopment organizations on GitHub and GitLab both! Project MUST BE aware of all its existence so we do not implement same things multiple times if they are already done as some of existing universal, reusable general development purpose Submodules! For any missing features that some Submodules we incorporate may be missing we MUST IMPLEMENT the properly and extend those Submodules furter! We do control all of the and we CAN and MUST maintain and extend the regularly! All development cycle rules we have MUST BE applied to them and fully respected!"*

Before scaffolding ANY new module, package, helper, or utility, the contributor (human or AI agent) MUST: (1) survey the canonical Submodule catalogue — `vasic-digital` and `HelixDevelopment` on both GitHub AND GitLab; (2) inventory existing Submodules; (3) reuse before reimplement — if a Submodule provides the functionality (or 80%+ of it), add it as a Git submodule rather than write fresh; (4) extend in-place when 80%+ matches but features are missing — add the missing features TO THAT SUBMODULE (PR upstream + bump pointer), never as a duplicating consuming-project helper; (5) apply all development-cycle rules to those Submodules; (6) document the survey result in the feature's tracker entry with a `Catalogue-Check:` field (`reuse <org/repo>@<sha>` / `extend <org/repo>@<sha>` / `no-match <date>`). Classification: universal.

Cascade requirement: This anchor (verbatim or by `§11.4.74` reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. Severity-equivalent to a process violation; duplicate implementations landed without catalogue check are release blockers. **Canonical authority:** constitution submodule `Constitution.md` §11.4.74 for the full mandate.

§11.4.78 — CodeGraph code-intelligence mandate (cascaded from constitution submodule)

Inherited from `constitution/Constitution.md` §11.4.78. Every project worked on by AI coding agents — and every owned submodule when developed standalone — MUST install, initialize, and use **CodeGraph** (`https://github.com/colbymchenry/codegraph`, npm package `@colbymchenry/codegraph`): a local SQLite semantic code-knowledge-graph exposed to AI agents over the Model Context Protocol (MCP), 100% local with no cloud or external API. Install globally via npm (no `sudo` — the npm prefix MUST be user-writable). Run `codegraph init` + `codegraph index`: `.codegraph/config.json` is tracked; `.codegraph/codegraph.db` is gitignored with `codegraph index` as its §11.4.77 regeneration mechanism; the `config.json` `exclude` list MUST exclude other-owned submodules and — non-negotiably — every §11.4.10 credential/secret path. Wire the `codegraph serve --mcp` MCP server into every CLI agent the developers use (Claude Code `.mcp.json`, OpenCode `opencode.json`, Qwen Code `.qwen/settings.json`, Crush `.crush.json`, Kimi CLI `~/.kimi/mcp.json`); every config references the bare `codegraph` command on `PATH`. Cover the integration with an anti-bluff verification suite whose per-agent end-to-end layer uses an unforgeable challenge (a fact obtainable only by calling a CodeGraph MCP tool); un-runnable agents are documented SKIP gaps per §11.4.3, never faked PASSes. Document everything in `docs/CODEGRAPH.md`.

Cascade requirement: this anchor (verbatim or by §11.4.78 ID reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, `AGENTS.md`, and `QWEN.md`. See the constitution submodule `Constitution.md` §11.4.78 for the full mandate. Non-compliance is a process violation.

§11.4.85 — Stress + Chaos Test Mandate (User mandate, 2026-05-24)

Forensic anchor — direct user mandate (verbatim, 2026-05-24):

> "Every fix or improvement you do MUST BE covered with full automation stress and chaos tests so we are sure nothing can break the functionality and all edge cases are monitored and polished and additionally fixed if that is needed! Everything must produce rock solid proofs and follow fully no-bluff policy!"

Every fix or improvement landed in this project MUST ship with full-automation **stress** AND **chaos** test suites that exercise edge cases, sustained load, concurrent contention, and failure-injection. Happy-path coverage alone is a §11.4 / §107 PASS-bluff at the resilience layer.

Stress (closed-set, mechanically auditable): sustained load (N ≥ 100 iterations OR ≥ 30 s wall-clock; per-iteration latency p50/p95/p99 recorded) + concurrent contention (N ≥ 10 parallel invocations; no deadlock, no resource leak) + boundary conditions (empty / max / off-by-one input; every boundary produces a categorised result, never an uncaught exception).

Chaos (closed-set, applied per fix-class appropriateness): process-death injection (kill primary or upstream mid-call; categorised recovery) + network-fault injection (drop/delay/reorder; `category=network|upstream` per §11.4.69) + input-corruption injection (corrupt .env / config / input file mid-test; detected + reported)

+ resource-exhaustion injection (disk full, OOM, FD exhaustion; refuse cleanly OR degrade gracefully — NEVER crash) + state-corruption injection (mid-flight lock loss, partial-write fault; recovery restores consistent state).

Anti-bluff (mandatory). Every stress + chaos test PASS cites a captured-evidence artefact path per §11.4.5 + §11.4.69 (per-iteration `latency.json`, `categorised_errors.txt`, `state_delta_snapshot.json`, `recovery_trace.log`). Helper library `stress_chaos.sh` provides `ab_stress_run`, `ab_stress_concurrent`, `ab_chaos_kill_pid_during`, `ab_chaos_drop_network_during`, `ab_chaos_corrupt_file_during`, `ab_chaos_oom_pressure_during`, `ab_chaos_disk_full_during`, each composing with `ab_pass_with_evidence` / `ab_skip_with_reason`. Chaos-injection cleanup is non-negotiable — corrupt-restore, disk-fill-cleanup, process-restart MUST run in `trap '...' EXIT`; cleanup failure = §11.4.14 violation.

4-layer coverage per §11.4.4(b): pre-build gate (stress + chaos test files exist + executable + parseable under sh -n + bash -n per §11.4.67; helper library exists; the fix's pre-build gate cites the stress + chaos test file path) + paired meta-test mutation per §1.1 (stripping chaos-injection or per-iteration evidence capture → gate FAILs) + on-device test (if LIVE_ADB_TESTABLE per §11.4.51, dispatched against real device, evidence under `qa-results/<run-id>/stress_chaos/`) + HelixQA Challenge entry (if user-visible feature per §11.4.4(b) layer 4).

Composes with §11.4 / §107 (resilience IS end-user quality), §11.4.1 (FAIL-bluffs forbidden), §11.4.5 (captured-evidence quality applies to latency distribution + error categories), §11.4.6 (no guessing — categorised errors only), §11.4.43 (TDD RED-first under load/chaos), §11.4.50 (N iterations identical exit + identical evidence-hashes), §11.4.52 (autonomous validation), §11.4.69 (universal sink-side positive-evidence taxonomy), §11.4.83 (recovery transcripts ARE end-user-channel proofs).

Canonical authority: constitution submodule `Constitution.md` §11.4.85.

Non-compliance is a release blocker regardless of context. No escape hatch — no `--skip-stress`, `--no-chaos`, `--happy-path-suffices`, `--stress-test-later` flag exists.

§11.4.87 — Endless-loop autonomous work + zero-idle agent dispatch + anti-bluff testing mandate (User mandate, 2026-05-26)

When operator instructs an AI agent to "continue in endless loop fully autonomously" (or semantically-equivalent), the agent MUST treat as HARD-CONTRACT covenant covering five obligations: (A) continue until `docs/Issues.md` non-terminal Status entries = 0 AND `docs/CONTINUATION.md` §3 Active work empty AND no subagent in-flight AND no external dep in-flight; (B) dispatch background subagents for parallelisable work — main + subagents concurrent, "waiting for results" is the ONLY idle reason; (C) every closure lands four-layer test coverage per §11.4.4(b) with captured-evidence "physical proofs" (tinycap WAV + RMS / screen recording + ffprobe / dumpsys + sink-probe / uiautomator dump / sysfs snapshots) — metadata-only / config-only / absence-of-error / grep-without-runtime PASS are critical defects; (D) §11.4 anti-bluff covenant family operative end-to-end (tests AND HelixQA Challenges bound equally per forensic anchor "tests pass but features don't work"); (E) loop terminates ONLY on all-conditions-met, explicit operator STOP, host-safety demand (§12 family), or scheduled wake on known-future-actionable signal.

Composes with §11.4 / §11.4.1 / §11.4.2 / §11.4.4 / §11.4.5 / §11.4.6 / §11.4.7 / §11.4.20 / §11.4.27 / §11.4.42 / §11.4.43 / §11.4.50 / §11.4.52 / §11.4.58 / §11.4.68 / §11.4.69 / §11.4.70 / §11.4.83 / §11.4.85 / §11.4.86 / §12.10. Pre-build gate `CM-COVENANT-114-87-PROPAGATION` + paired §1.1 mutation.

Canonical authority: constitution submodule `Constitution.md` §11.4.87.

Non-compliance is a release blocker regardless of context. No escape hatch — `--idle-OK`, `--skip-endless-loop`, `--bluff-permitted-for-this-task`, `--metadata-only-test-suffices`, `--no-physical-proof-required` are FORBIDDEN flags.

§11.4.69 — Universal Sink-Side Positive-Evidence Taxonomy + Mechanical Enforcement (cascaded from constitution submodule §11.4.69)

> Verbatim user mandate (2026-05-20): *"THIS MUST HAPPEN NEVER AGAIN!!! We MUST HAVE this all working! Not just for audio but for every single piece of the System!!! Proper full automation when executed with success MUST MEAN that manual testing will be as much positive at least regarding the success results! ... Solution MUST BE universal, generic that solves working flows for all System components and for all future and all existing projects! ... Everything we do MUST BE validated and verified with rock-solid proofs and anti-bluff policy enforcement and fulfillment!"*

Universal generalisation of §11.4.68 (audio-specific) across every user-visible feature class. Every user-visible feature MUST map to one entry in the closed-set §11.4.69 sink-side evidence taxonomy (`audio_output`, `audio_input`, `video_display`, `network_throughput`, `network_connectivity`, `bluetooth_a2dp`, `bluetooth_pair`, `touch_input`, `sensor`, `gpu_render`, `storage_read`, `storage_write`, `mediacodec_decode`, `mediacodec_encode`, `miracast`, `cast`, `boot_service`, `package_install`, `permission_grant`, `wifi_link`, `wifi_throughput`, `ethernet_link`, `display_topology`, `drm_playback`, `subtitle_render` — open to additions, never contraction). Every PASS for a feature in the taxonomy MUST cite a captured-evidence artefact path matching the required evidence shape. New helper contracts (additive during grace, mandatory after 2026-06-19): `ab_pass_with_evidence <description> <evidence_path>` (verifies path exists + non-empty), `ab_skip_with_reason <description> <closed-set-reason>` (reasons: `geo_restricted`, `operator_attended`, `hardware_not_present`, `topology_unsupported`, `network_unreachable_external`, `feature_disabled_by_config`; forbids `network_unreachable_external` for any taxonomy feature with a sink-side probe); bare `ab_pass` deprecated (WARN pre-grace, FAIL post-grace). Three pre-build gates + paired §1.1 mutations: `CM-SINK-EVIDENCE-PER-FEATURE`, `CM-NO-FAIL-OPEN-SKIP`, `CM-AB-PASS-WITH-EVIDENCE-EVERYWHERE`. No escape hatch — no `--skip-evidence`, `--config-only-pass`, `--allow-fail-open-skip`, `--legacy-ab-pass-permitted` flag.

Cascade requirement: This anchor (verbatim or by `§11.4.69` reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. Propagation gate `CM-COVENANT-114-69-PROPAGATION` enforces the anchor literal across the consumer fleet; paired mutation strips the literal → gate

FAILs. Severity-equivalent to a §11.4 PASS-bluff at the sink-side-evidence layer. **Canonical authority:** constitution submodule `Constitution.md` §11.4.69 for the full mandate.

§11.4.75 — Mechanical Enforcement Without Exception (cascaded from constitution submodule §11.4.75)

> Verbatim user mandate (2026-05-20): *"Why do these violations still happen!? This is a serious problem! We cannot rely on stability nor consistency if we cannot respect our Constitution, mandatory rules and constraints! Is there a way to make this always respected, followed and applied without exception fully and unconditionally!? WE MUST HAVE THIS WORKING FLAWLESSLY!!! Do investigate the root causes of such problems! Once all problems are identified WE MUST apply proper mechanisms for this not to happen NEVER EVER AGAIN!"*

The §11.4 covenant historically relied on agent + operator vigilance; three 2026-05-19→20 forensic incidents proved that late-binding enforcement fires hours-to-days after the violator commit reaches every remote. §11.4.75 closes the gap with FIVE independent mechanical enforcement layers — bypassing any single layer does not bypass the discipline: (1) local `pre-commit` git hook (refuses staged `.md` lacking sibling `.html`+`.pdf`); (2) `commit_all.sh` integration (`_constitution_sibling_check` + auto-`sync_all_markdown_exports.sh` self-repair); (3) local `pre-push` git hook (re-runs siblings + propagation-gate subset); (4) `post-commit` auto-repair hook (auto-generates orphan-`.md` siblings, idempotent + recursion-guarded); (5) local-only final-gate ritual (remote CI DISABLED per User mandate — operator runs `pre_build_verification.sh` + meta-test before every tag per §11.4.40). Helper contracts: `scripts/install_git_hooks.sh`, `scripts/git_hooks/{pre-commit,pre-push,post-commit,commit-msg}`, `_constitution_sibling_check`. The `commit-msg` hook enforces a `Bypass-rationale: <reason>` footer when `--no-verify` is detected; `docs/audit/bypass_events.md` accumulates the audit trail. Five gates with paired §1.1 mutations: `CM-COVENANT-114-75-PROPAGATION`, `CM-GIT-HOOKS-INSTALL-SCRIPT`, `CM-GIT-HOOKS-SOURCE-DIR`, `CM-COMMIT-ALL-SIBLING-CHECK`, `CM-CI-WORKFLOW-PRESENT`. No escape hatch — no `--skip-hooks`, `--bypass-enforcement`, `--allow-orphan-md`, `--ci-not-applicable`, `--mechanical-enforcement-not-needed` flag.

Cascade requirement: This anchor (verbatim or by `§11.4.75` reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. Propagation gate `CM-COVENANT-114-75-PROPAGATION`; paired mutation strips the literal → gate FAILs. Severity-equivalent to a §11.4 PASS-bluff at the enforcement layer. **Canonical authority:** constitution submodule `Constitution.md` §11.4.75 for the full mandate.

§11.4.76 — Containers-Submodule Mandate (cascaded from constitution submodule §11.4.76)

> Verbatim user mandate (2026-05-20): *"For any work or requirements of running services or codebase inside the Containers (Docker / Podman / Qemy / Emulators, and so on) we MUST USE / INCORPORATE the Containers Submodule properly: https://github.com/vasic-digital/containers (git@github.com:vasic-digital/containers.git). Containers Submodule contains all means for us to Containerize our code and services! If any feature or Containing System is missing or not supported we MUST EXTEND IT properly like we do all of our projects! No bluff work is allowed of any kind!"*

For ANY containerized workload (Docker / Podman / Qemu / Kubernetes / container-backed emulators), every consuming project MUST: (1) install `vasic-digital/containers` (`digital.vasic.containers`) as a Git submodule; (2) consume via `replace` directive during development + pinned commit SHAs in production; (3) boot infra on-demand via `pkg/boot` + `pkg/compose` + `pkg/health` so operators are never required to start `podman machine` / `docker compose up` manually — the boot is part of the test entry point (the on-demand-infra invariant); (4) extend the Submodule (PR upstream) for missing runtimes / lifecycle primitives — never reimplement in-project (per §11.4.74); (5) anti-bluff: integration tests claiming to exercise containerized components MUST actually boot them via the Submodule — short-circuit fakes that bypass boot are a §11.4 violation. Tracker rows touching containerization MUST record `Catalogue-Check: extend vasic-digital/containers@<sha>` (or `reuse`). Planned gate `CM-CONTAINERS-USED` scans container-touching PRs for `digital.vasic.containers/...` imports; paired mutation strips the import + asserts FAIL.

Cascade requirement: This anchor (verbatim or by `§11.4.76` reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. Propagation gate `CM-COVENANT-114-76-PROPAGATION`; paired mutation strips the literal → gate FAILs. **Canonical authority:** constitution submodule `Constitution.md` §11.4.76 for the full mandate.

§11.4.77 — Regeneration-Mechanism-Required Mandate (cascaded from constitution submodule §11.4.77)

> Verbatim user mandate (2026-05-20): *"We must be sure that after excluding anything from Git versioning we still have the mechanism which will out of the box obtain or re-generate missing content!"*

Every `.gitignore` entry excluding (a) >~100 MiB OR (b) any artefact essential to building / running / testing the project MUST carry a documented + automated mechanism to either re-obtain (download from authoritative source: vendor tarball, SDK installer, npm/pip/cargo/go-mod/container registry, dedicated git submodule, S3/GCS) OR re-generate (run from tracked source via build pipeline, code-gen, asset render, captured-evidence replay, container build). Required artefacts per qualifying entry: (1) `.gitignore-meta/<entry-slug>.yaml` declaring pattern + mechanism-type + script-path + expected-disk-usage + vendor-url-or-source +

integrity hash + requires-network + requires-credentials; (2) a non-interactive entry in `scripts/setup.sh` post-clone bootstrap; (3) a pre-build gate verifying regenerated content present OR a recent `.gitignore-meta/.regenerated/<slug>.ok` stamp; (4) README + `docs/guides/*.md` describing the mechanism + manual fallback + time/disk budget + §11.4.10 credentials. Bare `.gitignore` additions without the mechanism are a §11.4 PASS-bluff variant — codebase appears complete but a fresh clone cannot build/run. No escape hatch — no `--skip-regen-mechanism`, `--gitignore-is-enough`, `--operator-already-has-content` flag. Planned gate `CM-GITIGNORE-REGEN-MECHANISM` + paired §1.1 mutation (strip a required YAML key → gate FAILs).

Cascade requirement: This anchor (verbatim or by `§11.4.77` reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. Propagation gate `CM-COVENANT-114-77-PROPAGATION`; paired mutation strips the literal → gate FAILs. Severity-equivalent to a §11.4 PASS-bluff at the repository-hygiene layer. **Canonical authority:** constitution submodule `Constitution.md` §11.4.77 for the full mandate.

§11.4.79 — Own-Org Submodules MUST Be Included in the CodeGraph Index (cascaded from constitution submodule §11.4.79)

> Verbatim user mandate (2026-05-21): *"All Submodules we use in the project and that are part of organizations to which we have the full access via GitHub, GitLab and other CLIs MUST BE included into the codegraph database and initialized / scanned / synced!"*

Refines §11.4.78's exclude-list with a per-submodule-ownership split: (a) own-org submodules (full write access via the project's CLIs — canonical orgs `vasic-digital` + `HelixDevelopment`) MUST be INCLUDED in the index; (b) third-party submodules (the §11.4.74 `no-match → vendor` path) MUST be EXCLUDED. Operational steps: (1) `git submodule update --remote --merge` to pull latest before re-indexing, respecting load-bearing pins on third-party submodules; (2) adjust `.codegraph/config.json` exclude list to keep own-org paths in scope; (3) re-index via `scripts/codegraph_setup.sh`; (4) verify via `scripts/codegraph_validate.sh` with ≥1 probe resolving a symbol living ONLY inside an own-org submodule; (5) paired §1.1 mutation — temporarily add the own-org submodule to exclude → validate MUST FAIL on the cross-submodule probe → restore. An index that lies about reachable symbols is a PASS-bluff against AI agents. Own-org submodules silently excluded without an audit trail in `.codegraph/config.json` comments is a release blocker.

Cascade requirement: This anchor (verbatim or by `§11.4.79` reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. Propagation gate `CM-COVENANT-114-79-PROPAGATION`; paired mutation strips the literal → gate FAILs. **Canonical authority:** constitution submodule `Constitution.md` §11.4.79 for the full mandate.

§11.4.80 — CodeGraph Regular-Update + Sync Automation Mandate (cascaded from constitution submodule §11.4.80)

> Verbatim user mandate (2026-05-21): *"We MUST regularly check for the updates and execute codegraph npm updates so the latest version of it is always installed on the host machine! ... Make sure we have proper full automation bash scripts which will run regularly and that these are part of the constitution Submodule ... Make sure all updates, sync processes we do and important codegraph related events are all documented under docs/codegraph in Status and Status_Summary documents ... and regularly export them like all other Status docs into the PDF and HTML!"*

Three deliverables (all living in the constitution submodule, inherited by reference per §3 — consuming projects invoke at `${CONST_DIR}/scripts/codegraph_*.sh`, never copy): (1) `scripts/codegraph_update.sh` — npm-installs latest `@colbymchenry/codegraph` after a registry version check; appends old/new version to `docs/codegraph/Status.md`; anti-bluff verifies `codegraph --version` reflects the new version after install (npm exit 0 ≠ working binary). (2) `scripts/codegraph_sync.sh` — after a successful update runs `codegraph status` → `codegraph sync .` → `codegraph status` → the project's `scripts/codegraph_validate.sh`; appends every step's output to BOTH the project's and the constitution's `docs/codegraph/Status.md`. (3) `docs/codegraph/Status.md` + `Status_Summary.md` append-only ledgers, exported to `.html` + `.pdf` per §11.4.65. Cadence: weekly floor (per §11.4.45). A consuming project that has not run `codegraph_update.sh` in >2 weeks AND has open AI-agent work is a release blocker. Paired §1.1 mutation: downgrade installed version → script detects drift → restore.

Cascade requirement: This anchor (verbatim or by `§11.4.80` reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. Propagation gate `CM-COVENANT-114-80-PROPAGATION`; paired mutation strips the literal → gate FAILs. **Canonical authority:** constitution submodule `Constitution.md` §11.4.80 for the full mandate.

§11.4.81 — Cross-Platform-Parity Mandate (cascaded from constitution submodule §11.4.81)

> Verbatim user mandate (2026-05-21): *"Any Linux-only blocker / issue we have MUST BE created macOS and other supported platforms equivalent! So, depending on platform proper implementation will be used for particular OS! EVERYTHING MUST BE PROPERLY EXTENDED AND UPDATED!"*

Every consuming project whose supported-platforms manifest lists more than one OS MUST, for every feature/test/gate/challenge/mutation depending on platform-specific primitives, ship a per-OS-equivalent implementation chosen at runtime via `uname -s` (or equivalent detection). Three sub-mandates: **(A) Per-OS implementation REQUIRED** — Linux cgroup/systemd/`/proc` primitives MUST have documented per-OS equivalents (POSIX `setrlimit`/`ulimit`, macOS `launchd`, BSD `rctl`, Windows Job Object) chosen via runtime dispatch. **(B) Per-OS tests REQUIRED** — every platform-dependent gate test MUST have `case "$`

(uname -s)" in branches with positive captured evidence per §11.4.2 + §11.4.5 in each branch; SKIP-with-reason acceptable ONLY when the platform genuinely cannot enforce the invariant. **(C) Honest kernel-gap citation + adjacent equivalent test REQUIRED** — where a Linux primitive has NO equivalent due to a documented kernel limitation (canonical: XNU does not enforce RLIMIT_AS for unprivileged processes), the test MUST detect the gap at runtime, SKIP with exact kernel reason + reproducer + honest-gap-doc link, AND provide an ADJACENT test exercising the closest invariant the platform CAN enforce (e.g. RLIMIT_CPU+SIGXCPU as the macOS proxy), itself anti-bluff with a paired §1.1 mutation. Gate CM-CROSS-PLATFORM-PARITY scans for case "$(uname -s)" blocks asserting a non-SKIP branch (or honest-gap citation) per platform in the manifest; paired mutation strips a Darwin branch → gate FAILs. No escape hatch.

Cascade requirement: This anchor (verbatim or by §11.4.81 reference) MUST appear in every owned submodule's CONSTITUTION.md, CLAUDE.md, and AGENTS.md. Propagation gate CM-COVENANT-114-81-PROPAGATION; paired mutation strips the literal → gate FAILs. Release blocker on multi-platform projects. **Canonical authority:** constitution submodule Constitution.md §11.4.81 for the full mandate.

§11.4.82 — Iteration-Speedup Discipline Mandate (cascaded from constitution submodule §11.4.82)

> Verbatim user mandate (2026-05-22): *"How can we speed-up this whole development and fixing process? ... Do not forget to all speed optimizations critical rules and mandatory constraints MUST BE all added into our root (constitution Submodule) Constitution.md, CLAUDE.md, AGENTS.md and QWEN.md and all other relevant constitution Submodules files!"*

Iteration cycle time is a first-order quality enabler. Every consuming project's build / test / commit / debug pipeline MUST adopt these speedup disciplines AS MANDATORY (each independently enforceable): (A) Phase-1 forensic (superpowers:systematic-debugging) before any speculative source patch — speculative patches without FACT-grade root cause are §11.4.6 + §11.4.82 violations; (B) Live-ADB-First (or live-equivalent) before any rebuild — strengthens §11.4.51 to a release-blocker mandate; (C) 30-second pre-flight before launching rebuild orchestrators (device/sink reachability, host memory/disk, no stale locks, no orphan processes); (D) persistent build caches outside containers (ccache/ sccache/Gradle daemon bind-mounted to host); (E) module-only rebuild for loadable-module-only changes; (F) parallel multi-device testing with separate qa-results/<TS>/<device-tag>/ outputs; (G) subagent scope discipline + worktree isolation (≤30 min budget, single-responsibility, isolation: "worktree" default); (H) lock-file + stale-process hygiene (clean .git/index.lock, disable auto git-gc in concurrent repos); (I) cycle telemetry per §11.4.24 (commit hash, per-phase wall-clock, speedup-flag set, outcome — aggregated weekly). Gate CM-ITERATION-SPEEDUP-DISCIPLINE audits recent cycles for telemetry citing which of (A)-(I) applied; paired §1.1 mutation strips the speedup-flag column → gate FAILs. No escape hatch — no --skip-phase1-forensic, --no-pre-flight, --rebuild-everything-always, --unlimited-subagent-scope, --ignore-locks, --no-telemetry flag.

Cascade requirement: This anchor (verbatim or by `§11.4.82` reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. Propagation gate `CM-COVENANT-114-82-PROPAGATION`; paired mutation strips the literal → gate FAILs. Release blocker. **Canonical authority:** constitution submodule `Constitution.md` §11.4.82 for the full mandate.

§11.4.83 — docs/qa/ End-User Evidence Mandate (cascaded from constitution submodule §11.4.83)

> Verbatim user mandate (2026-05-22): *"every feature that ships MUST carry a recorded e2e communication transcript + any attached materials under `docs/qa/<run-id>/` (per-feature subdirectories). A feature with no QA transcript is itself a §107 PASS-bluff — it claims to work but has no auditable runtime evidence. Bot-driven automation MUST preserve full bidirectional communication threads as proof."*

Every feature that ships MUST carry a recorded end-to-end communication transcript plus any attached materials (screenshots, request/response payloads, audio, file uploads) committed under `docs/qa/<run-id>/` — one directory per feature run. Operative rule: (1) every consuming project MUST maintain a `docs/qa/` tree, each new feature under `docs/qa/<run-id>/` where `<run-id>` is monotonic + greppable (timestamp / ATM-NNN / other workable-item ID per §11.4.54); (2) transcripts MUST be full bidirectional — every prompt/command sent + every response received (one-sided is not a transcript); (3) attached materials MUST be committed in-repo (no external-only links — that is a §11.4.13 sink-side violation); (4) bot-driven / agent-driven QA automation MUST preserve the full conversation thread as the proof artefact; (5) release gates MUST refuse to tag a version that has any feature-shipping commit without its matching `docs/qa/<run-id>/` directory. A feature with no QA transcript is a §11.4 / §107 PASS-bluff. Composes with §11.4.2 / §11.4.5 / §11.4.13 / §11.4.65 / §11.4.69 / §1.1.

Cascade requirement: This anchor (verbatim or by `§11.4.83` reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. Propagation gate `CM-COVENANT-114-83-PROPAGATION`; paired mutation strips the literal → gate FAILs. Release blocker — no `--qa-evidence-optional` escape hatch. **Canonical authority:** constitution submodule `Constitution.md` §11.4.83 for the full mandate.

§11.4.84 — Working-Tree Quiescence Rule for Subagent Commits (cascaded from constitution submodule §11.4.84)

> Verbatim user mandate (2026-05-22): *"no subagent commit may proceed while any concurrent mutation gate is in flight in the same checkout. Before `git add`, the committing agent MUST `grep` its own working tree for mutation markers (`MUTATED for paired`, `// always pass`, `return json.Marshal` shortcut paths, etc.). Any unexplained file in the staging area triggers ABORT."*

No subagent (or main-thread) commit may proceed while any concurrent mutation gate, paired-mutation experiment, or other in-flight mutation is live in the same checkout. Before `git add`, the committing agent MUST grep its own working tree for mutation markers (`MUTATED for paired`, `// always pass`, `return json.Marshal` shortcut paths, `// MUTATION` / `# MUTATION` annotations, `_mutated_*` filename suffixes, etc.) and explicitly account for every modified file in the staging area; any unexplained file → ABORT. (Forensic case: a logo-fix subagent's `git add` swept an `// always pass` JWT-verify mutation residue into an unrelated commit pushed to all four mirrors — a real security-defect window.) Operative rule: (1) pre-`git add` greps for mutation markers + cross-checks `git status --porcelain` against the subagent's declared scope; unaccounted entries → ABORT; (2) any active mutation gate MUST be serialised (mutate → assert FAIL → restore → assert PASS) and the working tree verifiably clean before any unrelated commit; (3) concurrent subagents in the SAME checkout MUST coordinate through a lockfile (`.git/MUTATION_IN_PROGRESS`) — cleaner solution is `git worktree add` per subagent (composes with §11.4.20/§11.4.70); (4) post-commit `mutation-residue-scanner` MUST run before push — any commit containing a mutation marker → push BLOCKED.

Cascade requirement: This anchor (verbatim or by `§11.4.84` reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. Propagation gate `CM-COVENANT-114-84-PROPAGATION`; paired mutation strips the literal → gate FAILs. A mutation marker that lands in a tagged commit is a critical defect regardless of how briefly it persisted. **Canonical authority:** constitution submodule `Constitution.md` §11.4.84 for the full mandate.

§11.4.85 — Stress + Chaos Test Mandate (cascaded from constitution submodule §11.4.85)

> Verbatim user mandate (2026-05-24): *"Every fix or improvement you do MUST BE covered with full automation stress and chaos tests so we are sure nothing can break the functionality and all edge cases are monitored and polished and additionally fixed if that is needed! Everything must produce rock solid proofs and follow fully no-bluff policy!"*

Every fix or improvement landed MUST ship with full-automation **stress** AND **chaos** test suites exercising edge cases, sustained load, concurrent contention, and failure-injection. Happy-path coverage alone is a §11.4 / §107 PASS-bluff at the resilience layer. **Stress** (closed-set): sustained load (N ≥ 100 iterations OR ≥ 30 s wall-clock, p50/p95/p99 latency recorded) + concurrent contention (N ≥ 10 parallel invocations, no deadlock/leak) + boundary conditions (empty/max/off-by-one, each categorised). **Chaos** (closed-set, per fix-class appropriateness): process-death injection + network-fault injection (drop/delay/reorder) + input-corruption injection + resource-exhaustion injection (disk full, OOM, FD exhaustion — refuse cleanly OR degrade, NEVER crash) + state-corruption injection (mid-flight lock loss, partial-write). Every stress + chaos PASS MUST cite a captured-evidence artefact path per §11.4.5 + §11.4.69. Helper library `stress_chaos.sh` provides `ab_stress_run`, `ab_stress_concurrent`, `ab_chaos_kill_pid_during`, `ab_chaos_drop_network_during`, `ab_chaos_corrupt_file_during`, `ab_chaos_oom_pressure_during`, `ab_chaos_disk_full_during`, each composing with `ab_pass_with_evidence` / `ab_skip_with_reason`. Cleanup non-negotiable in `trap '...' EXIT` (cleanup failure = §11.4.14 violation). Four-layer

coverage per §11.4.4(b) + paired §1.1 mutation (strip chaos-injection or evidence-capture → gate FAILs). No escape hatch — no `--skip-stress`, `--no-chaos`, `--happy-path-suffices`, `--stress-test-later` flag.

Cascade requirement: This anchor (verbatim or by `§11.4.85` reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. Propagation gate `CM-COVENANT-114-85-PROPAGATION`; paired mutation strips the literal → gate FAILs. Release blocker. **Canonical authority:** constitution submodule `Constitution.md` §11.4.85 for the full mandate.

§11.4.86 — Roster/Corpus-Backed Status-Doc Auto-Sync Mandate (cascaded from constitution submodule §11.4.86)

> Verbatim user mandate (2026-05-25): *"Make sure that assets and players Status docs are ALWAYS regularly updated and in sync like all others Status docs — any time we add or modify the assets content(s) or we change or add new / remove existing pre-installed video and audio player apps! This MUST WORK OUT OF THE BOX!"*

Some Status docs (§11.4.45) are backed by a tracked roster (installed apps/components) or a tracked asset corpus (test/media asset directory) rather than narrative alone. Their freshness MUST NOT depend on operator vigilance — the moment a roster/corpus member changes (app added/removed/renamed; asset added/modified/removed) the Status doc + Status_Summary + HTML + PDF MUST resync out of the box, mechanically. Mechanism (all must hold): (1) drift-proof fingerprint — sha256 of the sorted member list (NOT mtime), persisted in a sidecar beside the Status doc; (2) a sync helper that regenerates the fingerprint + re-exports HTML+PDF via the §11.4.65 exporter, wired so sync is automatic; (3) a pre-build gate that FAILs when the live fingerprint differs from the persisted one (mirrors §11.4.12 `CM-ISSUES-SUMMARY-SYNC` + §11.4.45 `sync_integration_status`); (4) a paired §1.1 mutation corrupting the fingerprint and asserting the gate FAILs. Classification: universal — the consuming project supplies the specific docs, roster/corpus sources, helper, and gate name per §11.4.35.

Cascade requirement: This anchor (verbatim or by `§11.4.86` reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. Propagation gate `CM-COVENANT-114-86-PROPAGATION`; paired mutation strips the literal → gate FAILs. Release blocker — no `--skip-roster-sync`, `--allow-status-drift`, `--roster-sync-not-applicable` flag. **Canonical authority:** constitution submodule `Constitution.md` §11.4.86 for the full mandate.

§11.4.87 — Endless-Loop Autonomous Work + Zero-Idle Agent Dispatch + Anti-Bluff Testing Mandate (cascaded from constitution submodule §11.4.87)

> Verbatim user mandate (2026-05-26): *"continue in endless loop fully autonomously"* (and any semantically-equivalent phrasing).

When the operator instructs an AI agent to continue in an endless autonomous loop, the agent MUST treat it as a HARD-CONTRACT covenant: (A) continue working until `docs/Issues.md` Status-column has zero non-terminal entries AND `docs/CONTINUATION.md` §3 Active work is empty AND no background subagent is mid-execution AND no external dependency is in-flight; (B) dispatch background subagents for parallelisable work — main + every subagent operate concurrently, "waiting for results" is the ONLY acceptable idle reason; (C) every closure lands four-layer test coverage per §11.4.4(b) with captured-evidence (audio/video/network/UI/sysfs physical proofs); (D) the §11.4 anti-bluff covenant family (§11.4.1 / §11.4.2 / §11.4.6 / §11.4.7 / §11.4.27 / §11.4.50 / §11.4.52 / §11.4.68 / §11.4.69 / §11.4.83) is the operative truth-discipline — tests AND HelixQA Challenges bound equally; (E) the loop terminates ONLY on all-conditions-met, explicit operator STOP, host-session-safety demand, or scheduled wake on a known-future-actionable signal. No escape hatch — no `--idle-OK`, `--skip-endless-loop`, `--bluff-permitted-for-this-task`, `--metadata-only-test-suffices`, `--no-physical-proof-required` flag.

Cascade requirement: This anchor (verbatim or by `§11.4.87` reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. Propagation gate `CM-COVENANT-114-87-PROPAGATION`; paired mutation strips the literal → gate FAILs. Release blocker. **Canonical authority:** constitution submodule `Constitution.md` §11.4.87 for the full mandate.

§11.4.88 — Background-Push Mandate: Commit-Lock Release Immediately After Commit, Push Runs Detached (cascaded from constitution submodule §11.4.88)

Forensic anchor (2026-05-26): a single `commit_all.sh` held its flock ~5 hours because `do_push` ran synchronously after the commit landed — every subsequent commit blocked on a slow mirror push irrelevant to the local commit's durability. Implementation seam for §11.4.87(B) zero-idle. The mandate: (A) `.git/.commit_all.lock` MUST be released IMMEDIATELY after `git commit` returns 0 — the commit is durable on local disk regardless of remote push outcome; (B) push runs detached via `nohup ./push_all.sh ... > <log> 2>&1 &` + `disown` — the orchestrator's exit code reports COMMIT success, NOT push success; (C) `push_all.sh` acquires per-remote flock `.git/.push.<remote>.lock` so concurrent invocations targeting the same remote serialize but different-remote invocations run in parallel; (D) backgrounded push failures land in `qa-results/push_failures/<ts>_<remote>.log` — the next autonomous-loop tick checks per §11.4.87(A) "no external dependency in-flight" gate; (E) synchronous-push escape: explicit `--sync-push` CLI flag preserves legacy behaviour for §11.4.41 force-push merge-first audit paths. Gates `CM-COVENANT-114-88-PROPAGATION` + `CM-BACKGROUND-PUSH-WIRED` + paired §1.1 mutations. Synchronous push (without `--sync-push`) = §11.4 PASS-bluff at the execution layer.

Cascade requirement: This anchor (verbatim or by `§11.4.88` reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. Propagation gate `CM-COVENANT-114-88-PROPAGATION`; paired mutation strips the literal → gate FAILs. Release blocker — no escape hatch beyond `--sync-push` for force-push events. **Canonical authority:** constitution submodule `Constitution.md` §11.4.88 for the full mandate.

§11.4.89 — Background Test Execution Mandate (cascaded from constitution submodule §11.4.89)

> Verbatim user mandate (2026-05-27): *"Any tests we are executing, especially long test cycles, MUST BE performed in background in parallel with main work stream! This MUST NOT block our capabilities to work on queued workable items. Main work stream can be blocked or sit iddle only if absolutely needed and if it depends hard on results of some background execution."*

Symmetric anchor to §11.4.88 (background push) at the test-execution layer. Mandate: (A) long-running tests (>30 s expected: `pre_build`, `meta_test`, `test_all_fixes`, `recent_work_validate`, HelixQA banks, 4-phase cycles, full-suite retests, audio supervisors, dual-display recorders) MUST run via `nohup ... > <log> 2>&1 &` + `disown` with the log under a known dir (`qa-results/<test_id>_<ts>.log`); (B) the main stream proceeds to the §11.4.42 priority queue immediately; (C) hard-dependency gating — poll an exit-status file or `pgrep -af <test>` before steps that need the exit code, surfacing as §11.4.66 interactive options if the test is still running; (D) failures land in `<log>` files, the next loop tick checks; (E) foreground execution permitted ONLY for <30 s tests OR explicit operator authorisation; (F) per-script flock serialises same-script invocations, different-script invocations parallel. Gates `CM-COVENANT-114-89-PROPAGATION` + `CM-BACKGROUND-TEST-EXECUTION-WIRED` + paired §1.1 mutations.

Cascade requirement: This anchor (verbatim or by `§11.4.89` reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. Propagation gate `CM-COVENANT-114-89-PROPAGATION`; paired mutation strips the literal → gate FAILs. Release blocker — no escape hatch beyond explicit per-invocation operator authorisation. **Canonical authority:** constitution submodule `Constitution.md` §11.4.89 for the full mandate.

§11.4.90 — Obsolete Status + Per-Item Obsolescence Audit (cascaded from constitution submodule §11.4.90)

> Verbatim user mandate (2026-05-27): *"Bug No 6 ... seems obsolete after latest request for new behavior ... mark obsolete tickets with some light gray background ... text - the description to be strikethrough styled ... review all existing open or resolved workable items if they are obsolete - not valid any more ... There MUST NOT be any mistake! No bluff is allowed of any kind!"*

The §11.4.15 Status closed-set is extended with a terminal `Obsolete (→ Fixed.md)` value (orthogonal to Type per §11.4.16). Obsolescence reasons (closed vocabulary): `superseded-by-design-change | superseded-by-later-mandate | feature-removed | duplicate-of | unsupported-topology`. Every Obsolete heading MUST carry an `**Obsolete-Details:**` line (Since + Reason + Superseding-item + Triple-check evidence) within 8 non-blank lines. The §11.4.23 colorizer adds a `cell-status-obsolete` class — light-gray `#E0E0E0` background + strikethrough description. Audit cadence: every release-gate sweep per §11.4.40 + §11.4.42; triple-check is non-negotiable per the operator mandate.

Composes with §11.4.15 / §11.4.16 / §11.4.19 / §11.4.21 / §11.4.23 / §11.4.33 / §11.4.34 / §11.4.40 / §11.4.42 / §11.4.66 / §11.4.71. Gates `CM-COVENANT-114-90-PROPAGATION` + `CM-ITEM-OBSOLETE-DETAILS` + `CM-OBSOLETE-COLORIZER-WIRED` + paired §1.1 mutations.

Cascade requirement: This anchor (verbatim or by `§11.4.90` reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. Propagation gate `CM-COVENANT-114-90-PROPAGATION`; paired mutation strips the literal → gate FAILs. Release blocker. **Canonical authority:** constitution submodule `Constitution.md` §11.4.90 for the full mandate.

§11.4.91 — Summary-Doc Clarity Mandate (cascaded from constitution submodule §11.4.91)

> Verbatim user mandate (2026-05-27): *"Summary docs - Issues_Summary some not clear one line descriptions - like 'Composes with' ... For each workable item we MUST HAVE clearly understandable meaning ... every team member can clearly understand what that particular workable item is exactly about! There cannot be misunderstanding or unclearity of any kind and no bluff allowed!"*

Every summary entry (Issues_Summary, Fixed_Summary, README doc-link, Status_Summary pages 1+2, all one-liners) MUST contain a self-contained meaningful description ≥ 6 words OR ≥ 40 chars naming SUBJECT + PROBLEM/GOAL. Forbidden one-liner anti-patterns: section labels (`Composes with`, `Closure criteria`, `Fix direction`, etc.); bare metadata fragments (`Critical`, `Bug`, `In progress`, etc.); section-marker echoes; a §-letter alone. Generators (`generate_issues_summary.sh` / `generate_fixed_summary.sh` / `update_readme_doc_links.sh` / `generate_status_summary.sh`) MUST extract from the H1/H2 heading line per the §11.4.54 ATM-NNN convention, NEVER from arbitrary downstream text, and MUST refuse anti-pattern rows — emitting a `(MISSING DESCRIPTION — fix source heading)` placeholder with visual highlight. Gate `CM-SUMMARY-CLARITY-DESCRIPTIONS` scans every summary; an anti-pattern match = FAIL. Audit cadence: every §11.4.40 + §11.4.42 sweep.

Cascade requirement: This anchor (verbatim or by `§11.4.91` reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. Propagation gate `CM-COVENANT-114-91-PROPAGATION`; paired mutation strips the literal → gate FAILs. Release blocker. **Canonical authority:** constitution submodule `Constitution.md` §11.4.91 for the full mandate.

§11.4.92 — Multi-Pass Change-Evaluation Discipline (cascaded from constitution submodule §11.4.92)

> Verbatim user mandate (2026-05-27): *"Every change to the project or codebase we do MUST BE evaluated in several passes and in in-depth analisys for potential new issues or problems it can introduce! ... no bluff of any kind! After we do change or set of changes this mandatory steps MUST BE taken!"*

Every non-trivial change MUST pass a 5-pass evaluation BEFORE it is commit-ready: **(Pass 1)** main-task verification — change achieves the stated goal, captured-evidence per §11.4.5/§11.4.69; **(Pass 2)** regression-blast-radius analysis — enumerate every direct dependency, demonstrate no contract break; **(Pass 3)** cross-feature interaction analysis — audit parallel features sharing state/timing/hardware/shell environment; **(Pass 4)** deep-research validation per §11.4.8 — external precedent OR "NO external solution found — original work" + CodeGraph queries per §11.4.78/§11.4.79; **(Pass 5)** anti-bluff confirmation per §11.4 / §11.4.1 / §11.4.6 / §11.4.27 / §11.4.50 / §11.4.52 / §11.4.69 / §11.4.83 — no new bluff surface introduced. Each pass is documented (commit footers OR `docs/` entries OR `qa-results/` evidence). Only after all 5 passes complete may commit/push/test/release proceed. Trivial exemption: typo / revision-bump / MD-export-regen IF zero source touched AND the commit message cites the exemption explicitly. Gates `CM-COVENANT-114-92-PROPAGATION` + `CM-MULTI-PASS-EVALUATION-EVIDENCE` + paired §1.1 mutations.

Cascade requirement: This anchor (verbatim or by `§11.4.92` reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. Propagation gate `CM-COVENANT-114-92-PROPAGATION`; paired mutation strips the literal → gate FAILs. Release blocker. **Canonical authority:** constitution submodule `Constitution.md` §11.4.92 for the full mandate.

§11.4.93 — SQLite-Backed Single-Source-of-Truth for Workable Items (cascaded from constitution submodule §11.4.93)

> Verbatim user mandate (2026-05-27): *"There MUST be single source of truth for all of our workable items - SQlite database … proper scripts (we recommend Go programs) … reduce a chance for sync to be broken … generate always all docs from DB or to re-generate Db from all docs we have in opposite direction"*

The text-based Issues/Fixed/Summary/CONTINUATION constellation is converted to a SQLite-DB-backed single source of truth. Schema mandatory tables: `items` (atm_id PK + Type + Status incl. Obsolete + Severity + title + description ≥40 chars + created/modified + composes_with JSON + current_location); `item_history` (append-only audit per §11.4.34 By/Reason/Evidence); `obsolete_details` (§11.4.90); `operator_block_details` (§11.4.21); `firebase_metadata` (§11.4.47); `meta` (schema version + last sync + integrity hash). A Go binary at `cmd/workable-items/` provides `sync md-to-db` / `db-to-md` / `diff` / `validate` / `add` / `close`; bidirectional regen is byte-identical round-trip (closed-set whitespace/section-order tolerance). `commit_all.sh` refuses on non-empty diff; `sync_issues_docs.sh` invokes the Go binary; pre-build runs `workable-items validate`. Anti-bluff: unit + integration + stress (1000-row insert + 10 concurrent writers) + chaos (mid-write SIGKILL + corrupt-DB recovery + disk-full) + paired §1.1 mutation + HelixQA Challenge `CME-WORKABLE-ITEMS-001`. The Go binary lives in the constitution submodule (`constitution/scripts/workable-items/`) per §11.4.74. Gates `CM-COVENANT-114-93-PROPAGATION` + `CM-WORKABLE-ITEMS-DB-PRESENT` + `CM-WORKABLE-ITEMS-MD-DB-IN-SYNC` + paired §1.1 mutations. (NOTE: the DB tracking rule is AMENDED by §11.4.95 — DB is TRACKED, not gitignored.)

Cascade requirement: This anchor (verbatim or by `§11.4.93` reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. Propagation gate `CM-COVENANT-114-93-PROPAGATION`; paired mutation strips the literal → gate FAILs. Release blocker — text-based-only trackers are a §11.4 PASS-bluff at the data-architecture layer. **Canonical authority:** constitution submodule `Constitution.md` §11.4.93 for the full mandate.

§11.4.94 — Zero-Idle Priority-First Parallel-By-Default Operating Mode (cascaded from constitution submodule §11.4.94)

> Verbatim user mandate (2026-05-27): *"We MUST NEVER sit iddle / wait or sleep if there is possibility for us to work on something ... Always check if there is a possibility to work on something while we are not working actively on something! Pick always by priority - most critical workable items and other tasks MUST BE done first! ... Stay still / iddle if nothing is left to be done at all or waiting for something that is blocking us / you!!!"*

§11.4.94 binds §11.4.20 + §11.4.42 + §11.4.58 + §11.4.70 + §11.4.72 + §11.4.82 + §11.4.87 + §11.4.88 + §11.4.89 into a single always-on enforcement: (A) idle ONLY when every queued item is genuinely blocked on an external dependency (hardware / network upstream / build/test completion the conductor cannot accelerate) OR operator STOP OR §12 host-safety — "don't see what to do" is NEVER valid; (B) before ANY wake/sleep the conductor MUST survey parallel-work feasibility per §11.4.42 + §11.4.72 + §11.4.87, identify non-contending items, and dispatch in parallel per §11.4.20/§11.4.70 (subagent) + §11.4.58 (PWU disjoint scope) + §11.4.89 (background long tests); (C) priority order MANDATORY — pick highest-severity + §11.4.72 audio-first the conductor can autonomously progress; (D) subagent-driven default for non-trivial; (E) background default for >30 s wall-clock work via `nohup`+`disown`; (F) stability-preserving (composes with §11.4.92 multi-pass + §11.4.84 quiescence + §12.6–§12.9 host safety); (G) progress updates surfaced at milestone boundaries. Gates `CM-COVENANT-114-94-PROPAGATION` + `CM-PARALLEL-WORK-AUDIT` + paired §1.1 mutations.

Cascade requirement: This anchor (verbatim or by `§11.4.94` reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. Propagation gate `CM-COVENANT-114-94-PROPAGATION`; paired mutation strips the literal → gate FAILs. Release blocker. **Canonical authority:** constitution submodule `Constitution.md` §11.4.94 for the full mandate.

§11.4.95 — Workable-Items SQLite DB Is TRACKED in Git, NEVER Gitignored (cascaded from constitution submodule §11.4.95)

> Verbatim user mandate (2026-05-27): *"We shall not Git ignore our workable items SQlite DB since it is our single source of truth ... workable items SQlite DB regularly commited and pushed to all upstreams!"*

§11.4.93's earlier "gitignored per §11.4.30" clause is AMENDED — the DB at `docs/workable_items.db` is TRACKED in git, NEVER gitignored. It IS authoritative source data, NOT a build artefact. Every `workable-items sync md-to-db` that mutates state MUST stage + commit + push the DB alongside the MD regen per §11.4.19 atomic-move + §2.1 multi-upstream push. A WAL-checkpoint (`PRAGMA wal_checkpoint(TRUNCATE)`) is required before commit-stage so the transient `.db-wal` + `.db-shm` sidecars (gitignored per §11.4.30) are safely discardable. The §11.4.77 regeneration mechanism does NOT apply — the DB IS the source. Destructive DB ops require §9.2 hardlinked-backup + operator authorization; §11.4.41 force-push merge-first applies if DB history ever needs rewrite. Gates `CM-COVENANT-114-95-PROPAGATION` + `CM-WORKABLE-ITEMS-DB-TRACKED` + paired §1.1 mutation.

Cascade requirement: This anchor (verbatim or by `§11.4.95` reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. Propagation gate `CM-COVENANT-114-95-PROPAGATION`; paired mutation strips the literal → gate FAILs. Release blocker. **Canonical authority:** constitution submodule `Constitution.md` §11.4.95 for the full mandate.

§11.4.96 — Safe-Parallel-Work-With-Long-Build Catalogue + Mandate (cascaded from constitution submodule §11.4.96)

> Verbatim user mandate (2026-05-27): *"Are there except AOSP build process any other active jobs being done at the moment? Can we work on something in parallel while build is in progress so we slowly cleanup our slate? … do as much as possible work in background in parallel with main work stream and oreferrably using subagents-driven approach!"*

An operational catalogue for the canonical long-running workload (multi-hour containerised build per §12.9). **SAFE during build:** (A) MD/docs work; (B) generator/helper script work under `scripts/`; (C) pre-build + meta-test gate authoring + paired §1.1 mutations; (D) on-device test scripts; (E) constitution submodule edits + push; (F) any submodule commit + push per §11.4.88; (G) read-only live-ADB probes (`dumpsys`/`getprop`/`cat /proc/...`/`screencap`/`logcat`); (H) subagent dispatch per §11.4.20/§11.4.70 + §11.4.84 quiescence; (I) web research + external API queries with §11.4.10 credentials; (J) workable-items DB ops per §11.4.93+§11.4.95; (K) backgrounded pre-build + meta-test execution per §11.4.89. **UNSAFE during build:** (α) `git checkout`/`reset --hard`/`clean -df` on the source tree (use `git worktree`); (β) mass file deletes/renames under built source trees; (γ) submodule pointer updates affecting built artefacts; (δ) `out/` mutations; (ε) `make clean`/`m clobber`/`rm -rf out/`; (ζ) container destruction; (η) disk-filling breaching §12.9 free-space minimum; (θ) §12 host-session-safety breaches. Conductor responsibility: before EVERY pause point during a long build, consult the catalogue, identify (A)-(K) queue items per §11.4.42+§11.4.72, and dispatch ≥1 per §11.4.20/§11.4.70 subagent default + §11.4.89 background. "Build running, nothing else to do" is NEVER true per §11.4.94+§11.4.96. Gates `CM-COVENANT-114-96-PROPAGATION` + `CM-PARALLEL-WORK-DURING-BUILD-AUDIT` + paired §1.1 mutations.

Cascade requirement: This anchor (verbatim or by `§11.4.96` reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. Propagation gate `CM-COVENANT-114-96-PROPAGATION`; paired mutation strips the literal → gate FAILs. Release blocker. **Canonical authority:** constitution submodule `Constitution.md` §11.4.96 for the full mandate.

§11.4.97 — Maximum-Use-of-Idle-Time + Progress-Update Cadence (cascaded from constitution submodule §11.4.97)

> Verbatim user mandate (2026-05-27): *"keep it working, we should do as much as possible, if not it all but as much as we can as long as there is iddle time! it MUST be used! ... keep us updated about all progress and all phisycal proofs and gathered data as you progress through all open workable items!"*

Operating-mode capstone strengthening §11.4.87 + §11.4.94 + §11.4.96: (A) every minute of conductor idle time during which work could autonomously progress AND is not genuinely blocked = a §11.4.97 violation; "as much as possible, if not it all but as much as we can" is operative — dispatch CONTINUOUSLY through the entire idle window, not just at scheduled wakes; (B) progress-update cadence — emit an operator-facing 1-line update at every commit landed / subagent return / constitutional anchor / captured evidence / milestone closure, no operator prompt required; (C) continuous physical-proof gathering per §11.4.5 + §11.4.6 + §11.4.69 — every autonomous closure cites captured-evidence (evidence path goes into the §11.4.93 `item_history.evidence_path` when the DB lands); (D) composes with §11.4.5/6/13/20/27/42/50/52/69/70/72/83/85/87/88/89/94/96; (E) the idle-only-when-blocked closed-set is unchanged from §11.4.94(A). Gates `CM-COVENANT-114-97-PROPAGATION` + `CM-IDLE-TIME-AUDIT` + paired §1.1 mutations.

Cascade requirement: This anchor (verbatim or by `§11.4.97` reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. Propagation gate `CM-COVENANT-114-97-PROPAGATION`; paired mutation strips the literal → gate FAILs. Release blocker. **Canonical authority:** constitution submodule `Constitution.md` §11.4.97 for the full mandate.

---

§11.4.98 — Full-Automation Anti-Bluff Mandate (cascaded from constitution submodule §11.4.98)

> Verbatim user mandate (2026-05-28): *"Make sure we have full automation testing of all scenarios with real bot, main group and users without any manual intervention or contribution of real user! Everything MUST BE fully automatic and autonomous! These tests MUST BE able to rerun endless times when needed! ... Make sure there is no false positives in testing! Every test and its results MUST obtain real proofs of everything working! No bluff is allowed!"*

Closes the manual-intervention gap (§11.4 / §11.4.2 / §11.4.5 / §11.4.50 / §11.4.85 / §11.4.87 / §11.4.89 / §11.4.94 did not explicitly forbid it). A live/integration/e2e/Challenge test that requires a human action during execution (typing a message, clicking UI, hand-triggering a webhook, attaching a file — anything beyond startup) is by definition a §11.4 PASS-bluff at the automation layer. (A) Every governed test — unit/integration/e2e/Challenge/stress/chaos/live — MUST be fully self-driving end-to-end, reporting PASS/FAIL/SKIP-with-reason without any further human action after startup. (B) Single permissible exception: one-time credential bootstrap performed OUTSIDE test execution (`.env` from vault, shell exports, OAuth at first install, MTProto session activation) — configuration, not test driving. (C) Live messenger/channel/agent tests: no "operator must type" prompts (drive programmatically via second account / webhook fixture / loopback); no hard-coded session UUIDs that collide with the active dev session (Herald 2026-05-28 `claude --resume` silent exit -1 lesson); no 60 s human-response windows (§11.4.50 determinism violation); re-runnability proof — PASS at `-count=3` consecutive automated invocations with self-cleaning state; §11.4.98 obsolescence audit classifies every existing test COMPLIANT vs NON-COMPLIANT; no silent-skip-reported-as-PASS or stale-evidence-as-fresh. (D) With §11.4.85 + §11.4.89 + §11.4.87 + §11.4.94 forms a continuously-validated, non-flake, anti-bluff regime. (F) Manual-dependency tests not rewritten within 30 days graduate to §11.4.90 Obsolete citing §11.4.98.

Cascade requirement: This anchor (verbatim or by `§11.4.98` reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. Propagation gate `CM-COVENANT-114-98-PROPAGATION`; paired mutation strips the literal → gate FAILs. Release blocker. **Canonical authority:** constitution submodule `Constitution.md` §11.4.98 for the full mandate.

---

§11.4.99 — Latest-Source Documentation Cross-Reference Mandate (cascaded from constitution submodule §11.4.99)

> Verbatim user mandate (2026-05-28): *"Make sure we ALWAYS check against latest versions of services we use web / online docs before creating instructions! This situation is illustration of how we can misguide ourselves or get banned! ... These are mandatory rules / constraints and the result is consistency and safety of created instructions, guides and manuals!"*

Misguidance-by-stale-docs is the same severity class as a §11.4 PASS-bluff at the documentation layer (Herald 2026-05-28 case: a first-draft MTProto guide recommended VoIP fallback numbers and omitted the `recover@telegram.org` pre-login email — both contradicted Telegram's official docs + the gotd/td maintainer guide and could have caused a permanent account ban). Closes the gap §11.4.92 Pass 4 alludes to but does not mandate. (A) Before committing any operator-facing instruction/guide/manual/troubleshooting/setup doc, the author MUST: (1) fetch the LATEST official online documentation of the documented service/library via WebFetch / MCP / direct browsing — NEVER training data, memory, or prior committed docs; (2) cross-reference every instruction step against that source; (3) seek secondary authoritative sources (maintainer SUPPORT.md, official changelogs, vetted community FAQs) when the official source is sparse/silent; (4) cite source URLs + date in a `## Sources verified` footer in the doc; (5) cite a `Sources`

`verified <date>: <urls>` footer in the commit message. (B) Negative findings (gaps/silences/contradictions) MUST be documented explicitly. (C) Docs older than 6 months are STALE — re-verify before citing as operator authority, at every vN.0.0 release boundary, on service breaking-change announcements, or on operator error reports. (D) Risk-classified services (messengers, cloud APIs, payment systems, AI/LLM providers, code-hosting, package managers) carry a 90-day max staleness + explicit safety warnings. (E) Composes with but is INDEPENDENT of §11.4.92 Pass 4. (G) Commit missing either footer is BLOCKED at release-gate; stale-beyond-grace docs graduate to §11.4.90 Obsolete (`Reason=stale-documentation`).

Cascade requirement: This anchor (verbatim or by `§11.4.99` reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. Propagation gate `CM-COVENANT-114-99-PROPAGATION`; paired mutation strips the literal → gate FAILs. Release blocker. **Canonical authority:** constitution submodule `Constitution.md` §11.4.99 for the full mandate.

---

§11.4.101 — Autonomous-Decision-Over-Blocking Mandate (cascaded from constitution submodule §11.4.101)

> Verbatim user mandate (2026-05-28): *"when working in endless working loop fully autonomously try to decide most properly about points which would block execution and wait for us. If we haven't answered now work would be blocked whole night! If possible and if that will not cause any issues make proper and most reliable and safe decision so we achieve maximal efficiency and work gets fully done!"*

In autonomous / endless-loop mode (per §11.4.87), the agent MUST minimize operator-blocking and make the safe, reliable, reversible decision itself so work is not stalled (e.g. overnight) waiting for input — §11.4.87 says keep working, §11.4.101 says HOW to clear the decision points. **Proceed-autonomously (closed-set, ALL must hold):** (a) the action is reversible OR has a captured pre-op backup per §9.2; (b) the safe choice is determinable from captured evidence per §11.4.6 (no guessing — `LIKELY`/`probably`/`seems` is NOT a determination); (c) a wrong choice's blast radius is bounded AND recoverable; (d) it composes with anti-bluff §11.4, host-safety §12, data-safety §9. **Block-only-when (BLOCK via the §11.4.66 interactive mechanism ONLY when ALL hold):** the action is irreversible AND high-blast-radius AND the safe choice cannot be determined from evidence — e.g. external-account state the agent cannot inspect, hardware it cannot access, destructive ops without backup, force-push (also §9.2 + §11.4.41), spending money or sending data to third parties. `Operator-blocked` per §11.4.21 is reached only after this rule fires AND the self-resolution-exhaustion audit completes. An unavoidable block parks one work unit — it does NOT pause the loop; the agent keeps progressing every non-blocked item in parallel per §11.4.87 + §11.4.94 (posing the question then going idle is a §11.4.94 + §11.4.97 violation). Classification: universal (§11.4.17).

Cascade requirement: This anchor (verbatim or by `§11.4.101` reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. Propagation gate `CM-COVENANT-114-101-PROPAGATION`; paired mutation strips the literal → gate FAILs. Release blocker. **Canonical authority:** constitution submodule `Constitution.md` §11.4.101 for the full mandate.

---

§11.4.102 — Mandatory systematic-debugging activation + always-loaded skill-discovery + plugin-dependency availability (cascaded from constitution submodule §11.4.102)

> Verbatim user mandate (2026-05-29): *"Make sure that we ALWAYS trigger / start the"/superpowers:systematic-debugging" skills when any issues happen! If this is possible to activate and use in this situations out of the box when we spot problems / issues / bugs / misalignments / unconsistencies we MUST activate the skill(s) and make strongest efforts in full in depth analisys / debugging and determine root causes of all problem or obtain relevant data and information we need! … we MUST make sure that "/using-superpowers" skill is ALWAYS loaded, applied and used! All dependencies (plugins) that Claude Code or other market places are offering MUST BE installed if these are not already available for loading and use!"

Three cooperating invariants — the difference between guess-and-retry and investigate-to-root-cause-first. **(A) Mandatory systematic-debugging activation.** On ANY spotted issue / bug / test failure / gate failure / regression / misalignment / inconsistency / unexpected behaviour, the agent MUST activate `superpowers:systematic-debugging` (or the platform-equivalent structured-debugging discipline) **BEFORE proposing, writing, or applying any fix** — the **Iron Law: NO FIXES WITHOUT ROOT CAUSE INVESTIGATION FIRST.** Full four-phase arc: root-cause → pattern → hypothesis → implementation. Guess-and-retry, symptom-patching, and re-running a failed test hoping it passes ("probably transient / flaky") WITHOUT a completed investigation are §11.4.102 violations; calling a failure `transient`/`flaky`/`intermittent`/`probably-timing` without captured forensic evidence is simultaneously a §11.4.6 and §11.4.7 violation. **(B) Mandatory always-loaded `using-superpowers`.** `superpowers:using-superpowers` (or platform-equivalent skill-discovery discipline) MUST be loaded and applied at session start and consulted before any task; if ANY skill could apply — even at 1% relevance — it MUST be invoked rather than improvised from memory. **(C) Mandatory plugin / dependency availability.** Every skill plugin / marketplace package / capability dependency the project relies on MUST be installed + loadable BEFORE the dependent work proceeds; a missing plugin that blocks a mandated skill is a release-blocker until installed + confirmed loadable (install exit 0 ≠ skill loadable — confirm by observing the skill in the live capability list). Composes with §11.4.4 / §11.4.6 / §11.4.7 / §11.4.8 / §11.4.43 / §11.4.70 / §11.4.82(A) / §11.4.92. Classification: universal (§11.4.17). No escape hatch — no `--skip-systematic-debugging`, `--guess-and-retry-OK`, `--symptom-patch-permitted`, `--skip-skill-discovery`, `--plugin-optional`, `--missing-plugin-is-warning` flag.

Cascade requirement: This anchor (verbatim or by `§11.4.102` reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. Propagation gate `CM-COVENANT-114-102-PROPAGATION`; paired mutation strips the literal → gate FAILs. Release blocker. **Canonical authority:** constitution submodule `Constitution.md` §11.4.102 for the full mandate.

§11.4.103 — Continuous parallel-stream working routine (User mandate, 2026-05-29)

Cascaded from constitution submodule §11.4.103. Promotes the multi-stream operating pattern into the project's standing default working routine. The main work stream MUST always stay FREE; ALL commit AND push operations run detached. At least three parallel background subagent streams MUST run at all times alongside the main stream whenever three-plus non-contending actionable items exist; the moment any stream finishes a new stream MUST immediately start. Most-critical + most-visible first; audio always top per §11.4.72. Safe-during-build scope only (§11.4.96 SAFE catalogue). Heavy anti-bluff on every closure. Idle ONLY when genuinely externally blocked OR operator STOP OR §12 host-safety.

Cascade requirement: This anchor (verbatim or by `§11.4.103` reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. Propagation gate `CM-COVENANT-114-103-PROPAGATION`; paired mutation strips the literal → gate FAILs. Release blocker. **Canonical authority:** constitution submodule `Constitution.md` §11.4.103 for the full mandate.

§11.4.104 — Participant identity, attribution & notification-tagging (User mandate, 2026-05-31)

Cascaded from constitution submodule §11.4.104. Every messenger/notification surface MUST relate messages to a Participant (logical Subscriber/User); the same logical person MAY have a different username per messenger. Workable items MUST carry `created_by` + `assigned_to` (canonical handles). Notification tagging MUST tag `assigned_to` / `created_by` when human and not Operator and not Claude; NEVER tag Claude (system) or the Operator (no self-ping). Operator is designated by `HERALD_<CHANNEL>_OPERATOR_USERNAME` env var, not a DB flag.

Cascade requirement: This anchor (verbatim or by `§11.4.104` reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. Propagation gate `CM-COVENANT-114-104-PROPAGATION`; paired mutation strips the literal → gate FAILs. Release blocker. **Canonical authority:** constitution submodule `Constitution.md` §11.4.104 for the full mandate.

§11.4.105 — Natural-language intent recognition & clarification (User mandate, 2026-05-31)

Cascaded from constitution submodule §11.4.105. Users MUST NOT be required to know command syntax. Three-tier resolution: TIER 1 — recognize existing commands from natural language; TIER 2 — infer exact intent via LLM dispatch;

TIER 3 — reply, tag sender (`@username`), and ask a precise clarifying question. Never guess, never drop a message silently; only genuine ambiguity reaches Tier 3, which always replies-tags-and-asks.

Cascade requirement: This anchor (verbatim or by `§11.4.105` reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. Propagation gate `CM-COVENANT-114-105-PROPAGATION`; paired mutation strips the literal → gate FAILs. Release blocker. **Canonical authority:** constitution submodule `Constitution.md` §11.4.105 for the full mandate.

§11.4.106 — Docs Chain — mechanical documentation/DB sync engine (Operator mandate, 2026-05-31)

Cascaded from constitution submodule §11.4.106. `vasic-digital/docs_chain` is the canonical mechanical enforcer of documentation-sync mandates. Consumers MUST use the engine (referenced, never copied), register chains via `.docs_chain/contexts/*.yaml`, and never accept a faked transform. The engine mechanizes §11.4.12/.53/.45/.56/.57/.59/.60/.65/.86/.93/.95/.44/§12.10. A missing pandoc/weasyprint surfaces a typed `ToolAbsentError` + honest SKIP-with-reason, never a fake PASS.

Cascade requirement: This anchor (verbatim or by `§11.4.106` reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. Propagation gate `CM-COVENANT-114-106-PROPAGATION`; paired mutation strips the literal → gate FAILs. Release blocker. **Canonical authority:** constitution submodule `Constitution.md` §11.4.106 for the full mandate.

§11.4.107 — Anti-bluff AV/test-validation techniques mandate (User-driven research, 2026-06-02)

Cascaded from constitution submodule §11.4.107. Every test asserting audio/video output is genuinely playing MUST satisfy: single captured frame NOT proof — prove LIVE ADVANCING frames via freeze-detection oracle; independent frame-advance counter from compositor/decoder telemetry; loading/buffering is a distinct state; not-stale-from-previous cross-check; measured FPS / no-lost-frames; no-flash-on-wrong-output; drive through realistic feed/UI path; metamorphic relations; full-reference quality metrics vs golden source; mutation-test every analyzer with golden-good + golden-bad fixture pair; per-channel audio RMS/loudness + XRUN census; OCR confidence floor + ROI; thresholds calibrated on project's own fixtures.

Cascade requirement: This anchor (verbatim or by `§11.4.107` reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. Propagation gate `CM-COVENANT-114-107-PROPAGATION`; paired mutation strips the literal → gate FAILs. Release blocker. **Canonical authority:** constitution submodule `Constitution.md` §11.4.107 for the full mandate.

§11.4.108 — Four-layer fix-verification + runtime-signature-as-definition-of-done mandate (systematic-debugging Phase 4.5, 2026-06-03)

Cascaded from constitution submodule §11.4.108. A fix crosses FOUR distinct layers: (1) SOURCE, (2) ARTIFACT, (3) RUNTIME-ON-CLEAN-TARGET, (4) USER-VISIBLE. Green at layer 1 says nothing about layers 2–4. Every fix declares ONE machine-checkable runtime signature verified on a CLEAN/fresh deployment. Gates span all four layers. Deployment MUST yield a CLEAN state OR a pre-validation assertion proves running-artifact == built-artifact. On ≥3 "fixed-but-not-working" discoveries in one cycle: STOP patching symptoms, fix the VERIFICATION pipeline.

Cascade requirement: This anchor (verbatim or by `§11.4.108` reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. Propagation gate `CM-COVENANT-114-108-PROPAGATION`; paired mutation strips the literal → gate FAILs. Release blocker. **Canonical authority:** constitution submodule `Constitution.md` §11.4.108 for the full mandate.

§11.4.109 — Mandatory Anti-Forgetting Enforcement: PreToolUse Guard Hook + Subagent Constitutional Preamble + Orchestrator Pre-Action Checklist (Operator mandate)

Cascaded from constitution submodule §11.4.109. A `PreToolUse` guard hook (`constitution/scripts/hooks/guard-forbidden-commands.sh`) MUST be wired in `.claude/settings.json` blocking host-direct emulator, force-push/bypass, sudo, and host-power commands. `docs/AGENT_GUARDRAILS.md` MUST contain the `SUBAGENT CONSTITUTIONAL PREAMBLE` and `ORCHESTRATOR PRE-ACTION CHECKLIST` headings with anchor literal `11.4.109`. A hermetic hook test suite (≥20 cases) is required. The hook is inherited by reference — NEVER copied locally.

Cascade requirement: This anchor (verbatim or by `§11.4.109` reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. Propagation gate `CM-COVENANT-114-109-PROPAGATION`; paired mutation strips the literal → gate FAILs. Release blocker. **Canonical authority:** constitution submodule `Constitution.md` §11.4.109 for the full mandate.

§11.4.110 — Pre-build build-readiness verdict + change-impact clash detection mandate (operator mandate, 2026-06-03)

Cascaded from constitution submodule §11.4.110. A single deterministic READY-FOR-BUILD verdict gates every rebuild. A diff-driven change-impact + clash detector cross-checks every newly-introduced second-artifact dependency (new property read ⇄ property-context type + read-grant; new service ⇄ service-context

entry; etc.). Coverage-completeness is a gate — every changed file maps to ≥1 gate + ≥1 deployed-target test + ≥1 paired §1.1 mutation. Two-speed honesty: grep-speed always-on gates vs REQUIRES_BUILD heavy gates as diff-gated opt-in stages.

Cascade requirement: This anchor (verbatim or by `§11.4.110` reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. Propagation gate `CM-COVENANT-114-110-PROPAGATION`; paired mutation strips the literal → gate FAILs. Release blocker. **Canonical authority:** constitution submodule `Constitution.md` §11.4.110 for the full mandate.

§11.4.111 — Resolve-by-stable-name-not-by-enumeration-index mandate (research-derived, 2026-06-03)

Cascaded from constitution submodule §11.4.111. Any binding to a hardware device / resource handle / enumerated entity MUST resolve by a stable identifier (name / UUID / serial / label / controller-name / content-hash / sink-reported identity) and MUST NOT bind by enumeration index / ordinal / slot, UNLESS the platform documents that ordinal as deterministically pinned AND the pin is itself captured + asserted as part of the binding. Where a stable identifier exists at one layer, every other layer binding the same resource MUST use the same identifier.

Cascade requirement: This anchor (verbatim or by `§11.4.111` reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. Propagation gate `CM-COVENANT-114-111-PROPAGATION`; paired mutation strips the literal → gate FAILs. Release blocker. **Canonical authority:** constitution submodule `Constitution.md` §11.4.111 for the full mandate.

§11.4.112 — Structural-impossibility won't-fix classification mandate (research-derived, 2026-06-03)

Cascaded from constitution submodule §11.4.112. When deep research per §11.4.8 PROVES a goal is structurally impossible on the target platform (forbidden by platform design / hardware-protocol constraint / documented kernel-or-API limitation), the goal MUST be: classified `Won't-fix` + closed per §11.4.90 with closure reason `structurally-impossible`; documented with impossibility evidence; NOT re-attempted without NEW evidence the platform constraint changed. `structurally-impossible` is reserved for PROVEN platform/hardware/protocol impossibility — "could not find a way" is Operator-blocked, not won't-fix.

Cascade requirement: This anchor (verbatim or by `§11.4.112` reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. Propagation gate `CM-COVENANT-114-112-PROPAGATION`; paired mutation strips the literal → gate FAILs. Release blocker. **Canonical authority:** constitution submodule `Constitution.md` §11.4.112 for the full mandate.

§11.4.113 — Absolute no-force-push + merge-onto-latest-main mandate (User mandate, 2026-06-03)

Cascaded from constitution submodule §11.4.113. Force-push is STRICTLY FORBIDDEN with NO exception — `git push --force`, `--force-with-lease`, `+<ref>`, or any history-rewriting overwrite of a remote ref, against EVERY repository. The mandated 6-step integration procedure: (1) `git fetch --all --prune --tags`; (2) set base to LATEST commit on canonical `main`/`master`; (3) carefully MERGE every change on top; (4) resolve every conflict carefully; (5) commit the merge (stage only intended files); (6) push to ALL upstreams as fast-forward. REMOVES the force-push escape hatch from §11.4.41/§11.4.71/§9.2/ CONST-043.

Cascade requirement: This anchor (verbatim or by `§11.4.113` reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. Propagation gate `CM-COVENANT-114-113-PROPAGATION`; paired mutation strips the literal → gate FAILs. Release blocker. **Canonical authority:** constitution submodule `Constitution.md` §11.4.113 for the full mandate.

§11.4.114 — Last-known-good-tag regression isolation mandate (1.1.8-dev remediation, 2026-06-03)

Cascaded from constitution submodule §11.4.114. When a previously-working feature is observed broken, the FIRST diagnostic action MUST be to identify the last release tag at which it was KNOWN-GOOD and diff/bisect the broken state against it — BEFORE any open-ended root-cause hunt or speculative fix. The known-good revision is the regression oracle. Default to a SURGICAL forward-fix (keep post-good-tag features, revert ONLY the broken sub-part) over a wholesale revert. "It worked before" is a HYPOTHESIS until the known-good tag is identified and confirmed.

Cascade requirement: This anchor (verbatim or by `§11.4.114` reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. Propagation gate `CM-COVENANT-114-114-PROPAGATION`; paired mutation strips the literal → gate FAILs. Release blocker. **Canonical authority:** constitution submodule `Constitution.md` §11.4.114 for the full mandate.

§11.4.115 — RED-baseline-on-the-broken-artifact + polarity-switch mandate (1.1.8-dev remediation, 2026-06-03)

Cascaded from constitution submodule §11.4.115. Every RED test MUST be authored to REPRODUCE the defect on the CURRENT pre-fix artifact, capturing positive evidence that the defect is genuinely present. The SAME test source carries a single polarity switch (env flag `RED_MODE`, default `1` = reproduce-and-assert-

defect-present; flipped to `0` post-fix = standing GREEN regression-guard). One source, two roles: the bug-catcher IS the regression-guard. A RED test that passes on the known-broken artifact is a blind test — a finding, not evidence.

Cascade requirement: This anchor (verbatim or by `§11.4.115` reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. Propagation gate `CM-COVENANT-114-115-PROPAGATION`; paired mutation strips the literal → gate FAILs. Release blocker. **Canonical authority:** constitution submodule `Constitution.md` §11.4.115 for the full mandate.

§11.4.116 — Real-time conductor⁉autonomous-test-framework sync channel mandate (1.1.8-dev remediation, 2026-06-03)

Cascaded from constitution submodule §11.4.116. Any autonomous long-running test/QA/validation framework MUST expose: (1) a structured append-only JSONL event stream emitting session-start / phase-transition / per-test-start / captured-evidence-path / verdict events; (2) an atomically-rewritten status snapshot (write-temp-then-rename). Every PASS verdict event MUST carry the evidence path — a PASS event with no evidence path is a channel-layer PASS-bluff. A snapshot reporting PASS while the stream shows no evidence event is a contradiction → treat as FAIL.

Cascade requirement: This anchor (verbatim or by `§11.4.116` reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. Propagation gate `CM-COVENANT-114-116-PROPAGATION`; paired mutation strips the literal → gate FAILs. Release blocker. **Canonical authority:** constitution submodule `Constitution.md` §11.4.116 for the full mandate.

§11.4.117 — Computer-vision / OCR pixel-oracle fallback for non-introspectable UIs mandate (1.1.8-dev remediation, 2026-06-03)

Cascaded from constitution submodule §11.4.117. Any test needing to drive a UI control OR assert on-screen content MUST NOT assume the accessibility/semantic/DOM hierarchy is the source of truth. When the hierarchy is blank/partial/known-unreliable, the test MUST fall back to a PIXEL ORACLE: drive input by computer-vision template-match; assert content by ROI OCR with per-word confidence floor + region-of-interest. The CV/OCR analyzer is self-validated — golden-good fixture PASSes, golden-bad fixture FAILs, wired into meta-test.

Cascade requirement: This anchor (verbatim or by `§11.4.117` reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. Propagation gate `CM-COVENANT-114-117-PROPAGATION`; paired mutation strips the literal → gate FAILs. Release blocker. **Canonical authority:** constitution submodule `Constitution.md` §11.4.117 for the full mandate.

§11.4.118 — Discovery-pressure to confirm known-issue-set completeness mandate (1.1.8-dev remediation, 2026-06-03)

Cascaded from constitution submodule §11.4.118. A remediation/release cycle MUST NOT treat "every reported defect is fixed" as "the build is good." After/alongside fixing the reported set, the cycle MUST run a discovery + stress pass across ALL target devices/environments that deliberately exercises subsystems, journeys, and edge cases BEYOND the reported defects. The pass MUST produce an enumerated list of subsystems/user-journeys/stress scenarios actually exercised, each with its outcome. "We found no other issues" is a bluff unless accompanied by "here is the enumerated set we exercised."

Cascade requirement: This anchor (verbatim or by `§11.4.118` reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. Propagation gate `CM-COVENANT-114-118-PROPAGATION`; paired mutation strips the literal → gate FAILs. Release blocker. **Canonical authority:** constitution submodule `Constitution.md` §11.4.118 for the full mandate.

§11.4.119 — Single-resource-owner partitioning for parallel hardware testing mandate (1.1.8-dev remediation, 2026-06-03)

Cascaded from constitution submodule §11.4.119. When multiple parallel streams exercise SHARED hardware or any exclusive-access resource, exactly ONE stream MUST own each such resource at a time. The exclusive owner drives it; every other concurrent stream targeting the same resource MUST be READ-ONLY (passive probes only). Parallelism is partitioned by resource: distinct devices/sinks run fully concurrent, but the same device's exclusive resource is single-owner. Ownership enforced by advisory lock/token. Concurrent drivers of one exclusive resource produce cross-contaminated evidence — a PASS under contention is a §11.4 evidence-integrity bluff.

Cascade requirement: This anchor (verbatim or by `§11.4.119` reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. Propagation gate `CM-COVENANT-114-119-PROPAGATION`; paired mutation strips the literal → gate FAILs. Release blocker. **Canonical authority:** constitution submodule `Constitution.md` §11.4.119 for the full mandate.

§11.4.120 — Fix-breaks-its-own-gate reconciliation mandate (1.1.8-dev remediation, 2026-06-03)

Cascaded from constitution submodule §11.4.120. When a correct fix causes a pre-existing gate/test to FAIL because that gate asserted the OLD (now-removed) behaviour, the required response is RECONCILIATION: rewrite the gate to assert the NEW mechanism the fix introduced, backed by captured evidence, AND update its paired §1.1 mutation. The two forbidden responses are: (1) FAKE-PASSING the

gate (a §11.4 gate-layer bluff); (2) REVERTING the correct fix. After reconciliation the gate + mutation still form a valid §1.1 pair. Reconcile ONLY when investigation PROVES the gate asserted old-correct-now-removed behaviour.

Cascade requirement: This anchor (verbatim or by `§11.4.120` reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. Propagation gate `CM-COVENANT-114-120-PROPAGATION`; paired mutation strips the literal → gate FAILs. Release blocker. **Canonical authority:** constitution submodule `Constitution.md` §11.4.120 for the full mandate.

§11.4.121 — No-commit-while-build-writes-tracked-artifacts mandate (1.1.8-dev remediation, 2026-06-03)

Cascaded from constitution submodule §11.4.121. A commit (especially `git add -A` / any broad stage) MUST NOT run while a build/packaging/generation step is actively writing artifacts into tracked (version-controlled) directories — doing so races the writer and stages a PARTIAL or stale artifact. The commit MUST be deferred until the build step that writes tracked artifacts has COMPLETED. Before committing tracked build outputs, verify the writing step finished (process exit / completion marker / per-artifact mtime ≥ build-start). A build still in flight writing tracked dirs is a HOLD on the commit, not a race to win.

Cascade requirement: This anchor (verbatim or by `§11.4.121` reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. Propagation gate `CM-COVENANT-114-121-PROPAGATION`; paired mutation strips the literal → gate FAILs. Release blocker. **Canonical authority:** constitution submodule `Constitution.md` §11.4.121 for the full mandate.

§11.4.122 — No-silent-removal-of-existing-components-without-operator-confirmation mandate (User mandate, 2026-06-03)

Forensic anchor — verbatim user mandate (2026-06-03):

> "Never ever remove any application, system component or service from already existing codebase / System without interactively asked question to us! THIS IS MANDATORY RULE / CONSTRAINT!"

Forensic case study (FACT). During the 1.1.8-dev burn-down, two shipped capabilities — F2 (an Apple-TV-class application) and F4 (a Huawei HMS / Mobile-Services component) — were removed from the existing System WITHOUT first asking the operator; the operator reversed both. A removal the operator has to discover and reverse after the fact is a defect of the same severity class as a §11.4 PASS-bluff: the System silently lost a user-facing capability the operator never agreed to drop.

No application, system component, service, package, feature, driver, module, library, prebuilt asset — any already-existing end-user capability of the existing codebase / shipped System — may be removed (deleted, dropped from the package set, disabled-into-non-shipping, un-bundled, de-listed, or otherwise made unavailable to the end user) WITHOUT FIRST interactively asking the operator and receiving an

EXPLICIT keep-or-remove decision. The question MUST be posed through the platform's interactive clarification mechanism per §11.4.66 (`AskUserQuestion` on Claude Code) — NEVER a free-text "should I remove X?" buried in narrative, NEVER a silent removal justified post-hoc, NEVER an autonomous removal decision. A silent removal is a **release blocker** regardless of how well-intentioned the rationale (deduplication, "it was broken anyway", geo-restricted, incompatible, superseded) — the operator decides, the agent asks.

What counts as a removal (non-exhaustive): deleting an app/APK/binary from the build's package set (`PRODUCT_PACKAGES` / `device.mk` / equivalent), removing a service from the init/boot/service-registry set, dropping a kernel module / driver / config from the shipping configuration, un-bundling a prebuilt asset, deleting a submodule or its shipped output, removing a feature flag that gated a live capability, or any edit whose NET EFFECT is "an end-user capability that shipped before no longer ships." Adding, replacing-with-operator-approved-equivalent, or fixing a capability is NOT a removal. When uncertain whether an edit constitutes a removal, treat it AS a removal and ask (per §11.4.6 no-guessing + §11.4.101 — removal of an existing user-facing capability is high-blast-radius and MUST be operator-confirmed, never autonomously decided). The tracked DROP path: ask → operator approves → mark the item `Obsolete (→ Fixed.md)` with `Obsolete-Details` reason `feature-removed` + an operator-approval citation (§11.4.90) → then remove; the removal never precedes the operator's yes.

Classification: universal (§11.4.17) — a platform-neutral discipline reusable by ANY project that ships a set of user-facing capabilities; the consuming project supplies its concrete capability-manifest paths per §11.4.35. Composes §11.4.66 / §11.4.101 / §11.4.90 / §11.4.112 / §11.4.6 / §11.4.40 / §11.4.42. Propagation gate `CM-COVENANT-114-122-PROPAGATION` (literal `11.4.122`) + recommended gate `CM-NO-SILENT-COMPONENT-REMOVAL` + paired §1.1 meta-test mutation (gate-code = separate work item).

Canonical authority: constitution submodule `Constitution.md` §11.4.122. Non-compliance is a release blocker. No escape hatch — no `--remove-without-asking`, `--silent-removal`, `--autonomous-removal-OK`, `--dedup-removal-exempt`, `--it-was-broken-anyway` flag.

§11.4.123 — Rock-solid-proof-or-deep-research mandate (User mandate, 2026-06-03)

Forensic anchor — verbatim user mandate (2026-06-03):

> "Every single reported issue MUST BE fully and 100% validated with rock solid proofs! Nothing can be considered fixed or completed without hard evidence! No false results or bluff(s) of any kind is allowed! If we are not sure on how to achieve full testing, validation and verification of something we MUST ALWAYS perform deep web research for all possible data (articles, documentation, guides, and other resources) and opensourced codebases which we can use to solve our problems and perform testing with validation and verification which produces rock-solid evidence(s) and leaves no space for false results or any kind of bluff!"

Forensic case study (FACT). In the 1.1.8-dev remediation the validation method for two feature classes was, at first, genuinely unclear: relocating a `FLAG_SECURE` secure surface to a secondary display (pixel capture returns black) and asserting on-

screen content in non-introspectable streaming-app UIs (blank accessibility hierarchy). Rather than declaring them “untestable” or accepting a metadata-only PASS, the cycle performed deep web research (`docs/research/testing_frameworks_20260603/`) that yielded the CV/OCR/liveness/sink-probe oracle stack (now §11.4.107 + §11.4.112 + §11.4.117) — making rock-solid evidence possible where it had appeared impossible. “Unclear how to validate” is a research trigger, NEVER a bluff licence.

Every single reported issue, every fix, and every claimed completion MUST be fully and 100% validated with rock-solid CAPTURED proof per §11.4.5 / §11.4.69 / §11.4.107 before it may be marked fixed / implemented / completed (§11.4.33 closure vocabulary). Nothing may be considered fixed or complete without hard captured evidence — metadata-only / configuration-only / absence-of-error / grep-without-runtime PASS are all forbidden (§11.4 / §11.4.1); no false results, no bluff of any kind, at any layer.

The research-or-don’t-bluff rule (the operative addition): when the agent is UNSURE how to fully test / validate / verify something — when no obvious evidence-producing method exists OR the candidate method would yield only metadata/config/absence-of-error evidence — it MUST ALWAYS first perform deep web research per §11.4.8 + §11.4.99 (official docs, articles, guides, vendor references, standards, issue trackers, reusable open-source codebases) to DISCOVER or BUILD a validation method that produces rock-solid evidence and leaves no space for a false result. Declaring something “untestable” / “not automatable” / accepting a metadata-only PASS WITHOUT first exhausting this deep-research path is itself a §11.4.123 violation — same severity class as a PASS-bluff. The research output (cited source URLs + the evidence-producing method, OR the literal “NO external solution found — original work” per §11.4.8) is the captured proof the path was exhausted. Only after that research genuinely fails may the item be classified `PENDING_FORENSICS:` / `Operator-blocked` (§11.4.21) / `structurally-impossible` won’t-fix (§11.4.112) — with the cited research as the evidence the classification is earned, never a convenience.

Classification: universal (§11.4.17) — a platform-neutral discipline reusable by ANY project; the consuming project supplies its concrete capture mechanisms + research corpora per §11.4.35. Composes §11.4.5 / §11.4.6 / §11.4.8 / §11.4.52 / §11.4.69 / §11.4.99 / §11.4.107 / §11.4.118 / §11.4.21 / §11.4.112. Propagation gate `CM-COVENANT-114-123-PROPAGATION` (literal `11.4.123`) + recommended gate `CM-ROCK-SOLID-PROOF-OR-RESEARCH` + paired §1.1 meta-test mutation (gate-code = separate work item).

Canonical authority: constitution submodule `Constitution.md` §11.4.123. Non-compliance is a release blocker. No escape hatch — no `--metadata-pass-suffices`, `--skip-proof`, `--untestable-without-research`, `--config-only-closure-OK`, `--bluff-when-unsure` flag.

§11.4.124 — Dead/unwired-code investigate-before-remove mandate (User mandate, 2026-06-04)

Forensic anchor — verbatim user mandate (2026-06-04):

> “Before removing any seemingly-dead (zero-importer / unwired) codebase, we MUST investigate via git history where/how it was originally used and how it became dead. Removal is permitted ONLY when we have captured

> PROOF it is genuinely no longer needed — and that removal MUST be its own separate commit with a proper descriptive message. If there is no such proof, the code MUST be investigated for where/how it should be wired in properly, and any missing or unwired tests MUST be added. We MUST ALWAYS be extra careful with any codebase removal."

"Zero importers / never called / unwired ⇒ dead ⇒ delete" is a GUESS (§11.4.6), never a finding — a "no references" result proves only *current* non-reference, not genuinely-unneeded. Before removing ANY seemingly-dead element (zero-importer / never-called / unwired function / method / type / file / module / package / asset / config / build target) the agent MUST FIRST investigate via git history (`git log --follow`, `git log -S/-G` pickaxe across all history, blame on the deleted call-site) and capture as FACT: (1) WHERE/HOW it was originally wired in, (2) WHEN/HOW it became dead — call-site deleted deliberately / by mistake (regression) / never-completed / refactored-unreachable, (3) whether "no references" is real OR a hidden reference the static tool cannot see (reflection / dynamic dispatch / build-tags / codegen / DI / plugin registry / FFI / config-driven wiring). The investigation output (cited commits + determination) is the captured evidence. **Removal is conditional:** permitted ONLY with captured PROOF the element is genuinely no longer needed; that removal MUST be its OWN SEPARATE COMMIT (independently reviewable + revertible, composes §11.4.84 quiescence + §11.4.92 multi-pass) with a descriptive message citing the git-history evidence — plus §11.4.122 operator-confirmation when the element is an end-user capability; the §11.4.90 tracked path marks it `Obsolete (→ Fixed.md)`. **No proof ⇒ do NOT delete:** investigate WHERE/HOW to wire it in properly (restore a mistakenly-deleted call-site per §11.4.114; finish never-completed wiring) AND add any missing / unwired tests (§11.4.27 / §11.4.43 / §11.4.115 — the missing test is part of why it drifted into apparent-deadness). **Extra-caution default:** when uncertain whether removal-proof is sufficient, default to NOT removing (investigate + wire + test) per §11.4.6 + §11.4.101 + §11.4.122; "probably dead" is never sufficient — the bar is captured proof. Classification: universal (§11.4.17) — the consuming project supplies its static-analysis / importer-graph tooling + hidden-reference mechanisms per §11.4.35. Composes §11.4.6 / §11.4.8 / §11.4.84 / §11.4.90 / §11.4.92 / §11.4.101 / §11.4.114 / §11.4.122 / §11.4.27 / §11.4.43 / §11.4.115. Propagation gate `CM-COVENANT-114-124-PROPAGATION` (literal `11.4.124`) + recommended gate `CM-DEAD-CODE-INVESTIGATE-BEFORE-REMOVE` (a net-deletion commit must be removal-only + cite the git-history investigation OR be part of a tracked Obsolete item) + paired §1.1 meta-test mutation (gate-code = separate work item).

Canonical authority: constitution submodule `Constitution.md` §11.4.124. Non-compliance is a release blocker. No escape hatch — no `--zero-importers-means-dead`, `--delete-unwired-on-sight`, `--skip-git-history-investigation`, `--remove-without-proof`, `--bundle-removal-with-other-work` flag.

§11.4.125 — Code-review-agent gate before pre-build + main build (mandatory multi-layer review) (User mandate, 2026-06-04)

Forensic anchor — verbatim user mandate (2026-06-04):

> "After all fixes/changes/implementations are done, BEFORE running pre-build tests and the main build, dispatch code-review agent(s) that analyze all work done + all existing data/facts + the existing codebase + current git history to determine quality, safety, and whether the fixes/changes will

> REALLY work; they MUST validate and verify that every test covering the fixes/changes genuinely validates the work with NO chance of false results or bluff of any kind. Any finding MUST be fixed, polished, improved, and covered with additional tests before the build proceeds. Multiple strong layers of checks.”

After all fixes / changes / implementations in a batch are done, and BEFORE running the pre-build test sweep AND the main (artifact) build (for ANY project), the agent MUST dispatch one or more dedicated code-review agent(s) (subagent-driven by default per §11.4.70/§11.4.20) performing a multi-layer review that: (1) analyzes ALL work done in the batch (every fix/change + its source diff + stated intent); (2) analyzes ALL existing data + facts (captured evidence per §11.4.5/§11.4.69/§11.4.107, tracker entries, prior findings, the §11.4.108 runtime-signature registry); (3) analyzes the existing codebase (blast radius per §11.4.92, cross-feature interaction, contract integrity of every dependency); (4) analyzes current git history (what each change touched, how it composes with concurrent/recent work, whether it reproduces a known-broken pattern per §11.4.114/§11.4.124); (5) determines quality + safety + will-it-REALLY-work (robust + not error-prone — no solve-A-create-B; no host/data/security regression; genuinely delivers the end-user-visible behaviour per §11.4/§107); (6) validates + verifies the tests covering the work — every covering test genuinely exercises the work-under-test and catches its negation, with ZERO chance of a false result or bluff (a test that PASSes on broken-for-the-user work, a metadata-only/config-only/absence-of-error/grep-without-runtime assertion, or a gate whose paired §1.1 mutation does not make it FAIL is a finding). Any finding (defect / error-prone change / safety risk / will-not-really-work / bluff-or-false-result-capable test / missing-coverage gap) MUST be fixed, polished, improved, and covered with additional tests (four-layer per §11.4.4(b), TDD-RED-first per §11.4.43/§11.4.115) BEFORE the pre-build sweep + main build proceed; the review iterates (re-review after each remediation) until no blocking findings remain. The review is itself anti-bluff (its conclusions are captured evidence per §11.4.5/§11.4.69; a rubber-stamp review of a defective batch = PASS-bluff). It is one of MULTIPLE STRONG LAYERS — complementing, never replacing, the §1 pre-build sweep, §11.4.92 multi-pass (author-side self-review; §11.4.125 adds the structurally-separated reviewer seam per §11.4.70), §11.4.108 four-layer fix-verification, §11.4.110 build-readiness verdict, and the post-build / runtime-on-clean-target / user-visible layers. Composes §11.4 / §11.4.1 / §11.4.4 / §11.4.6 / §11.4.40 / §11.4.43 / §11.4.50 / §11.4.70 / §11.4.20 / §11.4.92 / §11.4.102 / §11.4.107 / §11.4.108 / §11.4.110. Classification: universal (§11.4.17). Propagation gate `CM-COVENANT-114-125-PROPAGATION` (literal `11.4.125`) + recommended gate `CM-CODE-REVIEW-GATE-BEFORE-BUILD` (build starts only with a fresh code-review-completed marker for the current batch, produced after the last fix + before the pre-build sweep + main build) + paired §1.1 mutation (gate-code = separate work item).

Canonical authority: constitution submodule `Constitution.md` §11.4.125. Non-compliance is a release blocker. No escape hatch — no `--skip-code-review`, `--build-without-review`, `--no-review-gate`, `--review-optional`, `--trust-the-author` flag.

§11.4.126 — Default autonomous-loop working mode from first prompt (User mandate, 2026-06-04)

Forensic anchor — verbatim user mandate (2026-06-04):

> “Make sure that you continue work in endless fully autonomous loop, do not stop until new fully validated and verified version (tag) is created and published (all submodules and main repo) or IN A CASE OF some other main stream work until it is fully completed with all side work streams and nothing else is left in our working queue! THIS MUST BE ALWAYS the default working mode without us asking you! We tend to achieve ABSOLUTE EFFICIENCY, with this and all other projects which will incorporate this MANDATORY RULE / CONSTRAINT!!! This way of (your) working will be ALWAYS applied / followed / executed / fully respected, as soon as we assign / send first request (prompt) in the session! This stops only if we explicitly say so or nothing is left to be done in current working scope (release that will come / upcoming version)!!! Any mimicking (imitation) of this behavior / rules / mandatory constraints, false results or any kind of bluff(s) is ABSOLUTELY FORBIDDEN!!!”

The endless fully-autonomous loop is the **DEFAULT working mode**, engaged automatically the moment the operator sends the FIRST request / prompt of a session — the operator MUST NOT have to ask for it, request it, restate it, or re-enable it per session. §11.4.87 framed the endless-loop covenant as an explicit-instruction opt-in (“continue in endless loop fully autonomously” or a semantically-equivalent phrasing); §11.4.126 is the **capstone** that promotes the same covenant to always-on: from the first prompt onward, every agent operates in the §11.4.87 loop discipline as the standing default, with §11.4.94 zero-idle, §11.4.97 maximum-idle-use, §11.4.101 autonomous-decision-over-blocking, and §11.4.103 continuous-parallel-stream all engaged by default — no per-session activation handshake. The continuation contract: the loop continues until ONE of two terminal conditions holds — (A) **Release scope** — a new, fully-validated-and-verified version (tag) is created AND published across all owned submodules AND the main repo to all configured remotes (per §2.1 multi-upstream push + §11.4.40 full-suite-retest-before-tag + §11.4.113 absolute-no-force-push merge-onto-latest-main); OR (B) **Non-release main-stream scope** — the main-stream goal is fully completed AND every side work stream is done AND the working queue holds nothing left for the current scope. Until (A) or (B) holds, the agent MUST keep working (claim the next priority item, dispatch the next parallel stream, progress every non-blocked item per §11.4.42 / §11.4.72 / §11.4.94 / §11.4.103). The loop STOPS ONLY on: (1) the operator explicitly saying so (STOP / pause / end); (2) nothing left to do in the current working scope — the upcoming release / current main-stream goal — with the queue genuinely empty per the (A)/(B) terminal conditions; (3) a §12 host-session-safety demand (the loop yields to host safety unconditionally). Idle-while-blocked parks one work unit, it does not stop the loop — the agent keeps progressing every non-blocked item in parallel per §11.4.101 + §11.4.94 + §11.4.97. Goal — ABSOLUTE EFFICIENCY (no operator-side restart overhead, no idle gaps, no stop-and-wait round-trips); applies to this project AND every project that incorporates this Constitution. Anti-bluff: mimicking / imitating this loop behaviour, narrating continuation without performing it, fabricating progress, or emitting false / bluff results of ANY kind is ABSOLUTELY FORBIDDEN — this composes the entire §11.4 anti-bluff covenant family (§11.4 / §11.4.1 / §11.4.2 / §11.4.5 / §11.4.6 / §11.4.50 / §11.4.69 / §11.4.107); the agent MUST genuinely perform the continuous work and capture positive evidence for every closure, and a report claiming the loop ran while no real work / no captured evidence was produced is a §11.4 PASS-bluff at the operating-mode layer. Classification: universal (§11.4.17). Composes with §11.4.87 (the endless-loop covenant — §11.4.126 promotes it from opt-in to always-on default) / §11.4.94 / §11.4.97 / §11.4.101 / §11.4.103 / §11.4.66 / §11.4.6 / §11.4.40 / §11.4.42 / §11.4.72 / §11.4.113 / §2.1 / §12. Propagation gate `CM-`

COVENANT-114-126-PROPAGATION (literal 11.4.126 across the consumer fleet) + paired §1.1 meta-test mutation (strip the literal → propagation gate FAILs; gate-code = separate work item).

Canonical authority: constitution submodule Constitution.md §11.4.126. Non-compliance is a release blocker. No escape hatch — no --ask-before-continuing, --single-turn-only, --not-default-loop, --mimic-OK flag.

§11.4.127 — Session-handoff resumption-prompt mandate (User mandate, 2026-06-06)

Forensic anchor — verbatim user mandate (2026-06-06): "make sure that in situations like this now when new session is needed you ALWAYS prepera such sentence - which will be valid for particular moment and the phase of the project and enough for work to continue."

When the agent determines a fresh session is needed (context-window limits, performance degradation) OR the operator asks whether a new session is needed / requests a handoff, the agent MUST ALWAYS prepare + proactively provide a ready-to-paste **resumption prompt valid for that EXACT moment and project phase** — self-contained enough that pasting it into a fresh session resumes work with ZERO loss. Two variants on demand: a SHORT first-sentence ("Read <handoff docs>, then continue <terminal goal> …") AND a FULL detailed block. The prompt MUST: (1) point to the live handoff doc(s) — .remember/remember.md if present + docs/CONTINUATION.md per §12.10 — read FIRST + git fetch --all; (2) state current PHASE + immediate NEXT action + terminal goal; (3) embed exact live-state anchors (build IDs / artifact MD5, device/target serials, commit HEAD, in-flight PIDs + log paths, captured-evidence paths); (4) restate binding constraints (anti-bluff §11.4, no-force-push §11.4.113, exact version/naming, hardware/target gotchas); (5) be MOMENT-VALID, NEVER a generic template. Handoff doc(s) MUST be current BEFORE the prompt is given (§12.10). A missing / stale / generic prompt is a §11.4.127 violation. Composes §12.10 / §11.4.6 / §11.4.66 / §11.4.87 / §11.4.103 / §11.4.126. Classification: universal (§11.4.17). Propagation gate CM-COVENANT-114-127-PROPAGATION (literal 11.4.127) + paired §1.1 meta-test mutation.

Canonical authority: constitution submodule Constitution.md §11.4.127. Non-compliance is a release blocker. No escape hatch — no --skip-handoff-prompt, --generic-prompt-OK, --no-resumption-sentence, --handoff-without-state flag.

§11.4.128 — Always-on device-recording mandate (User mandate, 2026-06-06)

Forensic anchor — direct user mandate (2026-06-06): we MUST ALWAYS live-record all available data from all devices we use for testing (or known to be under manual testing), EXTRA carefully so it never harms the device / its performance / causes side effects; raw recordings are NOT processed without need (token-conscious) and are ALWAYS git-ignored + code-intelligence-excluded; only curated evidence is committed, and only at release prep.

For EVERY test/debug device the project uses + every device under known manual testing, across EVERY reachable transport (USB / wireless ADB / SSH / serial / network introspection API), the project MUST ALWAYS live-record all analysable data: activities, all logs, performance metrics (CPU/memory/I/O/thermal/load), every sink-side report per §11.4.13, and any other live-changeable parameter. (1) **Extra-careful, side-effect-free** — non-invasive read-only probes only, bounded sampling, bounded write-volume, an observer-effect budget; a recorder that perturbs the device-under-test is a §11.4.128 violation, NOT evidence. (2) **Background + parallel + subagent-driven** per §11.4.103 + §11.4.70 — never blocks the main stream. (3) **Token-conscious — record-now, analyse-later** — raw data NOT processed without need; the only standing analyse-trigger is release-tag prep (§11.4.40 / §11.4.42) OR explicit operator ask. (4) **Raw is git-ignored (with a §11.4.77 regen-mechanism declaration) AND code-intelligence-excluded (§11.4.78/§11.4.79)** — only CURATED evidence is committed, and only at release prep under `docs/qa/<run-id>/` (§11.4.83). (5) **Deterministic layout** `<recording-root>/YYYY-MM-DD/<combined main+submodules state hash>/<DEVICE>_<SERIAL>/recording_NNN/<files>`. (6) **Anti-bluff** — a recorder claimed running but with no growing corpus is a §11.4 bluff; every curated finding traces to a real raw-corpus path; recorder health is itself captured evidence per §11.4.5/§11.4.69.

Composes §11.4.2 / §11.4.5 / §11.4.13 / §11.4.69 / §11.4.40 / §11.4.42 / §11.4.70 / §11.4.77 / §11.4.78 / §11.4.79 / §11.4.83 / §11.4.103 / §11.4.119. Classification: universal (§11.4.17). Propagation gate `CM-COVENANT-114-128-PROPAGATION` (literal `11.4.128`) + recommended gate `CM-DEVICE-RECORDING-ALWAYS-ON` + paired §1.1 mutation.

Canonical authority: constitution submodule `Constitution.md` §11.4.128. Non-compliance is a release blocker. No escape hatch — no `--skip-recording`, `--record-without-layout`, `--commit-raw-corpus`, `--index-raw-corpus`, `--analyse-corpus-always`, `--invasive-probe-OK` flag.

§11.4.129 — Huge-blocker release protocol (User mandate, 2026-06-06)

Forensic anchor — direct user mandate (2026-06-06): when a huge blocker is discovered during release validation we MUST stop all testing, fix ALL discovered issues, process all recorded data from the last session, land rock-solid fixes, author NEW validation+verification tests of ALL supported test types, rebuild, reflash, and RESTART the full validation+verification of every fix/change from the last release tag to now — on both devices in parallel, recorded, with real physical captured proofs and no bluff.

On discovery of a HUGE BLOCKER (release-blocking-severity defect: core user-facing capability broken, regression invalidating the in-flight cycle, or blast radius reaching the batch's other fixes) during release validation, execute in order with NO spot-check shortcut: (1) **STOP all testing** on every device (the §11.4.4 test-interrupt STOP at release granularity — continuing past a huge blocker is the §11.4 PASS-bluff). (2) **Fix ALL discovered issues** — not just the blocker; root-cause each per §11.4.102 + isolate regressions against the last known-good tag per §11.4.114. (3) **Process all recorded data from the last session** — analyse the §11.4.128 raw-corpus slice (this IS the §11.4.128(3) release-prep analyse-trigger). (4) **Land rock-solid fixes** per §11.4.123 + §11.4.43/§11.4.115 + §11.4.9. (5) **Author NEW validation+verification tests of ALL supported test types** per §11.4.27 + §11.4.85,

each anti-bluff + paired §1.1 mutation. (6) **Rebuild (full, not module-only) + reflash to a CLEAN target** per §11.4.108. (7) **RESTART the full validation+verification from the last release tag to now** per §11.4.40 — RESTART, never resume — on both/all owned devices IN PARALLEL per §11.4.103/§11.4.119, every run RECORDED per §11.4.128, real physical captured proofs per §11.4.5/§11.4.69/§11.4.107, no bluff. This anchor BINDS the existing release anchors for the huge-blocker case (adds STOP→fix-all→process-recordings→new-tests-all-types→rebuild→reflash→full-restart + the restart-not-resume rule), citing them rather than duplicating.

Composes §11.4.4 / §11.4.40 / §11.4.42 / §11.4.9 / §11.4.27 / §11.4.85 / §11.4.102 / §11.4.108 / §11.4.114 / §11.4.115 / §11.4.123 / §11.4.128 / §11.4.103 / §11.4.119. Classification: universal (§11.4.17). Propagation gate `CM-COVENANT-114-129-PROPAGATION` (literal `11.4.129`) + recommended gate `CM-HUGE-BLOCKER-FULL-RESTART` + paired §1.1 mutation.

Canonical authority: constitution submodule `Constitution.md` §11.4.129. Non-compliance is a release blocker. No escape hatch — no `--resume-after-blocker`, `--spot-validate-after-fix`, `--skip-recording-analysis`, `--skip-new-tests`, `--module-only-after-blocker`, `--single-device-restart` flag.

§11.4.130 — Post-remediation validate-the-fix-FIRST-after-redeploy (User mandate, 2026-06-06)

Forensic anchor — direct user mandate (2026-06-06): when a blocker discovered during release validation is fixed and a new artifact (rebuild / new flashing image / redeploy) is produced + the target reflashed, we MUST first re-test the SPECIFIC last-failing features + validate the just-incorporated fixes BEFORE the broader / full validation.

When a blocker / critical failure found during release validation is FIXED and a new artifact is produced + the target reflashed / redistributed / updated, the agent MUST: (1) **re-test the SPECIFIC last-failing features FIRST** (targeted guard tests for exactly the defects this fix addressed) BEFORE any broader / full-suite validation; (2) **validate the just-incorporated fixes with real captured evidence** — the §11.4.115 RED test flips GREEN at `RED_MODE=0` on the new artifact AND the §11.4.108 runtime-signature verifies on the CLEAN target the redeploy produced (metadata-only / config-only / absence-of-error / grep-without-runtime PASS forbidden per §11.4 / §11.4.1; proof per §11.4.5/§11.4.69/§11.4.107/§11.4.123); (3) **only after the targeted fix is CONFIRMED working** proceed to the §11.4.40 full retest from the last tag to now. Rationale: a first fix attempt may not work / may be incomplete / may regress again under the new artifact — confirming the targeted fix FIRST catches a fix-did-not-take case immediately instead of hours later at the END of a full cycle (then restarting per §11.4.129); cheap-confirmation-first is §11.4.82 applied to the post-blocker reflash. This is the §11.4.46 recent-work-validation gate specialised for the post-blocker-reflash case + the targeted-confirmation phase that GATES §11.4.129's step-7 full-restart. Honest boundary (§11.4.6): "the fix probably took" ≠ "the fix took" — the RED→GREEN flip + runtime-signature on the new artifact is the proof; a still-FAILing targeted re-test re-enters the §11.4.114/§11.4.115 isolate→RED→fix loop, never proceeds to the full cycle on a still-broken fix. Composes §11.4.4 / §11.4.40 / §11.4.46 / §11.4.108 / §11.4.114 / §11.4.115 / §11.4.123 / §11.4.129 / §11.4.82. Classification: universal (§11.4.17). Propagation gate `CM-COVENANT-114-130-PROPAGATION` (literal `11.4.130`) + recommended gate `CM-FIX-FIRST-AFTER-REDEPLOY` + paired §1.1 mutation.

Canonical authority: constitution submodule `Constitution.md` §11.4.130. Non-compliance is a release blocker. No escape hatch — no `--skip-targeted-retest`, `--full-cycle-first`, `--assume-fix-took`, `--validate-fix-at-end`, `--skip-red-green-flip-on-new-artifact` flag.

§11.4.131 — Standing session-resumption file mandate (User mandate, 2026-06-07)

Forensic anchor — verbatim user mandate (2026-06-07): "Make this markdown a standard file which will be written EVERY TIME when we need fresh session out of the box! It MUST BE always up to date and in sync so whenever new session is created all we have to do is just point to it!"

Every project MUST maintain a SINGLE canonical, always-current **session-resumption file** at a fixed, project-declared standard path (declared once per §11.4.35, never moved without a §11.4.66 operator decision). This file is the OUT-OF-THE-BOX entry point for any fresh session: creating a new session requires ONLY pointing the new agent at this one file. §11.4.131 promotes §11.4.127 (PREPARE a resumption prompt on demand) into a STANDING, version-controlled ARTIFACT — ALWAYS present, ALWAYS in sync. (A) **Existence + fixed path** — exists at the declared path at all times, encoded as a literal path in the project-layer instantiation (§11.4.35), never silently moved. (B) **Always written + always synced** — (re)written whenever a fresh session is needed OR the live state materially changes (new HEAD, build/artifact id, phase, device/target state, in-flight job, blocking decision) — the §12.10 trigger set; a stale resumption file is a §11.4.131 violation of the same severity class as a §12.10 stale-CONTINUATION violation. (C) **Content (composes §11.4.127)** — both SHORT + FULL variants; points to `.remember/remember.md` + `docs/CONTINUATION.md` read FIRST + `git fetch`; embeds exact live-state anchors (HEAD, build/artifact ids + checksums, device serials, in-flight PIDs + log paths, captured-evidence paths); states PHASE + immediate NEXT + terminal goal; restates binding constraints (anti-bluff §11.4, no-force-push §11.4.113, exact version/naming, hardware gotchas); MOMENT-VALID, never a generic template (§11.4.6). (D) **Export + freshness** — §11.4.65 scope (synchronized `.html`/`.pdf` siblings) + §11.4.44 revision header. (E) **Out-of-the-box resumption** — a fresh session, given ONLY this file's path, fully resumes with zero additional context. Composes §12.10 / §11.4.127 / §11.4.65 / §11.4.44 / §11.4.6 / §11.4.66 / §11.4.126. Classification: universal (§11.4.17). Propagation gate `CM-COVENANT-114-131-PROPAGATION` (literal `11.4.131`) + recommended gate `CM-SESSION-RESUMPTION-FILE-PRESENT` + paired §1.1 meta-test mutation.

Canonical authority: constitution submodule `Constitution.md` §11.4.131. Non-compliance is a release blocker. No escape hatch — no `--skip-resumption-file`, `--ephemeral-prompt-only`, `--stale-resumption-OK`, `--generic-template-OK` flag.

§11.4.132 — Risk-ordered validation priority mandate (User mandate, 2026-06-07)

Forensic anchor — verbatim user mandate (2026-06-07): "We MUST ALWAYS first test and validate features, functionalities and fixes/changes that have been worked most recently, the ones which were most problematic, which have the most chance to crash or break again, the ones which have been re-opened the most times!

Then, after we validate and verify all this with real (physical) proofs and hard evidence, with no false results and bluffs of any kind, we continue with all other existing tests in the test suites! This IS MANDATORY."

Tests / validations / verifications MUST run in **RISK-DESCENDING order** — the highest-risk set FIRST, and ONLY AFTER that set is fully GREEN with real (physical) captured evidence does the remainder of the suite run. Risk ranking is computed from a CLOSED set of factors, highest-risk first: (a) **most-recently-worked** features / fixes / changes; (b) **historically most-problematic** (longest defect history, most prior fixes/failures); (c) **highest crash/break/regress likelihood** (greatest blast radius / complexity / dependency surface); (d) **most-reopened** per §11.4.55 reopens-count (a high reopen count is the strongest empirical fragility signal). Each item in the highest-risk set MUST pass with real (physical) captured evidence per §11.4.5/§11.4.69/§11.4.107 — no metadata-only / config-only / absence-of-error / grep-without-runtime PASS (§11.4/§11.4.1), no false results, no bluff (§11.4.6). ONLY AFTER the entire highest-risk set is GREEN with captured proof does the rest of the suite run; running the suite in arbitrary order, or running lower-risk tests before the highest-risk set is GREEN, is a §11.4.132 violation. §11.4.132 REFINES/STRENGTHENS §11.4.130 (generalises "validate the just-fixed items first" to the full risk-ordered set) + §11.4.46 (adds explicit risk-ordering within the recent/high-risk set) + §11.4.42 (applies the implementation-layer priority discipline to VALIDATION ordering). Classification: universal (§11.4.17) — the consuming project supplies its recency / problematic-history / reopen-count sources (e.g. §11.4.93 workable-items DB `reopens_count`+`last_modified`) per §11.4.35. Composes §11.4.4/.5/.6/.7/.40/.42/.46/.50/.55/.69/.107/.130. Propagation gate `CM-COVENANT-114-132-PROPAGATION` (literal `11.4.132`) + recommended gate `CM-RISK-ORDERED-VALIDATION-PRIORITY` + paired §1.1 meta-test mutation.

Canonical authority: constitution submodule `Constitution.md` §11.4.132. Non-compliance is a release blocker. No escape hatch — no `--skip-risk-ordering`, `--any-order-OK`, `--suite-order-fixed` flag.

§11.4.133 — Target-System + hardware safety mandate (User mandate, 2026-06-08)

Forensic anchor — verbatim user mandate (2026-06-08): "Make sure that all changes we do to the System are ALWAYS safe for the System itself and for the hardware the system runs on! This is MANDATORY."

Every change to the TARGET system (firmware, kernel, init/boot scripts, drivers, sysfs/devfreq/voltage/clock/thermal/regulator register writes, partition/bootloader/U-Boot, HAL, framework, prebuilts, device config) MUST ALWAYS be safe for BOTH (a) the target System itself — MUST NOT brick, boot-loop, corrupt data, or render the device unrecoverable — AND (b) the hardware it runs on — MUST NOT exceed safe electrical/thermal/voltage/clock limits or damage panels/storage/radios/regulators. Concrete obligations: (1) reversible-first — verify irreversible high-blast-radius changes (bootloader/U-Boot MD5, partition layout) against known-good values + capture a pre-op backup (§9.2) BEFORE applying; (2) NO unverified hardware-control writes — never write an unverified value to a voltage/clock/regulator/thermal-throttle/current-limit sysfs node or register that could exceed datasheet limits, the safe range established as FACT (§11.4.6), never guessed; (3) thermal/perf changes (forcing a performance governor, pinning the top OPP, disabling thermal management) MUST respect the device's cooling design, validated by captured thermal evidence; (4) flashing MUST use the sanctioned tool

+ a freshly-built integrity-verified image — never an ad-hoc partition write or stale/unverified artifact; (5) unprovable-safety ⇒ blocked — a change whose target/hardware safety cannot be established from captured evidence is treated as UNSAFE and blocked (§11.4.6 + §11.4.101 reversible-first + §11.4.123 rock-solid-proof). DISTINCT from §12 host-session safety: §12 protects the DEVELOPER's HOST + session; §11.4.133 protects the TARGET device + its hardware — both apply, neither weakens the other. Classification: universal (§11.4.17) — the consuming project supplies its concrete hardware-control surfaces, datasheet-safe ranges, known-good bootloader/image hashes, and sanctioned flashing tool per §11.4.35. Composes §12 / §11.4.6 / §11.4.101 / §11.4.108 / §11.4.123. Propagation gate `CM-COVENANT-114-133-PROPAGATION` (literal `11.4.133`) + recommended gate `CM-TARGET-HARDWARE-SAFETY` + paired §1.1 meta-test mutation.

Canonical authority: constitution submodule `Constitution.md` §11.4.133. Non-compliance is a release blocker. No escape hatch — no `--unsafe-hardware-write`, `--skip-system-safety`, `--brick-risk-accepted` flag.

§11.4.134 — Code-review iterate-until-GO + rock-solid-evidence mandate (User mandate, 2026-06-08)

Forensic anchor — verbatim user mandate (2026-06-08): "For any fixes/changes given back to us for re-work by the code-review process, once we fix/improve everything per the code-review's requests, we MUST RE-RUN code-review AGAIN until we get a GO from it with NO new issues reported or warnings of any kind! All results produced by this whole process MUST ALWAYS give us rock-solid PHYSICAL evidence that the fixed/improved codebase really works now as expected, with no false results and no bluff(s) of any kind."

When the §11.4.125 code-review returns ANY finding — BLOCKING, nit, or warning — and the author fixes/improves the batch per that review, the code review MUST BE RE-RUN, and MUST KEEP being re-run after each remediation round, until it returns a clean GO with ZERO new issues AND ZERO warnings of any kind. A single pass that "addressed the findings" is NOT sufficient: the corrected batch MUST pass a FRESH adversarial review (a re-review can surface NEW findings introduced by the very fixes that closed the prior ones — the §11.4.1 fix-A-creates-B failure mode). The loop terminates ONLY on a clean GO (no new findings, no warnings); a residual warning is itself a finding that re-arms the loop. Every round's verdict AND every fix's validation MUST carry rock-solid PHYSICAL captured evidence per §11.4.5 / §11.4.69 / §11.4.107 (captured audio / video / sysfs / dumpsys / sink-side / runtime-signature) proving the fixed/improved codebase REALLY works as expected — never metadata-only / configuration-only / absence-of-error / grep-without-runtime; no false results, no bluff at any round; a reported GO unbacked by captured physical evidence is itself a §11.4 PASS-bluff at the review-loop layer. §11.4.134 REFINES / STRENGTHENS §11.4.125 (iterate "until no blocking findings remain"): it makes the loop EXPLICIT (re-run after every remediation round, not once), raises termination to ZERO findings AND ZERO warnings (not merely zero-blocking), and BINDS rock-solid physical evidence to every round. Classification: universal (§11.4.17). Composes §11.4.125 / §11.4.1 / §11.4.4 / §11.4.5 / §11.4.6 / §11.4.69 / §11.4.107 / §11.4.50 / §11.4.108 / §11.4.123. Propagation gate `CM-COVENANT-114-134-PROPAGATION` (literal `11.4.134`) + recommended gate `CM-CODE-REVIEW-ITERATE-UNTIL-GO` + paired §1.1 meta-test mutation (gate-code = separate work item).

Canonical authority: constitution submodule `Constitution.md` §11.4.134. Non-compliance is a release blocker. No escape hatch — no `--skip-rereview`, `--single-review-pass`, `--warnings-ok`, `--evidence-optional` flag.

§11.4.135 — Standing regression-guard suite + every-fixed-defect-gets-a-permanent-regression-test (User mandate, 2026-06-08). Every project MUST maintain a STANDING regression-guard suite that runs on EVERY build+deploy and BLOCKS the release tag on any failure. Every closed defect (stable ticket id, e.g. ATM-NNN) MUST, in the SAME commit as its fix (extending the §11.4.43 DOCUMENT step), register a permanent §11.4.115 RED-on-broken-artifact regression test into the suite — `RED_MODE=1` capturing the historical defect on a pre-fix artifact (the proof the guard is real), `RED_MODE=0` the standing GREEN guard asserting the defect is ABSENT. A closure without a registered guard is a §11.4.123 violation. The suite runs FIRST in the post-deploy cycle (highest-risk set per §11.4.132) and is a §11.4.40 release-gate blocker. Forensic anchor (FACT): the wrong-subtitle-on-2nd-display defect was "fixed" via a source-side `CONTROL_MENU_LABEL_DENYLIST` that NO test mirrored or re-ran, so the NEXT chrome class recurred silently while the GREEN suite passed. Industry-standard bug-driven testing (Google content-driven testing; AOSP CTS/Tradefed) made mechanical + enforced. Composes §11.4.4 / §11.4.40 / §11.4.43 / §11.4.46 / §11.4.50 / §11.4.107 / §11.4.108 / §11.4.115 / §11.4.118 / §11.4.123 / §11.4.124 / §11.4.130 / §11.4.132. Classification: universal (§11.4.17). Propagation gate `CM-COVENANT-114-135-PROPAGATION` (literal `11.4.135`) + recommended gates `CM-REGRESSION-GUARD-REGISTERED` / `CM-REGRESSION-GUARD-SUITE-WIRED` + paired §1.1 mutation. **Canonical authority:** constitution submodule `Constitution.md` §11.4.135. Non-compliance is a release blocker. No escape hatch — no `--skip-regression-guard`, `--no-guard-on-close`, `--guard-optional` flag.

§11.4.136 — Real-content end-to-end playback-test mandate (User mandate, 2026-06-08). Refines/strengthens §11.4.107. Any test asserting media playback works MUST drive REAL content (catalog stream or offline reference clip) through the user's path (§11.4.48 UI-driven → §11.4.117 CV/OCR fallback) and assert it genuinely PLAYS via the §11.4.107 liveness battery PLUS a decoder-health census — a numeric drop-buffer budget, no buffer-timestamp re-order/discard, no codec-reject (cite Android/Media3 ExoPlayer OEM pre-OTA playback-test mandate: "too many dropped buffers" >25, "unexpected presentation timestamp", "test timed out"). Metadata-only / launch-only / registration-only / single-frame / config-only PASS is forbidden (§11.4 / §11.4.1). A golden/reference clip corpus (BBC ExoPlayer testing samples) is the offline ground-truth. Composes §11.4.5 / §11.4.48 / §11.4.50 / §11.4.107 / §11.4.117 / §11.4.123 / §11.4.13 / §11.4.69. Classification: universal (§11.4.17). Propagation gate `CM-COVENANT-114-136-PROPAGATION` (literal `11.4.136`) + recommended gate `CM-REAL-CONTENT-PLAYBACK-TEST` + paired §1.1 mutation. **Canonical authority:** constitution submodule `Constitution.md` §11.4.136. Non-compliance is a release blocker. No escape hatch — no `--launch-proves-playback`, `--skip-decoder-health`, `--metadata-playback-pass-suffices` flag.

§11.4.137 — Subtitle/caption content-correctness oracle + secure-display-proxy-honesty mandate (User mandate, 2026-06-08). Refines §11.4.117 + §11.4.107 + §11.4.112. Forensic anchor (FACT): tests tasked to "physically verify the 2nd-display subtitle" PASSed GREEN while subtitles did NOT show / showed WRONG — the streaming player surface is FLAG_SECURE so `screencap -d <secondary>` returns BLACK (autonomous PIXEL verification structurally impossible per §11.4.112), so the test fell back to the accessibility-scraped/

persist.atmosphere.subdebug proxy, and the proxy accepted a chrome/menu LABEL (Аудио и субтитры) as a valid subtitle because the prose floor accepted any multibyte prose and NO menu-label denylist + NO position/cadence check existed. The mandate: a subtitle-correctness test MUST classify the cue's *content class* — a present cue is NOT a correct cue. CHROME (FAIL) if a known control/menu label (closed multilingual deny-list MIRRORED from source, case-folded incl. non-ASCII), time/numeric chrome, not prose, OUTSIDE the lower safe-title band (CEA-708 9-anchor grid), OR STATIC across the window (real subtitle changes → ≥2 distinct prose cues, a metamorphic relation). DIALOGUE (PASS) only when prose + not-denied + not-chrome + position-ok + cadence ≥2 OR fuzzy-matches the SOURCE-extracted cue via normalized edit distance (§11.4.123 host ground truth). The oracle MUST be self-validated golden-good/golden-bad (§11.4.107(10)) and the deny-list MUST be verified present in the SHIPPED artifact (§11.4.108) — a source-green denylist with no test mirror + no artifact check is the exact recurrence pattern forbidden here. Secure-display honesty (§11.4.112): where FLAG_SECURE makes pixel verification impossible, the rock-solid autonomous proof is the player's caption telemetry + source-track presence + content-class oracle — NEVER a faked pixel "physical" pass; human-eye pixel confirmation is operator_attended (§11.4.52) with a tracked migration item. App-agnostic (keys off content class). Composes §11.4.3 / §11.4.5 / §11.4.6 / §11.4.107 / §11.4.108 / §11.4.112 / §11.4.115 / §11.4.117 / §11.4.123 / §11.4.13 / §11.4.69. Classification: universal (§11.4.17). Propagation gate CM-COVENANT-114-137-PROPAGATION (literal 11.4.137) + recommended gate CM-SUBTITLE-CONTENT-CORRECTNESS-ORACLE + paired §1.1 mutation (strip the denylist/position/cadence check → golden-bad Аудио и субтитры PASSes → gate FAILs). **Canonical authority:** constitution submodule Constitution.md §11.4.137. Non-compliance is a release blocker. No escape hatch — no --present-cue-is-correct, --skip-chrome-oracle, --length-heuristic-suffices, --pixel-pass-on-secure-display, --skip-position-check, --skip-cadence-check flag.

§11.4.138 — Operator-escape => mandatory bluff-audit + permanent guard (User mandate, 2026-06-08). When the operator (or any out-of-band channel) finds a defect that the GREEN test suite passed, this is by definition a §11.4 PASS-bluff — it MUST trigger, before the fix is closed: (1) a §11.4.102 systematic-debugging pass to FACT-root-cause; (2) a bluff-audit identifying the EXACT assertion that should have caught it but didn't, cited to file:line (canonical example: lib/subtitle_content_validation.sh:sub_is_prose() returning TRUE for Аудио и субтитры); (3) a permanent §11.4.135 regression guard registered in the SAME commit as the fix, with its §11.4.115 RED capturing the operator-found defect; (4) the bluff-audit committed under docs/research/<scope>/<defect>_bluff_audit/. Closing an operator-found defect WITHOUT the bluff-audit + permanent guard is itself a §11.4 violation (the bluff that let it through is still live and the defect will recur). Composes §11.4 / §11.4.1 / §11.4.102 / §11.4.108 / §11.4.115 / §11.4.118 / §11.4.123 / §11.4.135. Classification: universal (§11.4.17). Propagation gate CM-COVENANT-114-138-PROPAGATION (literal 11.4.138) + recommended gate CM-OPERATOR-ESCAPE-BLUFF-AUDIT + paired §1.1 mutation. **Canonical authority:** constitution submodule Constitution.md §11.4.138. Non-compliance is a release blocker. No escape hatch — no --close-without-bluff-audit, --operator-find-is-just-a-bug, --skip-permanent-guard flag.

§11.4.139 — Fresh-process clean-artifact runtime-signature mandate (User mandate, 2026-06-08). Refines §11.4.108. Before any post-deploy validation — ESPECIALLY a non-pixel proxy verification (the subdebug/accessibility-cue channel used for FLAG_SECURE displays) — the harness MUST assert running-artifact == built-artifact: the deploy yielded a CLEAN target (mutable-overlay/

userdata wiped) OR a pre-validation check proves no stale overlay shadows the deployed code (e.g. every guarded package — incl. the Presenter that emits the subtitle cue — resolves to the system partition, no per-user override). A stale shadow of the cue-emitting component (e.g. a Presenter APK predating the denylist) makes the proxy report on code that was never deployed — any PASS is a §11.4 PASS-bluff. Each fix declares ONE machine-checkable runtime signature verified on the clean target (the §11.4.108 registry IS the definition of done); for the subtitle class the signature is "the shipped Presenter APK contains the denylist literal (case-insensitive) AND the subdebug channel emits `candidate REJECTED reason=chrome-label` for a menu label." Composes §11.4.46 / §11.4.108 / §11.4.130 / §11.4.135 / §11.4.137. Classification: universal (§11.4.17). Propagation gate `CM-COVENANT-114-139-PROPAGATION` (literal `11.4.139`) + recommended gate `CM-CLEAN-ARTIFACT-RUNTIME-SIGNATURE` + paired §1.1 mutation. **Canonical authority:** constitution submodule `Constitution.md` §11.4.139. Non-compliance is a release blocker. No escape hatch — no `--validate-against-running-state`, `--skip-clean-precondition`, `--shadow-OK` flag.

§11.4.140 & §11.4.141 — action-prefix system + token-efficiency (cascaded from constitution submodule 60e2d66, CONST-047/049)

> **§11.4.140 — Universal action-prefix system (`ACTION_NAME ::`) (User mandate, 2026-06-09; GRAMMAR_ADDENDUM 2026-06-09).** When a user prompt's FIRST non-blank line starts with a recognised action prefix, you MUST: (1) look the action token up in the action registry `constitution/actions/registry.yaml` (or `$HELIX_ACTION_REGISTRY`); (2) if it is a registered action, REPLACE the prefix with that action's `expansion` text and apply its `rules`; (3) execute the remainder of the prompt under the expanded instruction. **Four EQUIVALENT forms** — same action, same expansion, same execution: (1) `ACTION_NAME :: <rest>` (bare `::`), (2) `PREFIX::ACTION_NAME :: <rest>` (namespaced `::`), (3) `/ACTION_NAME <rest>` (bare slash), (4) `/PREFIX::ACTION_NAME <rest>` (namespaced slash). Thus `BACKGROUND :: x` ≡ `DEFAULT::BACKGROUND :: x` ≡ `/BACKGROUND x` ≡ `/DEFAULT::BACKGROUND x`. `PREFIX` is an action NAMESPACE; the reserved default namespace is **`DEFAULT`**, and an action runs WITH or WITHOUT the prefix. Grammar (all hold): anchored at the FIRST non-blank line only (mid-prose tokens never match); the action token AND the namespace are UPPERCASE-only `[A-Z][A-Z0-9_]*` (lowercase never matches); the namespace separator `::` inside the token carries NO surrounding spaces (`PREFIX::ACTION_NAME`), DISTINCT from the action-body separator `" :: "` (one ASCII space on each side of `::` — avoids C++ `Foo::Bar`, YAML `key: value`, URLs) in forms 1/2 and the slash-body separator (one space) in forms 3/4; stacked prefixes (`A :: B :: rest`) apply outer-to-inner, left-to-right (expand `A`, re-scan, expand `B`, then the residual is the task); a leading `\` escapes the prefix for BOTH the `::` and the slash form (`\BACKGROUND :: x`, `\/BACKGROUND x` — treat literally, strip the backslash, NO expansion) so action names can be discussed. **Conflict rule (slash form):** `/ACTION_NAME` (form 3) is honored as the action ONLY when `ACTION_NAME` (case-folded) does not collide with a built-in/host slash command (registry `slash_bare: auto` + `slash_conflicts: [..]`); form 4 (`/PREFIX::ACTION_NAME`) is ALWAYS unambiguous and always honored. An unknown token that matches the grammar shape (any of the 4 forms) but

> is NOT registered is NEVER silently expanded or silently dropped — ask which registered action was meant (§11.4.66 / §11.4.105) or treat it as a literal prompt, NEVER invent an expansion (§11.4.6); any prompt not satisfying the grammar is an ordinary prompt and the system is a no-op. The registered action **`BACKGROUND`** expands to: *"The following prompt that we will provide MUST BE executed in background in parallel with all main work streams using the subagents-driven development approach! All work done MUST PRODUCE rock solid evidence covered with hard physical proof(s) that all done is working as expected and as specified without any false results and without any bluff!"* (composes §11.4.20 / §11.4.70 subagent-driven, §11.4.58 / §11.4.103 parallel streams, §11.4.89 background execution, §11.4.5 / §11.4.69 / §11.4.107 captured physical evidence, §11.4 anti-bluff). The system is UNIVERSAL (every CLI agent reads this block via its context carrier per §11.4.35), extensible (new action = new registry row), decoupled + reusable (§11.4.28), and loads out-of-the-box. Classification: universal (§11.4.17). **Canonical authority:** constitution submodule `Constitution.md` §11.4.140. Non-compliance is a release blocker. No escape hatch — no `--skip-action-prefix`, `--ignore-prefix`, `--no-registry`, `--invent-expansion-OK`, `--single-layer-only` flag.

§11.4.141 — Token-efficiency mandate (research-derived + operator mandate, 2026-06-09). Every project worked on by AI coding agents MUST cut token spend (input AND output) toward **30–40% of current (a 60–70% reduction)** WITHOUT degrading quality/performance/safety or breaking any existing mechanism, via a composable, safety-ranked measure set: (1) **prompt-cache the static governance prefix** — the always-loaded governance forms a byte-stable cache-breakpointed prefix with no volatile bytes ahead of it; cache reads cost ~0.1× base input (the dominant cost driver — measured ~170K tokens of governance re-sent every turn, externally corroborated by Claude Code issue #24147); caching is transparent so it removes no rule, weakens no gate, changes no verdict — only billing (PRIMARY, biggest + safest lever); (2) **subagent model-tiering + output-to-file** — mechanical non-judgment work (search/grep/status/doc-export/read-only probes) to a Haiku-class model, the strong model RESERVED for all reasoning/verdicts/fix-design (§11.4.102)/code-review (§11.4.125)/demotion (§11.4.7), large output persisted to a file not an inline 350–520K-token transcript; the cheap model never emits a PASS so §11.4.50 + anti-bluff are untouched; (3) **thin always-loaded INDEX + on-demand detail** — concise index (one line per fix/anchor, EACH carrying the literal `11.4.N` token so propagation gates pass) with the canonical full text kept gate-scanned in `constitution/Constitution.md` and reachable in one hop — a de-duplication realising §11.4.35, never a deletion; (4) **CodeGraph/retrieval-first over full-file loading** (§11.4.78/§11.4.79); (5) **output-token reduction** — terse status + `effort:"low"` on the mechanical allowlist only; (6) **tool-call batching + no re-reads**; (7) **compaction/context-editing for long sessions**. **Mandatory measured proof:** a token-accounting harness measures tokens-per-development-cycle BEFORE vs AFTER on a frozen deterministic workload from the authoritative `usage` object (input/cache_read/cache_creation/output split; NEVER `tiktoken`, NEVER the client-side cost estimate), reproduced N times (§11.4.50), pass = AFTER ≤ 40% of BEFORE OR the measured best-safe reduction with a cited cold-cache reason; the AFTER run MUST show ZERO regression on the pre-build sweep + meta-test mutation sweep + propagation gates + a strong-model reasoning probe + a cache-warm proof (`cache_read_input_tokens > 0`) — cost reduction with quality regression is a §11.4 FAIL. The headline number is the *measured* reduction, never the design estimate (§11.4.6/§11.4.123). No measure may break/degrade any existing mechanism, and the rule is structured so none can. Composes

§11.4.5/.6/.20/.40/.50/.58/.69/.70/.78/.79/.80/.103/.106/.123/.125/.128/§12.6/§1.1. Classification: universal (§11.4.17). Propagation gate `CM-COVENANT-114-141-PROPAGATION` (literal `11.4.141`) + recommended gate `CM-TOKEN-EFFICIENCY` + paired §1.1 mutation (inject a pre-breakpoint volatile token → cache collapses → measured reduction falls below bar → gate FAILs). **Canonical authority:** constitution submodule `Constitution.md` §11.4.141. Non-compliance is a release blocker. No escape hatch — no `--skip-token-efficiency`, `--no-cache-governance`, `--assert-reduction-without-measuring`, `--tier-down-reasoning`, `--inline-all-governance`, `--tiktoken-estimate-OK` flag.